TRABALHO DE CONCLUSÃO DE CURSO

Colete Ortopédico Inteligente para Monitoramento Postural em Tempo Real

Documentação do software

Firmware do ESP32, app web embarcado, simulador e testes do colete postural (BioSpine)

Gerado em 21/10/2026.
Fonte: os arquivos `.md` da pasta `Documentação/` do repositório; este PDF é gerado a partir deles por `python3 Documentação/gerar_pdf.py`. Para corrigir algo, edite o `.md` e gere de novo.

Sumário

Documentação — Colete Postural (BioSpine) 5

Visão em uma figura 5
Por onde começar 5
Capítulos 5
Referência por módulo 6
PDF 6
Figuras para o texto do TCC 7
Glossário 7
Situação 7

01 — Visão geral do sistema 8

1. O problema e a proposta 8
2. O que o sistema faz 8
3. Hardware 9
4. Plataforma de software 11
5. Estrutura do repositório 11
6. Uso em resumo 12

02 — Arquitetura do firmware 13

1. Ideia geral 13
2. Camadas e módulos 13
3. Duas tarefas 15
4. Inicialização 17
5. Ciclo de controle e fluxo de dados 18
6. Limitações conhecidas da arquitetura 21

03 — Medição da inclinação do tronco 22

1. O que precisa ser medido 22
2. O sensor MPU-6050 22
3. Offset do giroscópio 23
4. Por que não usar ângulos de Euler (pitch e roll) 24
5. Filtro complementar sobre o vetor gravidade 25
6. Desvio 26
7. Frente/trás e lateral (só para exibir) 26
8. Evidências (testes) 26
9. Limitações 27

04 — Detecção de má postura e calibração 28

1. Parâmetros 28
2. Regra de decisão 28
3. Calibração 30
4. Quando um flex entra na decisão 32
5. Evidências (testes) 32

05 — Sensores analógicos: flex e FSR 33

1. Papel de cada sensor 33
2. Circuito e leitura 33
3. Do pino à decisão 33
4. O "fantasma" do ADC e o flex 2 35
5. FSR 36
6. Mensagens de diagnóstico 36
7. Evidências (testes) 36

06 — Motor vibratório e memória não volátil (NVS) 37

1. Circuito do motor 37
2. PWM 37
3. Padrões de vibração 37
4. Lógica do motor 38
5. Memória não volátil (NVS) 39

07 — Protocolo e interfaces 40
1. Rede e HTTP 42
2. Protocolo WebSocket (JSON) 42
3. Serial (115200 baud) 46
4. Compatibilidade com o BioSpine V3 48

08 — App web embarcado 50
1. Objetivo 50
2. Como a página chega ao celular 51
3. Cara de app (PWA) 51
4. Estrutura da tela 52
5. Estado e fluxo de dados 52
6. Conexão e reconexão 54
7. Leitura principal 54
8. Lista de sensores 55
9. Gráfico dos últimos 30 s 55
10. Gravação em CSV 55
11. Vista 3D 56
12. Ajustes 58
13. Tema, acessibilidade e armazenamento local 59
14. Testar sem a placa 59
15. Funções principais do script 60
16. Armadilhas e cuidados 60

09 — Simulador, testes e ferramentas 62
1. Por que existe 62
2. Testes unitários (test/test_posture/test_main.cpp) 63
3. Simulador do firmware (sim/ + tools/sim.sh) 65
4. Página web sem placa: sim_web.py × mock_server.py 68
5. Gráfico para o TCC: tools/grafico_serial.py 70
6. Imagem para gravar: tools/gerar_bin.sh 71
7. platformio.ini 71
8. Como estender 72
9. Limitações (o que nada disso prova) 72
10. Particularidades conhecidas 72

10 — Decisões de projeto 73
D-01 — O colete funciona sozinho; o app é opcional 73
D-02 — App web embarcado: Access Point + HTTP + WebSocket 73
D-03 — WebServer do core + WebSocketsServer (links2004) 73
D-04 — Driver próprio do MPU-6050, sem a biblioteca MPU6050_light 73
D-05 — Desvio = ângulo entre vetores gravidade; a calibração grava g0 74
D-06 — Filtro complementar sobre o vetor gravidade, com rejeição de aceleração 74
D-07 — Offset do giroscópio no boot e recalculado sozinho quando parado 74
D-08 — Detecção com limite, histerese e tempo mínimo (ok → atenção → alerta) 74
D-09 — Sem calibração, o colete não vibra 75
D-10 — Calibração com espera, média e recusa se mexer; retorno pela vibração 75
D-11 — Sensores analógicos opcionais e detectados sozinhos; flex 2 desligado por padrão; FSR só telemetria 75
D-12 — Duas tarefas FreeRTOS: controle (prioridade 3) e loop (Wi-Fi e serial) 75
D-13 — Agendamento por millis(), sem delay() 75

D-14 — Parâmetros ajustáveis sem recompilar, validados no firmware e salvos na NVS 76
D-15 — Motor por PWM a 20 kHz, alerta em pulsos de 300/700 ms, LED junto 76
D-16 — Wi-Fi com potência reduzida (11 dBm), sem economia de energia, heartbeat no WebSocket 76
D-17 — I2C a 100 kHz, dois endereços e reconexão automática do MPU 76
D-18 — Um firmware para as duas montagens (GUIA e BioSpine V3) 76
D-19 — Lógica pura separada do hardware; testes no PC e simulador do firmware inteiro 76
D-20 — Protocolo JSON num módulo separado do Wi-Fi 77

11 — Mapa entre o texto do TCC e o código 78
1. Afirmações do texto e onde estão no código 78
2. Correções sugeridas no texto 78
3. O que escrever sobre o software (roteiro sugerido) 79
4. Perguntas prováveis da banca 79

Módulo config 81

Módulo main (núcleo e API app_*) 85

Módulo tilt 92

Módulo posture 96

Módulo filters 101

Módulo mpu 104

Módulo sensors 109

Módulo motor 114

Módulo settings 118

Módulo console 121

Módulo protocolo 124

Módulo web 126

Manutenção da documentação 129

Documentação — Colete Postural (BioSpine)

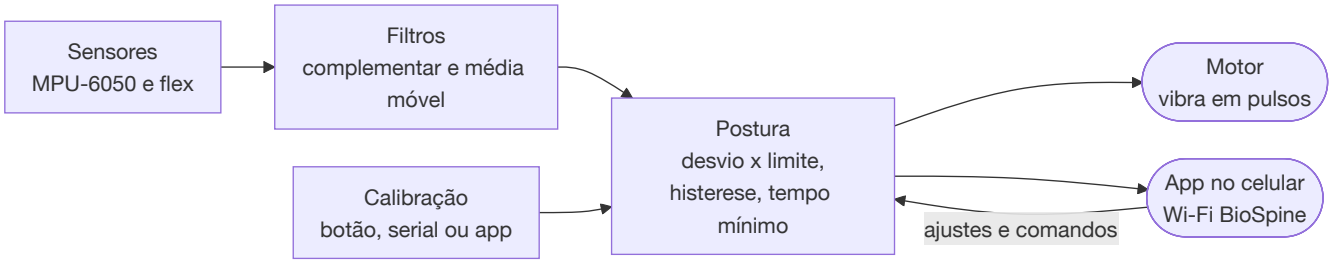
> Código: todo o repositório · Relacionados: `../README.md` (comandos do dia a dia), `../VALIDACAO.md` (teste no hardware), `montagem.pdf` (montagem; fora do repositório, na pasta acima)

Documentação do software do TCC "Colete Ortopédico Inteligente para Monitoramento Postural em Tempo Real": o que o sistema faz, como foi pensado, como cada parte funciona e por que foi feita assim.

Para ler, imprimir ou entregar: os PDFs em `pdf/`. O `00-documentacao-completa.pdf` tem capa, sumário com páginas e todos os documentos; há também um PDF por documento. Os `.md` desta pasta são a fonte (é neles que se edita) e os PDFs são gerados a partir deles.

Visão em uma figura

O colete lê a inclinação do tronco e a curvatura da coluna, compara com a postura calibrada e vibra quando a postura ruim dura mais que o tempo configurado. O celular é opcional.



Por onde começar

Quem	Caminho sugerido
Quem vai escrever ou defender o TCC	01 → 02 → 03 → 04 → 11 (correções do texto, figuras e perguntas da banca)
Quem vai mexer no código	02 → 10 → página do módulo em referencia/ → MANUTENCAO.md
Agente de IA	MANUTENCAO.md primeiro (mapa arquivo → documento e regras), depois o documento do assunto

Capítulos

#	Documento	Assunto
01	Visão geral	Problema, requisitos, hardware, pinagem, plataforma, estrutura do repositório
02	Arquitetura	Módulos, duas tarefas FreeRTOS, comunicação entre elas, boot, ciclo de controle, fluxo de dados
03	Inclinação	MPU-6050, offset do giroscópio, filtro complementar sobre o vetor gravidade, desvio, frente/lateral
04	Detecção e calibração	Parâmetros, limite + histerese + tempo mínimo, máquina de estados, calibração
05	Sensores analógicos	Flex (divisor, média móvel, detecção, curvatura), "fantasma" do ADC, FSR
06	Motor e NVS	Circuito e PWM do motor, padrões de vibração, chaves salvas na memória
07	Protocolo e interfaces	Wi-Fi, HTTP, JSON do WebSocket, comandos da serial, `diag`, compatibilidade BioSpine V3
08	App web	Página embarcada: estrutura, fluxo de dados, gráfico, CSV, vista 3D, ajustes

#	Documento	Assunto
09	Simulador, testes e ferramentas	21 testes, simulador do firmware, roteiros, ponte web, gráfico do TCC, geração do `.bin`
10	Decisões de projeto	D-01 a D-20: o que foi decidido, por quê e onde está no código
11	Mapa TCC ↔ código	Afirmações do texto, correções, roteiro do capítulo de software, figuras, perguntas da banca
—	Manutenção	Como manter esta documentação junto com o código e gerar os PDFs

Referência por módulo

Uma página por módulo de `src/`, com cada tipo, constante e função (o que faz, parâmetros, retorno, quem chama).

Módulo	Página	Assunto
`config.h`	config	Todas as constantes: pinos, tempos, limites, padrões
`main.cpp`, `app.h`	main	`setup`, `loop`, tarefa de controle, trava, pedidos, eventos, calibração, botão
`tilt`	tilt	Vetores, `TiltFilter`, `angleBetweenDeg`, `tiltComponentsDeg`
`posture`	posture	`Params`, `DetectorPostura`, `Calibrador`, `flexPct`
`filters.h`	filters	`MediaMovel`, `Presenca`, `leituraAdcValida`
`mpu`	mpu	Driver do MPU-6050
`sensors`	sensors	Agenda das leituras, `Leituras`, detecção dos canais
`motor`	motor	PWM, alerta, sequências de pulsos
`settings`	settings	NVS, `Config`, `Calibracao`, modos dos sensores
`console`	console	Interpretador da serial, `diag`, plotter, BioSpine
`protocolo`	protocolo	Mensagens JSON do app
`web`	web	Access Point, HTTP, WebSocket

PDF

Arquivo	Conteúdo
`pdf/00-documentacao-completa.pdf`	Capa, sumário com páginas e todos os documentos, na ordem desta página; os links entre documentos funcionam dentro do PDF
`pdf/<documento>.pdf`	Um por `.md`, no mesmo caminho (`referencia/mpu.md` → `pdf/referencia/mpu.pdf`); link para outro documento aparece como texto

Depois de editar qualquer `.md`, gere de novo e commite a pasta `pdf/` junto:

```
python3 Documentação/gerar_pdf.py              # todos os PDFs (~20 s)
python3 Documentação/gerar_pdf.py --completo   # só o completo
python3 Documentação/gerar_pdf.py 05-sensores-analogicos.md   # só esse PDF separado (não refaz o completo)
```

Precisa de Node.js e de um Chrome/Chromium instalado (outro caminho: `CHROME=/caminho/do/navegador`). Na primeira vez instala as dependências em `ferramentas/node_modules` (fora do git; versões fixas no `package-lock.json`). Como funciona: `ferramentas/gerar_pdf.mjs` converte cada `.md` em HTML (markdown-it), desenha as fórmulas (KaTeX) e os diagramas (mermaid.js) e o Chrome imprime em A4. Diagramas largos demais saem numa página deitada.

Figuras para o texto do TCC

Todos os diagramas são **Mermaid** escritos dentro dos `.md`: o GitHub e o VS Code (com extensão de Mermaid) desenham direto, e qualquer pessoa ou agente edita como texto, junto com o código.

Para colar no Word, gere as imagens:

```
python3 Documentação/gerar_figuras.py           # PNG (fundo branco, 2x) e SVG de todos os
diagramas
python3 Documentação/gerar_figuras.py 04-deteccao-e-calibracao.md   # só de um documento
```

As imagens saem em `Documentação/figuras/<documento>-<n>.png` (`n` = ordem do diagrama no documento). Essa pasta fica fora do git (`.gitignore`): a fonte é o texto Mermaid, e as imagens se geram de novo a qualquer momento. Precisa de Node.js; usa o Chrome instalado. Alternativa sem instalar nada: colar o bloco em https://mermaid.live e exportar. A lista de figuras sugeridas para o texto está em 11.

Glossário

Termo	Significado
ADC	Conversor analógico-digital: transforma a tensão do pino em número (0–4095 com 12 bits)
Access Point (AP)	Modo em que o próprio ESP32 cria a rede Wi-Fi, sem roteador
Calibração	Gravação da postura correta da pessoa (vetor gravidade e leituras do flex)
Desvio	Ângulo entre a orientação atual do tronco e a da calibração
Filtro complementar	Combinação do giroscópio (curto prazo) com o acelerômetro (longo prazo)
FreeRTOS	Sistema operacional de tempo real do ESP32; permite tarefas com prioridades
FSR	Sensor resistivo de força (pressão)
Histerese	Margem entre o valor que liga e o que desliga o alerta, para não oscilar
I2C	Barramento de dois fios (SDA, SCL) usado pelo MPU-6050
IMU	Unidade de medição inercial: acelerômetro + giroscópio
LEDC	Periférico de PWM do ESP32
LSB	Menor passo da leitura digital; "131 LSB/°/s" = 131 unidades por grau por segundo
NVS	Memória não volátil do ESP32, guardada na flash (o ESP32 não tem EEPROM)
PWM	Modulação por largura de pulso: liga e desliga rápido para controlar a potência
Serial Plotter	Gráfico em tempo real da saída serial (Arduino IDE ou `tools/grafico_serial.py`)
WebSocket	Conexão contínua de mão dupla entre a página e o colete

Situação

Tudo foi desenvolvido e testado **sem a placa** (testes nativos, simulador, página contra o firmware simulado). O que só se confirma no hardware (driver do MPU, sinal do giroscópio, ADC real, Wi-Fi no celular, concorrência das duas tarefas, quedas de tensão) está listado e roteirizado no `VALIDACAO.md`. Registre os números medidos lá antes de citá-los no texto do TCC.

01 — Visão geral do sistema

> Código: `src/config.h`, `platformio.ini` · Relacionados: 02 — Arquitetura, 10 — Decisões, guia de montagem `montagem.pdf` (na pasta acima do repositório)

1. O problema e a proposta

O TCC "Colete Ortopédico Inteligente para Monitoramento Postural em Tempo Real" propõe um colete que percebe quando o tronco sai da postura correta e avisa o usuário com vibração. A má postura prolongada (celular, computador, horas sentado) causa dor e desvios na coluna; o aviso na hora em que acontece ajuda a criar consciência corporal (reeducação postural).

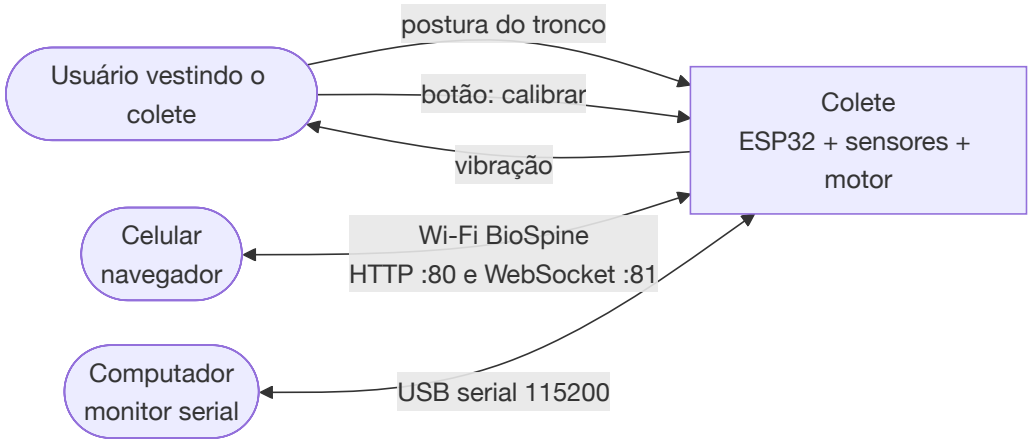
Este repositório contém o **software** do colete:

- **Firmware** do ESP32 (C++ com framework Arduino, compilado pelo PlatformIO): lê os sensores, decide se a postura está ruim e aciona o motor vibratório.
- **App web embarcado**: uma página servida pelo próprio ESP32, aberta no celular pela rede Wi-Fi do colete, que mostra a postura em tempo real e permite calibrar e ajustar o colete.
- **Ferramentas de desenvolvimento**: testes automáticos da lógica, simulador do firmware no PC e scripts para gerar o gráfico dos resultados.

Princípio central: **o colete funciona sozinho**. Sem celular e sem computador ele continua detectando e vibrando. O app só mostra dados e envia comandos (D-01).

2. O que o sistema faz

O diagrama mostra quem interage com o colete e por qual meio.



Requisitos funcionais

ID	Requisito	Onde está
RF-01	Medir a inclinação do tronco com o MPU-6050 (acelerômetro + giroscópio)	`src/mpu.cpp`, `src/sensors.cpp`, `src/tilt.cpp` — 03
RF-02	Medir a curvatura da coluna com sensor flex (1 ou 2)	`src/sensors.cpp`, `src/filters.h` — 05
RF-03	Calibrar a postura correta da pessoa (botão, serial ou app)	`src/main.cpp`, `src/posture.cpp` — 04
RF-04	Detectar má postura com limite, histerese e tempo mínimo	`src/posture.cpp` — 04

ID	Requisito	Onde está
RF-05	Avisar com o motor vibratório em pulsos	`src/motor.cpp` — 06
RF-06	Ajustar parâmetros sem recompilar e guardá-los depois de desligar	`src/settings.cpp`, `src/console.cpp`, `src/protocolo.cpp` — 06, 07
RF-07	Mostrar a postura em tempo real no celular, com gráfico e vista 3D	`src/web.cpp`, `src/web/index.html` — 08
RF-08	Testar cada componente do hardware (`diag`)	`src/console.cpp` — 07
RF-09	Exportar dados para a figura de resultados do TCC (Serial Plotter, CSV, PNG)	`src/console.cpp`, `tools/grafico_serial.py`, botão "Baixar CSV" do app — 09

Requisitos não funcionais

ID	Requisito	Como é atendido
RNF-01	Funcionar sem celular, sem computador e sem internet	Decisão e alerta rodam no ESP32; Wi-Fi em modo Access Point próprio
RNF-02	Não parar de detectar quando o Wi-Fi trava	Tarefa de controle separada, com prioridade maior que a do `loop()`, onde rodam HTTP, WebSocket e serial (02)
RNF-03	Tolerar montagem incompleta ou com defeito	Detecção automática de MPU, flex, flex 2 e FSR; reconexão do MPU; fallback só com o MPU (05)
RNF-04	Não incomodar à toa	Tempo mínimo de 3 s, histerese, vibração em pulsos, PWM fora da faixa audível
RNF-05	Ser testável sem a placa	Lógica pura separada, testes nativos e simulador (09)
RNF-06	Baixo custo	Componentes comuns (ESP32 DevKit, GY-521, flex 2.2"), sem nuvem nem app pago

Fora do escopo (versão 2)

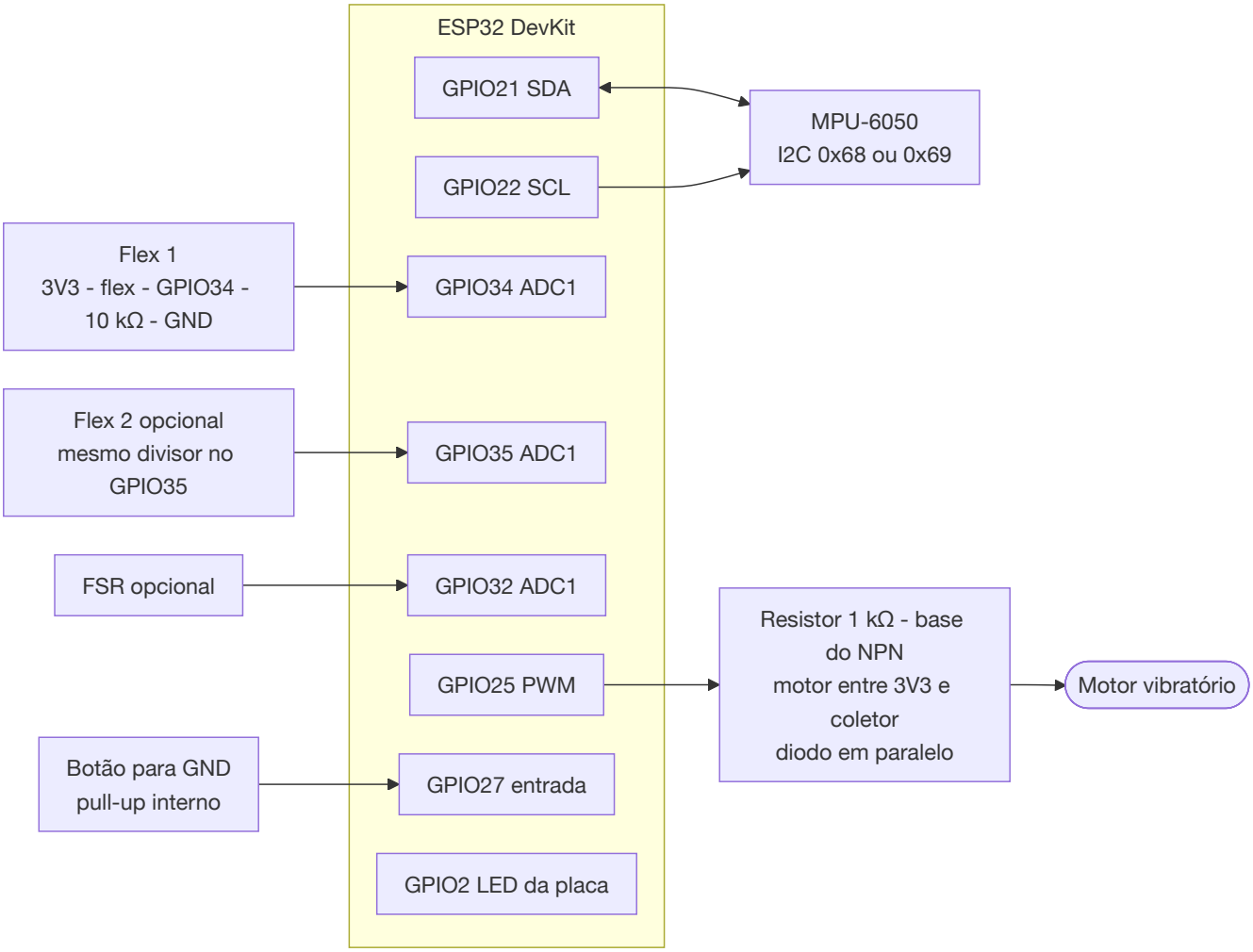
App nativo (Android/iOS) e Bluetooth, nuvem (banco de dados, MQTT), bateria LiPo com circuito de carga (a alimentação é por power bank).

3. Hardware

Componente	Função	Observação
ESP32 DevKit 30 pinos (DOIT, ESP32-WROOM-32)	Microcontrolador: lê sensores, decide, aciona o motor, Wi-Fi	240 MHz, 520 kB de SRAM, 4 MB de flash, ADC de 12 bits
MPU-6050 (módulo GY-521)	IMU: acelerômetro e giroscópio de 3 eixos, via I2C	Configurado em ±2 g e ±250 °/s
Sensor flex 2.2"	Curvatura da coluna (resistência sobe ao dobrar)	~10 kΩ reto, tolerância ±30%
2º sensor flex (opcional)	Curvatura do outro lado	Desligado por padrão
FSR (opcional, montagem BioSpine)	Pressão	Só telemetria, não alerta
Motor vibratório tipo moeda	Alerta	3 V, 60 mA
Transistor NPN (2N2222 / BC337) + resistor 1 kΩ	Aciona o motor	O GPIO não aguenta 60 mA
Diodo 1N4148 / 1N4007	Proteção contra o pico de tensão do motor (flyback)	Em paralelo com o motor
Resistor 10 kΩ	Divisor de tensão do flex	

Componente	Função	Observação
Botão táctil	Calibração	Para GND, com pull-up interno
Capacitor 100 µF (opcional)	Evita reset do ESP32 quando o motor liga	
Power bank + cabo USB de dados	Alimentação	

O diagrama mostra as ligações elétricas entre o ESP32 e os periféricos (detalhe físico no `montagem.pdf`).



Pinagem

O firmware atende **duas montagens ao mesmo tempo** (a do `montagem.pdf`/GUIA e a do README do BioSpine V3), porque nenhum pino conflita (D-18). O que não estiver ligado é detectado e ignorado.

Função	Pino	Constante (`src/config.h`)	montagem.pdf	BioSpine V3
MPU-6050 SDA / SCL	21 / 22	`PIN_I2C_SDA`, `PIN_I2C_SCL`	sim	sim
Flex 1	34	`PIN_FLEX`	sim	flex esquerdo
Flex 2	35	`PIN_FLEX2`	reserva	flex direito
FSR (pressão)	32	`PIN_FSR`	—	sim
Motor (via NPN)	25	`PIN_MOTOR`	sim	—
Botão de calibração	27	`PIN_BUTTON`	sim	—
LED azul da placa	2	`PIN_LED`	—	sim (acende junto com o motor)

Regras que justificam a escolha dos pinos:

- **Todo sensor analógico no ADC1** (GPIO 32–39). O ADC2 não funciona com o Wi-Fi ligado.
- GPIO34 e GPIO35 são só entrada: servem para sensor, nunca para saída.
- Saídas longe dos pinos de boot (0, 2, 12, 15). O GPIO2 só acende o LED da placa, que já está ligado nele.
- MPU alimentado em 3V3 (níveis de I2C seguros; o GY-521 já tem pull-up).

Posição no colete

- **MPU-6050**: centro das costas, entre as escápulas, sobre a coluna, bem preso. Componentes para fora e pinos para baixo (o eixo Y do sensor aponta para cima). A calibração compensa pequenos desalinhamentos.
- **Flex**: na vertical, ao longo da coluna torácica, onde ela mais curva quando a pessoa "corcunda".
- **Motor**: encostado no corpo, perto do ombro ou da nuca.
- **ESP32 e power bank**: num bolso na parte de baixo do colete.

4. Plataforma de software

Item	Versão / valor	Onde
Plataforma PlatformIO	`espressif32@6.9.0` (core Arduino-ESP32 2.0.17)	`platformio.ini`
Placa	`esp32dev`	`platformio.ini`
WebSocket	`links2004/WebSockets@2.7.3`	`platformio.ini`
JSON	`bblanchon/ArduinoJson@7.2.1`	`platformio.ini`
Servidor HTTP	`WebServer` (já vem no core)	`src/web.cpp`
Página do app	HTML/CSS/JS num arquivo só, embutido na flash (`board_build.embed_txtfiles`)	`src/web/index.html`
Testes	Unity, ambiente `native` (roda no PC)	`test/`, `platformio.ini`
Biblioteca do MPU	**nenhuma**: driver próprio por registradores (D-04)	`src/mpu.cpp`

5. Estrutura do repositório

```
colete-postural/
├── platformio.ini        ambientes esp32dev (placa) e native (testes no PC)
├── README.md             comandos do dia a dia e resumo
├── VALIDACAO.md          roteiro de teste no hardware
├── AGENTS.md             instruções para agentes de IA (aponta para Documentação/MANUTENCAO.md)
├── Documentação/         esta documentação (.md = fonte; pdf/ = PDFs gerados por gerar_pdf.py)
├── src/
│   ├── main.cpp          setup/loop, tarefa de controle, calibração, botão, fila de eventos
│   ├── app.h             Status e funções app_* (ponte entre loop e tarefa de controle)
│   ├── config.h          pinos, tempos e valores padrão
│   ├── tilt.*            vetor gravidade, filtro complementar, ângulos (lógica pura)
│   ├── posture.*         limite, histerese, tempo mínimo, calibrador (lógica pura)
│   ├── filters.h         média móvel e detecção de sensor presente (lógica pura)
│   ├── mpu.*             driver do MPU-6050
│   ├── sensors.*         leitura de MPU, flex, flex 2 e FSR
│   ├── motor.*           PWM do motor, padrões de pulso, LED
│   ├── settings.*        parâmetros e calibração na NVS
│   ├── console.*         comandos pela serial
│   ├── protocolo.*       JSON do app (roda também no simulador)
│   ├── web.*             Wi-Fi AP, HTTP, WebSocket
│   └── web/              index.html, manifest.json, icon.svg (vão para a flash)
├── test/test_posture/    21 testes da lógica pura
├── sim/                  imitação de Arduino, Wire e Preferences para rodar o firmware no PC
├── tools/                simulador, roteiros, ponte web, gráfico, geração do .bin
└── dist/                 firmware pronto para gravar sem compilar
```

6. Uso em resumo

1. Ligar o colete (power bank). Um pulso curto do motor indica que ligou.
2. Vestir, ficar na postura correta e **apertar o botão** (ou "Calibrar postura" no app). Um pulso: "fique parado". Cerca de 2,5 s depois, dois pulsos: calibrado. Um pulso longo: mexeu durante a média, repetir.
3. Pronto: se a postura passar do limite por mais de 3 s, o motor vibra em pulsos até a postura voltar.
4. Opcional: no celular, conectar no Wi-Fi `BioSpine` (senha `colete123`) e abrir `http://192.168.4.1`.

Sem calibração o colete **não vibra** (D-09). A calibração fica salva. Comandos de compilação, gravação e teste: `README.md` do repositório e 09.

02 — Arquitetura do firmware

> Código: `src/main.cpp`, `src/app.h`, todos os módulos de `src/` · Relacionados: referencia/main.md, 10 — Decisões

1. Ideia geral

O firmware é um **pipeline** que se repete o tempo todo:

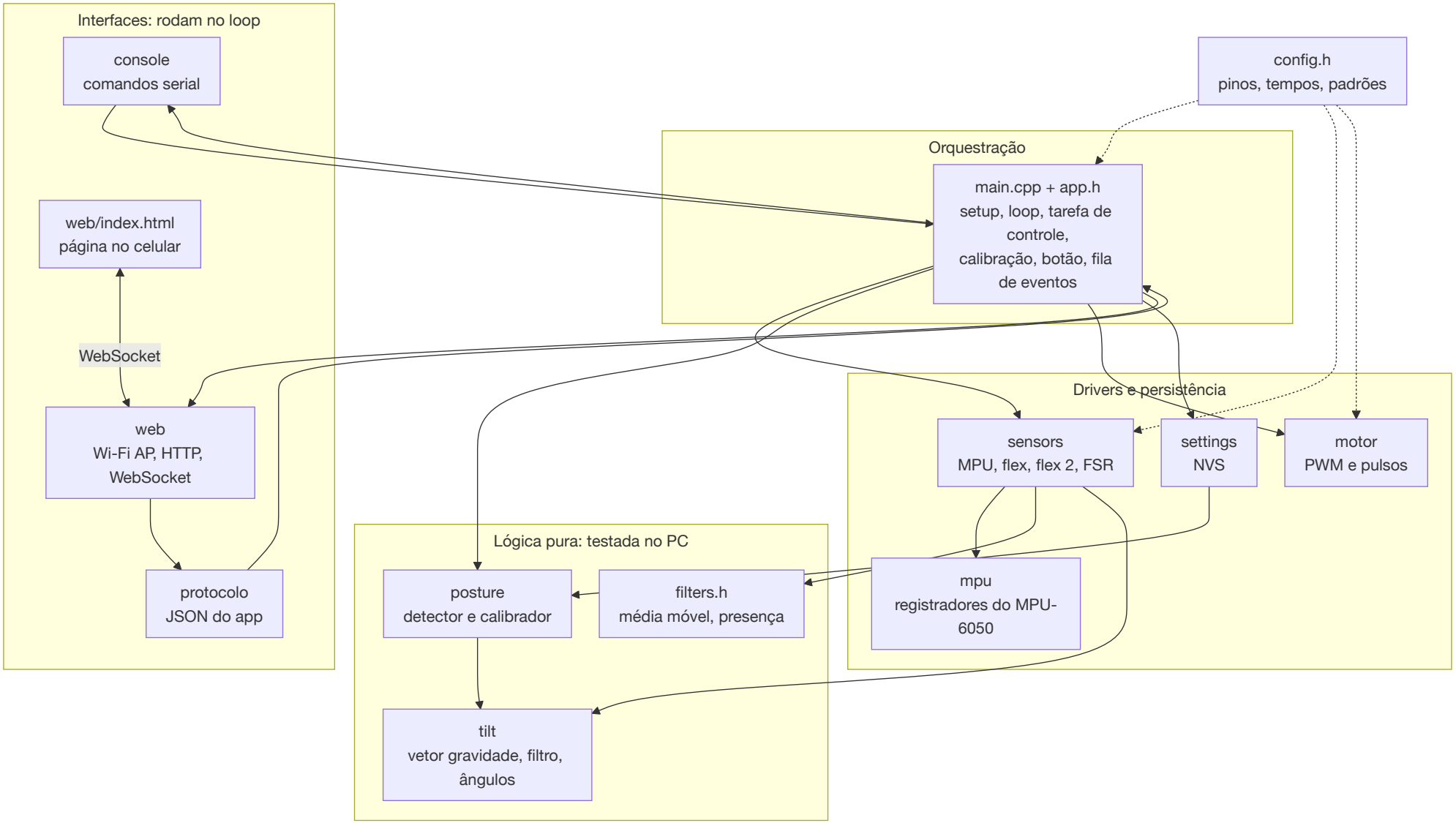
```
sensores → filtro → postura → motor
```

Em volta dele ficam as **interfaces** (serial e app web), que só leem o estado e fazem pedidos. Duas escolhas organizam todo o código:

1. **Lógica pura separada do hardware.** O cálculo de inclinação (`tilt`), a decisão de postura (`posture`) e os filtros dos sensores (`filters`) não usam nada do Arduino. Por isso rodam e são testados no PC (09).
2. **Duas tarefas.** Uma tarefa de controle, de prioridade maior, faz o pipeline; o `loop()` do Arduino cuida do Wi-Fi e da serial. Se o Wi-Fi travar, o colete continua vibrando (D-12).

2. Camadas e módulos

Cada caixa é um módulo de `src/` (par `.h`/`.cpp`, ou só `.h`). A seta `A --> B` quer dizer "A usa B".


Interfaces: rodam no loop
console
comandos serial
web/index.html
página no celular
WebSocket
web
Wi-Fi AP, HTTP,
WebSocket
protocolo
JSON do app
config.h
pinos, tempos, padrões
Orquestração
main.cpp + app.h
setup, loop, tarefa de
controle,
calibração, botão, fila
de eventos
Drivers e persistência
sensors
MPU, flex, flex 2, FSR
settings
NVS
motor
PWM e pulsos
mpu
registradores do MPU-
6050
Lógica pura: testada no PC
posture
detector e calibrador
filters.h
média móvel, presença
tilt
vetor gravidade, filtro,
ângulos

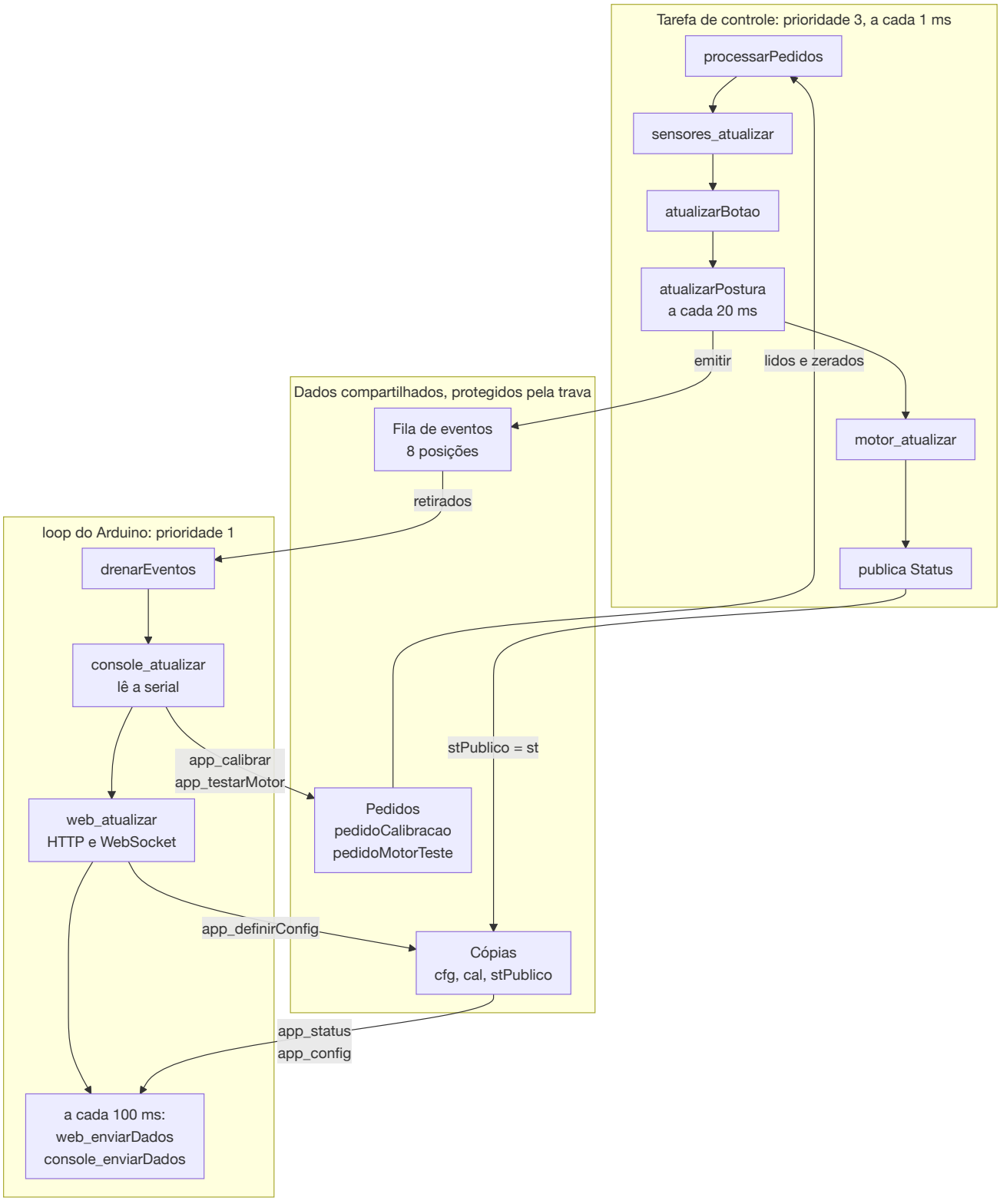
Módulo	Papel	Roda em	Referência
`main.cpp`, `app.h`	Liga tudo: `setup`, `loop`, tarefa de controle, calibração, botão, estado compartilhado	as duas tarefas	referencia/main.md
`config.h`	Pinos, períodos, limites e valores padrão em um lugar só	—	referencia/config.md
`tilt`	Álgebra de vetores, filtro complementar sobre a gravidade, desvio, frente/lateral	controle, PC	referencia/tilt.md
`posture`	Parâmetros, detector de má postura (limite, histerese, tempo mínimo), calibrador	controle, PC	referencia/posture.md
`filters.h`	Média móvel, presença com persistência, validade da leitura do ADC	controle, PC	referencia/filters.md
`mpu`	Driver do MPU-6050 por registradores, offset do giroscópio	controle	referencia/mpu.md
`sensors`	Agenda e lê MPU (200 Hz) e ADCs (100 Hz); detecta sensores ligados	controle	referencia/sensors.md
`motor`	PWM do motor, alerta em pulsos, sequências de confirmação, LED	controle	referencia/motor.md
`settings`	Lê e grava parâmetros e calibração na NVS	as duas	referencia/settings.md
`console`	Comandos da serial, `diag`, saída do plotter, compatibilidade BioSpine	loop	referencia/console.md
`protocolo`	Monta e interpreta o JSON do app; independente do Wi-Fi	loop	referencia/protocolo.md
`web`	Access Point, servidor HTTP (página), servidor WebSocket	loop	referencia/web.md

3. Duas tarefas

O ESP32 roda FreeRTOS. O core Arduino cria uma tarefa para o `loop()` (prioridade 1, núcleo 1). O `setup()` cria mais uma, a **tarefa de controle** (`tarefaControle`, prioridade 3, núcleo 1, pilha de 8 kB), que chama `passoControle(millis())` e dorme 1 ms (`vTaskDelay(1)`), para sempre.

Por quê: o envio pelo Wi-Fi pode bloquear o `loop()` por segundos. Quando o celular sai do alcance, o envio TCP do WebSocket fica esperando o buffer esvaziar. Com tudo numa linha só, o colete pararia de detectar nesse tempo. Separado assim, a tarefa de controle sempre ganha a CPU (prioridade maior) e o alerta não depende do Wi-Fi.

O diagrama (simplificado: cada seta mostra um exemplo de chamada) mostra o que cada tarefa faz e os três canais protegidos entre elas. As exceções sem trava estão na seção 6.



Regras de convivência

Dado	Escrito por	Lido por	Como
`cfg` (Config)	loop (`app_definirConfig`)	controle (a cada passo) e loop	cópia sob a trava
`cal` (Calibracao)	controle (`concluirCalibracao`)	controle e loop	cópia sob a trava
`stPublico` (Status)	controle (fim de cada passo)	loop (`app_status`)	cópia sob a trava

Dado	Escrito por	Lido por	Como
`pedidoCalibracao`, `pedidoMotorTeste`	loop	controle	lidos e zerados sob a trava
fila de eventos	controle (`emitir`)	loop (`drenarEventos`)	anel de 8 posições sob a trava

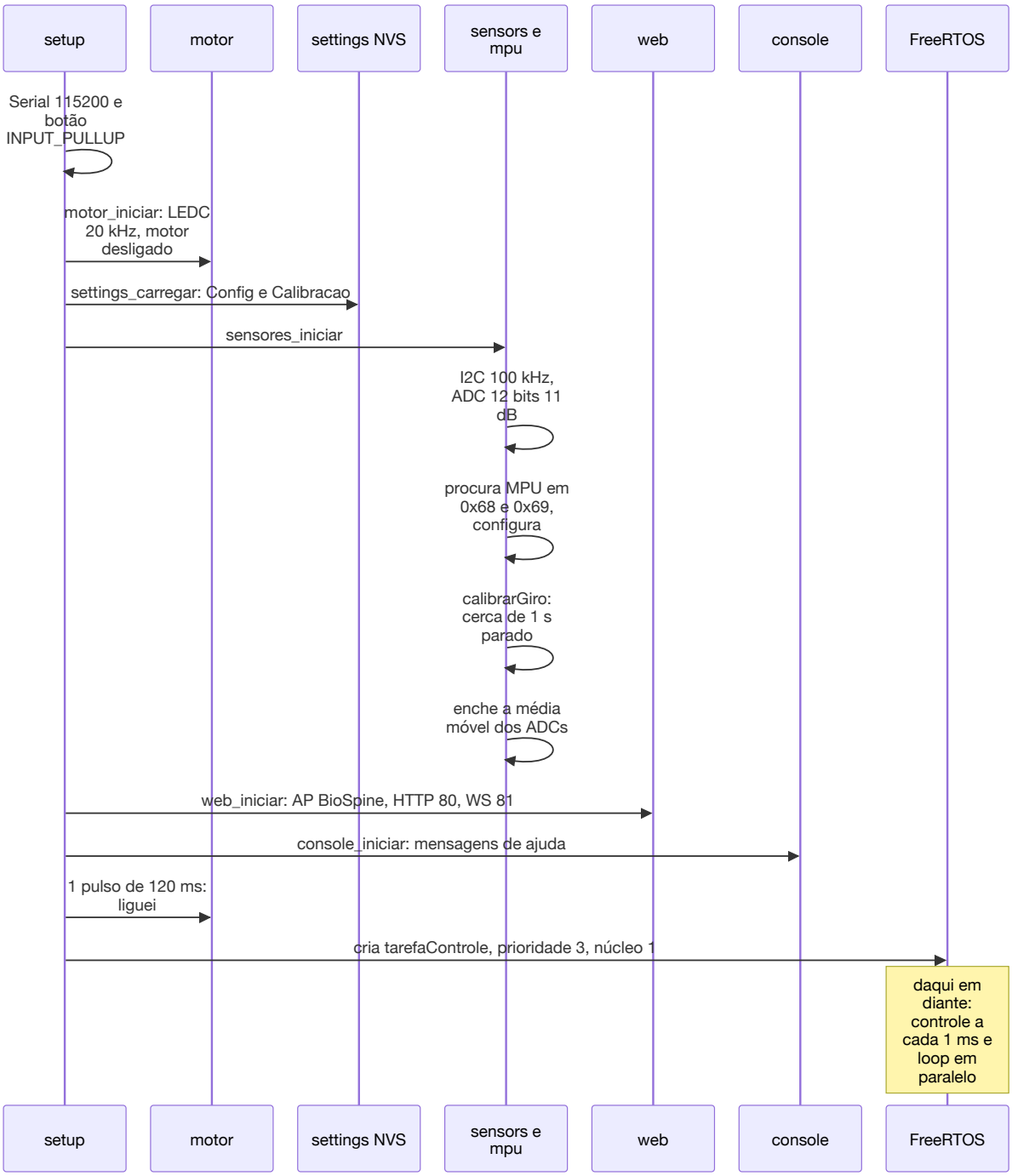
- A trava é um `portMUX` (seção crítica do FreeRTOS). Dentro dela só há **cópias de structs pequenas**: nada de serial, Wi-Fi ou NVS, porque a seção crítica desliga as interrupções do núcleo.
- **A tarefa de controle nunca chama `Serial` nem `web_*`.** Quando precisa avisar algo, chama `emitir()`, que põe um evento na fila. O loop retira e entrega para a serial (`console_evento`) e para o app (`web_evento`). Se a fila estiver cheia (8 eventos sem o loop retirar), o evento novo é descartado: o colete continua funcionando, só a mensagem se perde.
- A tarefa de controle trabalha numa cópia local do status (`st`) e só publica no fim do passo. O loop nunca vê um status pela metade.
- O **estado** de `sensors`, `motor`, `tilt`, `posture` e `filters` (filtro, detector, calibrador, médias, fases do motor) é usado só pela tarefa de controle (e pelo `setup()`, antes de a tarefa existir), por isso não tem trava própria. O loop também chama funções **sem estado** de `posture` (`validarParams`, `estadoNome`, `paramsPadrao`), o que é seguro. As leituras de `sensors` feitas pelo loop estão na seção 6.

Modo simulador

Com `-DSIMULADOR` (compilação para o PC, `tools/sim.sh`), não há FreeRTOS: a trava vira vazia e o próprio `loop()` chama `passoControle()` antes do resto. Todo o resto do código é o mesmo que roda na placa (09).

4. Inicialização

O diagrama mostra a ordem do `setup()`. O motor é configurado primeiro, então o pino do motor já fica em nível baixo (motor desligado) durante a espera de ~1 s do giroscópio. A configuração e a calibração salvas são carregadas antes do `console_iniciar`, que já usa a calibração para o lembrete "Calibração pendente".



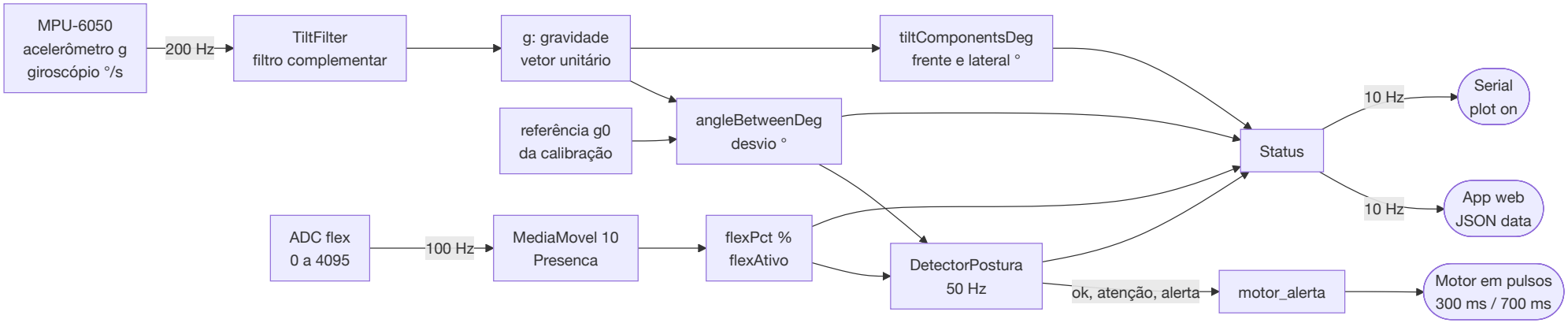
5. Ciclo de controle e fluxo de dados

`passoControle` roda a cada ~1 ms, mas cada etapa tem o seu período, controlado por `millis()` (sem `delay`):

Etapa	Período	Constante	Onde
Passo da tarefa de controle	~1 ms	`vTaskDelay(1)`	`tarefaControle` em `main.cpp`
Leitura do MPU + filtro	5 ms (200 Hz)	`PERIODO_MPU_MS`	`sensores_atualizar`
Leitura dos ADCs + média + presença	10 ms (100 Hz)	`PERIODO_ADC_MS`	`sensores_atualizar`
Postura (desvio, detector, calibração, alerta)	20 ms (50 Hz)	`PERIODO_POSTURA_MS`	`atualizarPostura`
Botão (debounce)	cada passo, estável por 50 ms	`BOTAO_DEBOUNCE_MS`	`atualizarBotao`

Etapa	Período	Constante	Onde
Motor (fases dos pulsos)	cada passo	—	`motor_atualizar`
Envio para app e plotter	100 ms (10 Hz)	`PERIODO_ENVIO_MS`	`loop`
JSON contínuo do BioSpine (`START`)	1 s	—	`console_enviarDados`
Nova tentativa de achar o MPU	2 s	`MPU_REINICIO_MS`	`sensores_atualizar`
Heartbeat do WebSocket	5 s	—	`web_iniciar`

O diagrama mostra como uma leitura vira vibração e dado no celular.


MPU-6050
acelerômetro g
giroscópio °/s
200 Hz
TiltFilter
filtro complementar
g: gravidade
vetor unitário
tiltComponentsDeg
frente e lateral °
referência g0
da calibração
angleBetweenDeg
desvio °
Status
10 Hz
Serial
plot on
10 Hz
App web
JSON data
ADC flex
0 a 4095
100 Hz
MediaMovel 10
Presenca
flexPct %
flexAtivo
DetectorPostura
50 Hz
ok, atenção, alerta
motor_alerta
Motor em pulsos
300 ms / 700 ms

Os capítulos seguintes explicam cada etapa: inclinação em 03, decisão e calibração em 04, flex e FSR em 05, motor e memória em 06, serial e protocolo em 07, página em 08.

6. Limitações conhecidas da arquitetura

- O loop lê as `Leituras` dos sensores (`sensores()`) direto, sem trava, em dois lugares: na checagem do MPU de `app_calibrar` e no `diag`. São leituras de exibição de campos simples; no pior caso o `diag` mostra números de dois instantes diferentes.
- O `diag` faz a varredura do barramento I2C (`sensores_scanI2C`) a partir do loop, enquanto a tarefa de controle lê o MPU pelo mesmo barramento. Depende da trava interna da biblioteca `Wire` do core; só se confirma no hardware.
- A tarefa de controle não fala com Wi-Fi nem serial, mas tem esperas curtas e conhecidas: timeout do I2C (até 20 ms) com o MPU solto, ~150 ms quando o MPU volta (reset do chip em `Mpu6050::iniciar`) e a gravação da calibração na NVS.
- A calibração grava na NVS (`settings_salvarCalibracao`) a partir da tarefa de controle. Gravar na flash leva alguns milissegundos; acontece só uma vez por calibração, quando a pessoa está parada e o alerta está desligado.
- A concorrência das duas tarefas não é testada no PC (o simulador roda tudo numa linha só). Ela só pode ser conferida no hardware: pontos V1.3 e V5.12 do `VALIDACAO.md` e uso prolongado sem travamento nem reset.

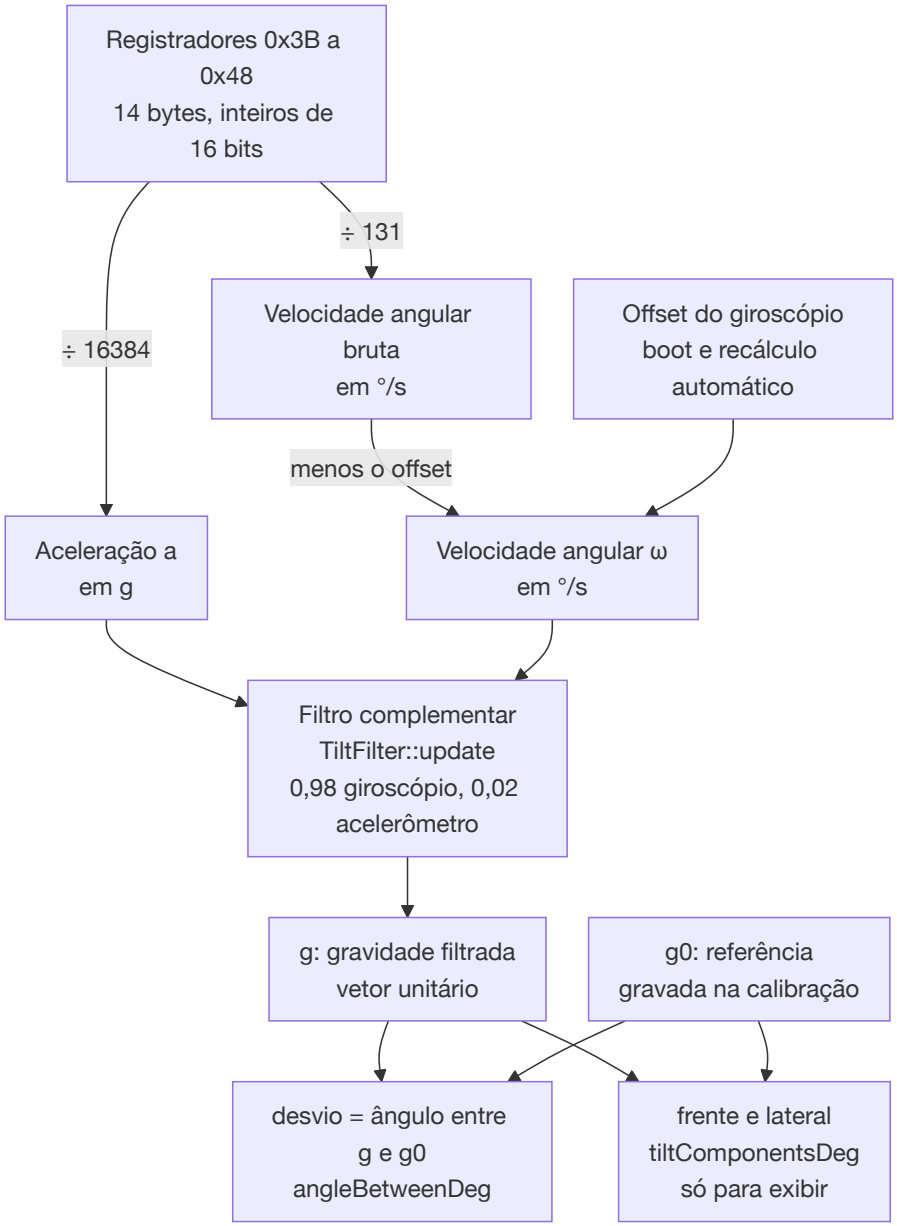
03 — Medição da inclinação do tronco

> Código: `src/mpu.h`, `src/mpu.cpp`, `src/sensors.cpp`, `src/tilt.h`, `src/tilt.cpp`, `src/config.h` · Relacionados: referencia/mpu.md, referencia/tilt.md, referencia/sensors.md, D-04 a D-07

1. O que precisa ser medido

O colete não precisa saber se o tronco está "na vertical". Precisa saber **quanto o tronco saiu da postura correta daquela pessoa**. Cada corpo é diferente e o sensor nunca fica preso perfeitamente alinhado. Por isso a medida central é o **desvio**: o ângulo entre a orientação atual e a orientação gravada na calibração.

O diagrama mostra o caminho completo, do registrador do sensor até o desvio em graus.



2. O sensor MPU-6050

O MPU-6050 tem um **acelerômetro** e um **giroscópio**, ambos de 3 eixos.

- O acelerômetro parado mede um vetor de módulo 1 g alinhado com a vertical. O código chama esse vetor de **gravidade** (`g`). Ele é absoluto (não acumula erro), mas tem ruído e também mede as acelerações do

movimento (andar, sentar, uma batida).
- O giroscópio mede a velocidade de rotação em cada eixo (°/s). É suave e rápido, mas tem um erro constante (offset) que, integrado, faz o ângulo "derivar" com o tempo.

Configuração (driver próprio, `Mpu6050::iniciar`)

Registrador	Valor	Efeito
`PWR_MGMT_1` (0x6B)	0x80, espera 100 ms	Reset do chip
`PWR_MGMT_1` (0x6B)	0x01	Acorda, usando o relógio do giroscópio X (mais estável que o oscilador interno)
`SMPLRT_DIV` (0x19)	0x00	Sem divisor de amostragem
`CONFIG` (0x1A)	0x03	Filtro passa-baixa interno (DLPF) em ~44 Hz: corta vibração e ruído acima disso
`GYRO_CONFIG` (0x1B)	0x00	Faixa ±250 °/s → 131 LSB por °/s
`ACCEL_CONFIG` (0x1C)	0x00	Faixa ±2 g → 16384 LSB por g
`ACCEL_CONFIG2` (0x1D)	0x03	Só na família MPU-6500 (clones com `WHO_AM_I` 0x70, 0x71, 0x73, 0x74): o filtro do acelerômetro fica neste registrador

As faixas mais sensíveis (±250 °/s e ±2 g) são as do texto do TCC e bastam: o tronco não gira a mais de 250 °/s nem acelera mais que 2 g no uso normal.

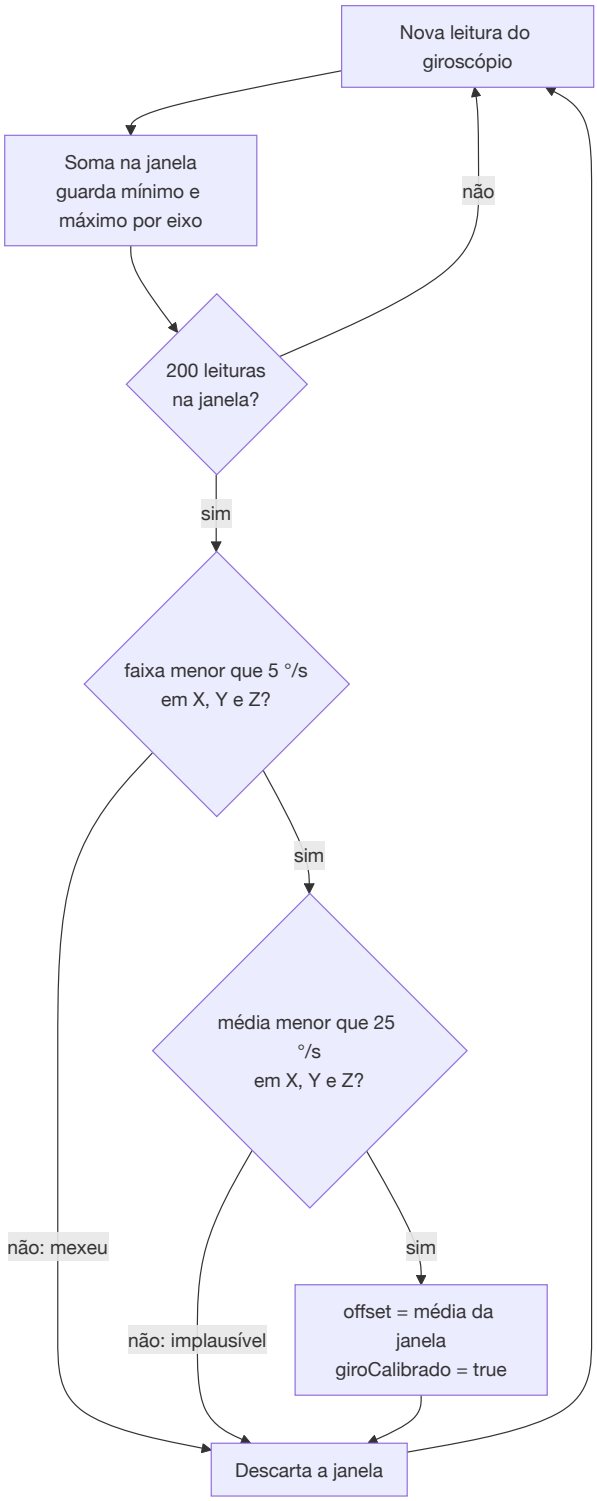
Leitura

- `Mpu6050::lerBruto` lê 14 bytes de uma vez a partir de `ACCEL_XOUT_H` (0x3B): aceleração X, Y, Z, temperatura (ignorada) e giroscópio X, Y, Z, cada um com 16 bits com sinal, byte alto primeiro.
- Conversão: $a = \text{bruto}/16384$ (g) e $\omega = \text{bruto}/131$ (°/s).
- Frequência: 200 Hz (`PERIODO_MPU_MS` = 5 ms), agendada em `sensores_atualizar`. O intervalo real entre leituras (`dt`) é medido com `micros()` e entra no filtro.
- Barramento I2C a 100 kHz (`MPU_I2C_HZ`), mais tolerante a fio longo no colete, com timeout de 20 ms.
- Endereço: procura em 0x68 e depois em 0x69 (pino AD0 solto ou em 3V3), atendendo as duas montagens.
- Robustez: 10 leituras seguidas com falha marcam o MPU como perdido (`ok_ = false`). A partir daí `sensores_atualizar` tenta encontrá-lo de novo a cada 2 s (`MPU_REINICIO_MS`). Cada tentativa segura a tarefa de controle por pouco tempo: alguns milissegundos de timeout do I2C enquanto o sensor não responde, e ~150 ms (esperas do reset do chip) na tentativa em que ele volta. Na reconexão o offset do giroscópio não é medido de novo: vale o anterior, e o recálculo automático cuida do resto.

3. Offset do giroscópio

Parado, o giroscópio não mede exatamente zero: cada eixo tem um offset de alguns °/s. Ele é medido e subtraído de toda leitura (`Mpu6050::ler`).

- **No boot** (`Mpu6050::calibrarGiro`): 200 leituras com 5 ms de intervalo (~1 s). Se em algum eixo a diferença entre a maior e a menor leitura passar de 5 °/s (`GIRO_PARADO_DPS`), a placa mexeu e o offset **não** é gravado.
- **Recálculo automático** (`Mpu6050::acumularOffset`): a cada 200 leituras (~1 s a 200 Hz) o driver confere a janela. Se a placa ficou parada (faixa < 5 °/s em todos os eixos) e a média é plausível (< 25 °/s, `GIRO_OFFSET_MAX`), a média vira o novo offset. Isso cobre o caso de ligar o colete já vestido, quando o boot não consegue medir.



Sem offset o sistema continua funcionando: o filtro complementar corrige a deriva com o acelerômetro. O offset só deixa a resposta a movimentos rápidos mais fiel.

4. Por que não usar ângulos de Euler (pitch e roll)

A forma comum de obter inclinação é calcular dois ângulos, pitch e roll, com fórmulas de `atan2` sobre os eixos do acelerômetro. Essas fórmulas supõem o sensor **deitado** (eixo Z para cima), como na bancada. No colete o sensor fica **em pé** nas costas (eixo Y para cima) e o eixo Z aponta para trás.

Nessa posição, uma fórmula do tipo $\mathrm{atan2}(a_x, a_z)$ tem os dois argumentos pequenos e o segundo troca de sinal quando o tronco passa pela vertical. Exemplo com 10° de inclinação lateral:

Tronco	a_x	a_z	$\text{atan2}(a_x, a_z)$
2° para trás	0,174	–0,034	101,2°
2° para a frente	0,174	+0,034	78,8°

Um movimento real de 4° vira um salto de 22° no ângulo calculado. Com a biblioteca `MPU6050_light` (sugerida no plano inicial) acontecia o mesmo: salto de ~20° perto da vertical com inclinação lateral. O teste `test_desvio_sem_salto_perto_da_vertical` reproduz o cenário e garante que o método adotado não salta.

Solução adotada: trabalhar direto com o **vetor gravidade** e medir o ângulo entre dois vetores. O ângulo entre vetores não tem singularidade, não depende de quais eixos do sensor apontam para onde e dispensa saber como o sensor foi preso. Pitch e roll ficam só para exibição (seção 7).

5. Filtro complementar sobre o vetor gravidade

Acelerômetro e giroscópio erram de jeitos opostos: o primeiro é ruidoso no curto prazo e correto no longo prazo; o segundo é suave no curto prazo e deriva no longo prazo. O **filtro complementar** junta o melhor dos dois: o giroscópio segue os movimentos rápidos e o acelerômetro corrige a deriva devagar.

A diferença deste projeto é aplicar o filtro ao **vetor** gravidade (`TiltFilter`), e não a ângulos. A cada leitura (`TiltFilter::update`, com ω convertido para rad/s):

1. Previsão pelo giroscópio. Um vetor fixo no mundo, visto por uma placa que gira com velocidade angular $\vec{\omega}$, parece girar ao contrário:

$$\frac{d\vec{g}}{dt} = -\vec{\omega} \times \vec{g} = \vec{g} \times \vec{\omega}$$

Integrando um passo (método de Euler) e renormalizando para manter o vetor unitário:

$$\vec{g}_{prev} = \text{normalize}\big(\vec{g} + (\vec{g} \times \vec{\omega})\,\Delta t\big)$$

2. Correção pelo acelerômetro. Mistura ponderada com a direção medida pelo acelerômetro:

$$\vec{g} = \text{normalize}\big(\alpha\,\vec{g}_{prev} + (1-\alpha)\,\hat{a}\big), \qquad \alpha = 0{,}98 \ (\texttt{MPU_FILTRO_GIRO})$$

3. Rejeição de aceleração. Se o módulo medido $|\vec{a}|$ estiver fora de [0,7; 1,3] g, o acelerômetro não está medindo só a gravidade (passo, batida, sentar de uma vez). Nesse instante vale só a previsão: $\vec{g} = \vec{g}_{prev}$.

4. Reinício. Na primeira leitura, ou se $\Delta t \le 0$ ou $\Delta t > 0{,}5$ s (sensor reconectado, pausa longa), o filtro recomeça com $\vec{g} = \hat{a}$ (`TiltFilter::reset`).

Constante de tempo

Com $\alpha = 0{,}98$ e $\Delta t = 5$ ms, a constante de tempo do filtro é

$$\tau = \frac{\alpha\,\Delta t}{1-\alpha} = \frac{0{,}98 \times 0{,}005}{0{,}02} \approx 0{,}25 \text{ s}$$

ou seja, variações mais lentas que ~0,65 Hz ($1/2\pi\tau$) vêm do acelerômetro e as mais rápidas, do giroscópio. Para postura (movimentos de segundos) a resposta é imediata e o ruído do acelerômetro fica atenuado. Aumentar α (ex.: 0,99) deixa o sinal mais suave e a correção da deriva mais lenta.

Por que este filtro

- Um parâmetro só (α), comportamento previsível e custo mínimo de CPU.
- Só a inclinação interessa; a rotação em torno da vertical (olhar para o lado) não muda a postura e não precisa ser medida. Por isso não há magnetômetro nem estimativa de orientação completa.
- Sobre o vetor não existe ângulo "dando a volta" nem singularidade, qualquer que seja a montagem.

6. Desvio

$$\text{desvio} = \operatorname{atan2}\big(|\vec{g} \times \vec{g}_0|,\ \vec{g} \cdot \vec{g}_0\big) \quad (\text{em graus, de } 0 \text{ a } 180)$$

(`angleBetweenDeg`). É o ângulo total entre a orientação atual e a de referência, sem sinal.

Por que `atan2` e não `acos` do produto escalar:

- `acos` tem derivada infinita perto de 0°, justamente a região de trabalho (desvios pequenos): o erro de arredondamento do produto escalar vira erro grande de ângulo.
- `acos` exige vetores exatamente unitários; um produto escalar de 1,0000001 dá `NaN`. `atan2` só depende da razão entre os dois termos: funciona com qualquer módulo e nunca dá `NaN`.

Inclinações em direções diferentes se compõem como ângulo no espaço, não somando:

Tronco	desvio	frente	lateral
25° para a frente	25,0°	25,0°	0,0°
15° para o lado	15,0°	0,0°	15,0°
10° para a frente e 10° para o lado	14,1°	10,0°	10,2°

Referência antes da calibração: sem calibração, a referência é a primeira gravidade lida depois do boot (`gBoot` em `main.cpp`). Serve só para o app mostrar que o sensor reage; o detector não alerta sem calibração (04).

7. Frente/trás e lateral (só para exibir)

Para o app e o gráfico, o desvio é decomposto em dois ângulos com sinal (`tiltComponentsDeg`):

1. $\hat{r} = \text{normalize}(\vec{g}_0)$: a vertical da referência.
2. Eixo "frente" $\hat{f}$: o eixo Z da placa projetado no plano perpendicular a $\hat{r}$ ($\hat{f} = \text{normalize}(\hat{z} - \hat{r}(\hat{z} \cdot \hat{r}))$). Se a placa estiver deitada ($|\hat{z} \cdot \hat{r}| > 0{,}9$, caso da bancada), usa o eixo Y.
3. Eixo lateral: $\hat{l} = \hat{r} \times \hat{f}$.
4. $\text{frente} = \operatorname{atan2}(\vec{g} \cdot \hat{f},\ \vec{g} \cdot \hat{r})$ e $\text{lado} = \operatorname{atan2}(\vec{g} \cdot \hat{l},\ \vec{g} \cdot \hat{r})$.

Com o sensor preso com os componentes para fora, inclinar o tronco para a frente dá `pitch` positivo. Se o sensor for montado ao contrário, só o sinal do rótulo inverte; o desvio e o alerta não mudam (o app tem as opções "Inverter frente/trás" e "Inverter lado" para corrigir a exibição).

8. Evidências (testes)

Teste (`test/test_posture/test_main.cpp`)	O que garante
`test_angulo_entre_vetores`	0°, 90°, 30° e 0,5° corretos (precisão em ângulo pequeno)
`test_desvio_sem_salto_perto_da_vertical`	Nenhum salto ao passar pela vertical com 10° de lateral
`test_componentes_frente_e_lado`	Decomposição frente/lateral com o sensor em pé
`test_componentes_com_sensor_de_cabeca_para_baixo`	Sensor girado 180° (pinos para cima): "frente" continua positivo para a frente
`test_componentes_com_placa_deitada`	Na bancada (placa deitada) troca o eixo de referência e não dá `NaN`
`test_filtro_converge_para_acelerometro`	Parado por 10 s, o filtro chega à direção do acelerômetro (erro < 0,5°)
`test_filtro_segue_giroscopio`	Com o acelerômetro em choque (2 g, ignorado), segue só o giroscópio: −90 °/s por 0,2 s = 18°
`test_filtro_reinicia_com_dt_grande`	Pausa de 2 s reinicia o filtro direto pelo acelerômetro

Detalhes de todos os testes: 09.

9. Limitações

- **Sinal do giroscópio no filtro.** A convenção $d\vec{g}/dt = \vec{g} \times \vec{\omega}$ foi conferida no papel, mas o simulador usa a mesma convenção do código e por isso não prova nada. A confirmação é no hardware (`VALIDACAO.md`, V2.6: inclinar rápido não pode dar "repique").
- **Acelerações longas** dentro de [0,7; 1,3] g (curva de carro, elevador) passam pela correção e distorcem um pouco o desvio durante o movimento; o tempo mínimo de 3 s antes de vibrar absorve a maior parte disso.
- **Rotação em torno da vertical** não é medida, de propósito: não muda a postura.

04 — Detecção de má postura e calibração

> Código: `src/posture.h`, `src/posture.cpp`, `src/main.cpp` (`atualizarPostura`, `iniciarCalibracao`, `concluirCalibracao`, `atualizarBotao`, `sensorAtivo`), `src/config.h` · Relacionados: referencia/posture.md, referencia/main.md, D-08 a D-10

1. Parâmetros

Os parâmetros da decisão ficam na struct `Params` e podem ser mudados pela serial (`set ...`) ou pelo app, sem recompilar. São salvos na NVS (06). Os padrões são folgados de propósito: funcionam sem ajuste fino.

Parâmetro	Significado	Padrão	Faixa aceita (`validarParams`)
`limiteGraus`	Desvio máximo aceito	20° (`PADRAO_LIMITE_GRAUS`)	2–60°
`histerese`	Margem abaixo do limite para sair do alerta (em ° para o MPU e em pontos % para o flex)	3 (`PADRAO_HISTERESE`)	0 até (menor limite − 1)
`tempoMs`	Tempo em má postura antes de vibrar (tempo mínimo)	3000 ms (`PADRAO_TEMPO_MS`)	0–60000 ms
`limiteFlex`	Curvatura máxima do flex em relação à calibração	15% (`PADRAO_LIMITE_FLEX`)	2–90%
`intensidade`	Força do motor (duty do PWM)	100% (`PADRAO_INTENSIDADE`)	0–100%

`validarParams` prende cada valor na faixa (valor vindo da serial ou do app pode ser qualquer coisa, até `NaN`, que vira o mínimo). A histerese precisa ser menor que os dois limites: se fosse igual ou maior, "limite − histerese" seria zero ou negativo e a postura nunca sairia do alerta.

2. Regra de decisão

A cada 20 ms (50 Hz, `PERIODO_POSTURA_MS`), `atualizarPostura` monta uma `Entrada` e chama `DetectorPostura::atualizar`. A regra tem três ingredientes:

1. **Limite.** A postura passa a contar como ruim quando **qualquer** sensor ativo passa do limite:

   ```
   acima  = desvio > limite
            OU (flex ativo  E flexPct  > limiteFlex)
            OU (flex 2 ativo E flex2Pct > limiteFlex)
   ```

2. **Histerese.** A postura só volta a contar como boa quando **todos** os sensores ativos ficam abaixo do limite menos a histerese:

   ```
   abaixo = desvio < limite − histerese
            E (flex inativo  OU flexPct  < limiteFlex − histerese)
            E (flex 2 inativo OU flex2Pct < limiteFlex − histerese)
   ```

   Entre os dois valores (por exemplo, desvio entre 17° e 20° com os padrões) o estado anterior é mantido. Sem essa faixa, um desvio oscilando em torno de 20° ligaria e desligaria o motor sem parar.

3. **Tempo mínimo.** A má postura só vira **alerta** (motor) depois de durar `tempoMs`. Um movimento rápido (pegar algo no chão, amarrar o sapato) não vibra.

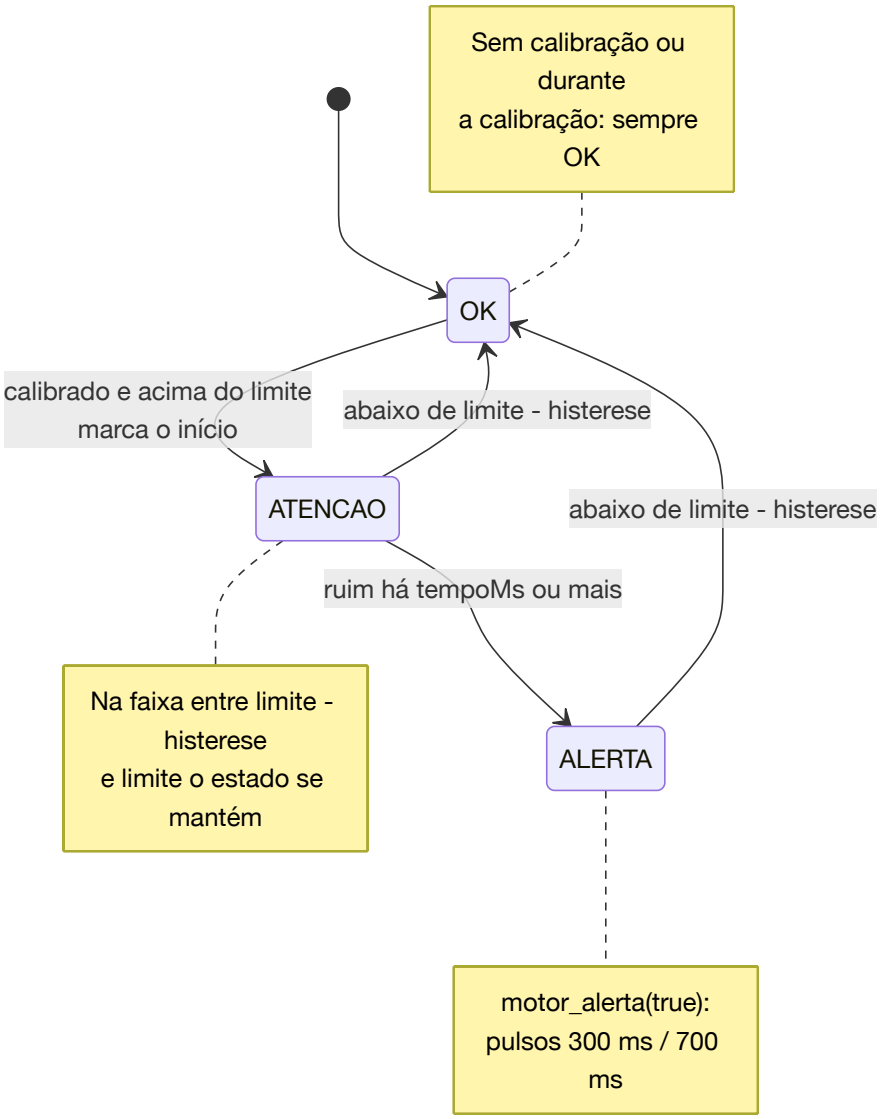
Mais duas regras de segurança:

- **Sem calibração, nunca alerta** (D-09): sem referência não existe "postura correta", e vibrar contra uma referência arbitrária seria incômodo e errado.

- **Sem MPU** (fio solto), o desvio é tratado como 0 e só o flex pode alertar.

Máquina de estados

O diagrama mostra os três estados do `DetectorPostura` e o que faz mudar de um para outro.



Estado	Nome no JSON	Motor	Cor no app
`ESTADO_OK`	`ok`	desligado	verde
`ESTADO_ATENCAO`	`atencao`	desligado (contando o tempo)	amarelo
`ESTADO_ALERTA`	`alerta`	pulsando	vermelho

Detalhes:

- O tempo conta a partir da entrada em "ruim" e **não** recomeça enquanto o desvio fica na faixa da histerese.
- Com `tempoMs = 0` o estado vai direto de ok para alerta na mesma atualização.
- `tempoRuimMs` (no status e no app) é há quanto tempo a postura está ruim; o app usa para a barra de contagem.

Exemplo com os padrões (limite 20°, histerese 3°, tempo 3 s)

t (s)	desvio	Estado	Por quê
0	10°	ok	abaixo do limite
1	25°	atenção	passou de 20°: começa a contar
2	25°	atenção	ruim há 1 s
4	25°	**alerta**	ruim há 3 s: motor começa a pulsar
5	18°	**alerta**	18° está entre 17° e 20°: mantém
6	16°	ok	abaixo de 17°: motor para
10	30°	atenção	pico
11	15°	ok	voltou antes de 3 s: **não vibrou**

3. Calibração

A calibração grava a **postura correta** da pessoa: o vetor gravidade `g0` e as leituras dos flex (`flex0`, `flex20`). Tudo o que o detector mede é relativo a ela.

Como é pedida

Origem	Como	Caminho no código
Botão (GPIO27)	Apertar (estável por 50 ms, `BOTAO_DEBOUNCE_MS`)	`atualizarBotao` → `iniciarCalibracao(ORIGEM_BOTAO)`, direto na tarefa de controle
Serial	`calib` ou `calibrar`	`app_calibrar(ORIGEM_SERIAL)` → pedido
Serial (BioSpine V3)	`CALIBRATE`	`app_calibrar(ORIGEM_SERIAL_JSON)` → pedido; responde JSON ao terminar
App	`{"cmd":"calib"}`	`protocolo_comando` → `app_calibrar(ORIGEM_APP)` → pedido

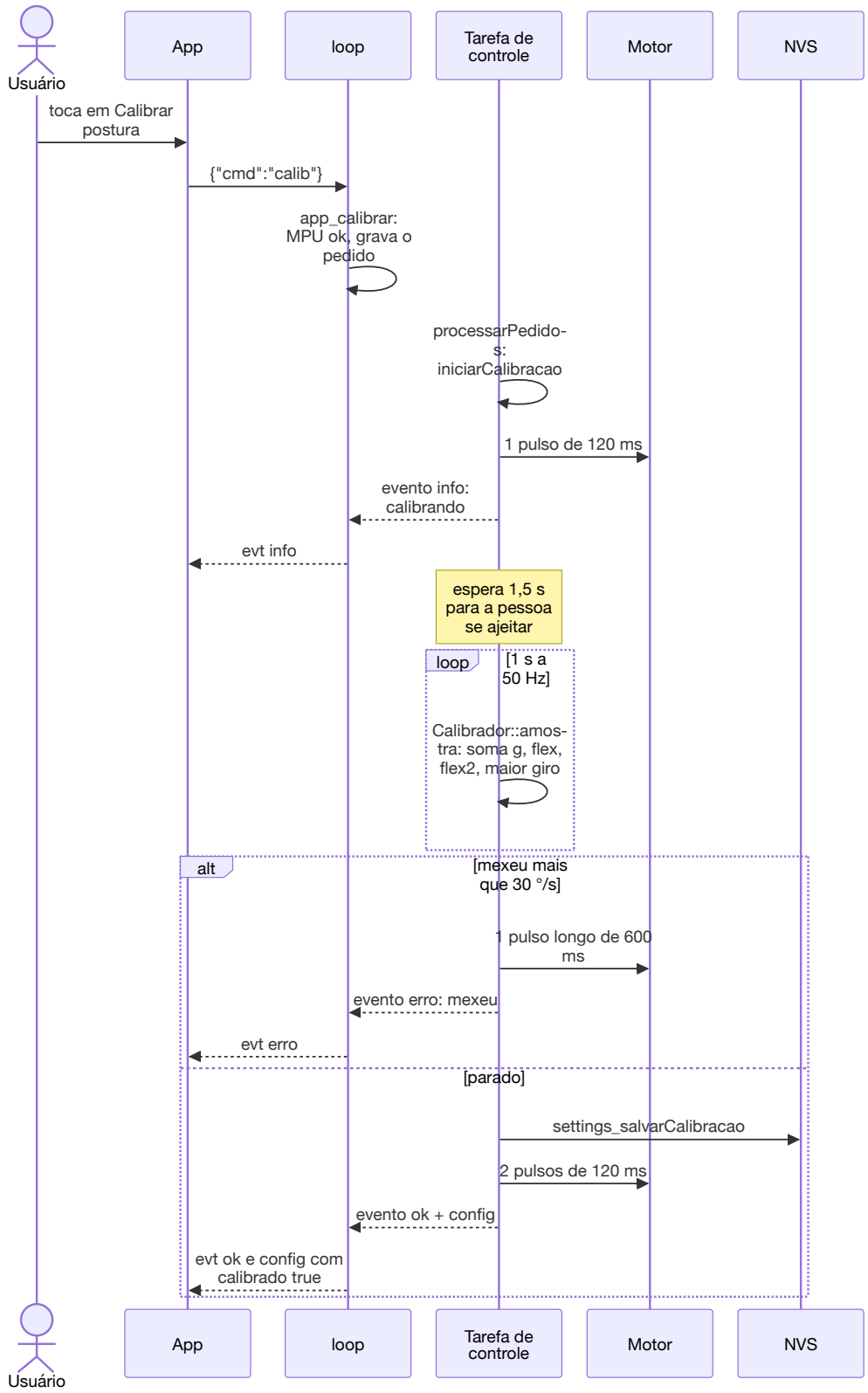
`app_calibrar` (no loop) recusa na hora se o MPU não estiver respondendo. Os pedidos vindos do loop são executados pela tarefa de controle no passo seguinte (`processarPedidos`), como explicado em 02.

Etapas

1. **Início** (`iniciarCalibracao`): se já houver uma calibração em andamento, o pedido é ignorado. Senão, zera o detector, dá **1 pulso curto** (120 ms, `MOTOR_CONFIRMA_MS`: "começou, fique parado") e avisa a serial e o app.
2. **Espera** de 1,5 s (`CALIBRACAO_ESPERA_MS`): tempo para a pessoa soltar o botão e se ajeitar. O movimento de apertar o botão não entra na média.
3. **Média** durante 1 s (`CALIBRACAO_MS`): a cada atualização de postura (50 Hz, ~50 amostras), `Calibrador::amostra` soma o vetor gravidade, as leituras dos dois flex e guarda a maior velocidade angular.
4. **Conclusão** (`concluirCalibracao`):
   - Se a maior velocidade angular passou de 30 °/s (`CALIBRACAO_GIRO_MAX`), a pessoa se mexeu: a calibração é **recusada** com **1 pulso longo** (600 ms, `MOTOR_ERRO_MS`) e a calibração anterior continua valendo.
   - Senão: `g0` = direção média (soma dos vetores normalizada); `flex0`/`flex20` = média da leitura, só se o sensor estiver detectado (ou forçado ligado), senão 0. Grava na NVS, zera o detector, dá **2 pulsos curtos** ("pronto") e avisa a serial e o app (que recebe também a nova config, com `calibrado: true`).

Durante a calibração o estado fica forçado em ok e o motor não alerta.

A sequência mostra uma calibração pedida pelo app, do toque no botão até a confirmação.



Por que assim

- **Média em vez de leitura única:** reduz o ruído da referência; somar os vetores e normalizar dá a direção média.
- **Espera antes da média:** quem aperta o botão nas costas mexe o tronco; a espera tira esse movimento.
- **Recusa se mexer:** uma referência gravada em movimento faria o colete vibrar na postura correta. É melhor recusar e pedir de novo, com retorno pela vibração (dá para calibrar sem olhar tela nenhuma).
- **Flex só se detectado:** gravar a referência de um pino solto criaria um flex "fantasma" na detecção.

4. Quando um flex entra na decisão

O flex só entra na regra se estiver **ativo** (`sensorAtivo` em `main.cpp`), o que depende do modo configurado, da detecção automática (05) e de a referência ter sido gravada com ele ligado (100 ≤ referência ≤ 4000, `ADC_MIN_VALIDO`/`ADC_MAX_VALIDO`).

Modo (`flexModo`, `flex2Modo`)	Detectado?	Referência válida?	Entra na decisão?	`get` mostra
`off`	qualquer	qualquer	não	desligado
`on`	qualquer	sim	sim	ativo
`on`	qualquer	não	não	não detectado, ou detectado, recalibre para usar
`auto`	sim	sim	sim	ativo
`auto`	sim	não	não	detectado, recalibre para usar
`auto`	não	qualquer	não	não detectado

Padrões: flex 1 em `auto`, flex 2 em `off` (D-11). No modo `auto`, um flex que solta depois da calibração sai da decisão sozinho (deixa de ser detectado) e o colete segue só com o MPU. **No modo `on` isso não acontece:** o modo forçado ignora a detecção, então um flex solto (leitura ~0, curvatura ~100%) dispara o alerta. Use `on` só para teste. Um flex ligado depois da calibração só entra depois de recalibrar.

5. Evidências (testes)

Teste	O que garante
`test_sem_calibracao_nunca_alerta`	Sem calibração, nem desvio de 80° por 10 s alerta
`test_atencao_e_depois_alerta`	ok → atenção → alerta só depois de `tempoMs`
`test_histerese`	Fica em alerta na faixa entre limite – histerese e limite; sai abaixo dela
`test_pico_curto_nao_alerta`	Pico menor que `tempoMs` não vibra, e a contagem recomeça do zero no pico seguinte
`test_flex_dispara_e_respeita_histerese`	Flex sozinho leva ao alerta e respeita a histerese
`test_flex_inativo_e_ignorado`	Flex inativo não influencia, mesmo em 100% (flex solto lê 0)
`test_flex2`	Flex 2 sozinho acima do limite já leva a atenção (entra no OU), com o flex 1 em 0%
`test_flex_pct`	Fórmula da curvatura e casos de borda
`test_validar_params`	Faixas, `NaN` e histerese menor que os limites
`test_calibrador_media`	Espera, coleta, média e maior giro do calibrador

O fluxo completo (botão, pulsos, recusa, NVS) é exercitado no simulador pelos roteiros de `tools/` (09).

05 — Sensores analógicos: flex e FSR

> Código: `src/sensors.h`, `src/sensors.cpp`, `src/filters.h`, `src/posture.cpp` (`flexPct`), `src/main.cpp` (`sensorAtivo`), `src/config.h` · Relacionados: referencia/sensors.md, referencia/filters.md, D-11

1. Papel de cada sensor

Sensor	Pino	Entra no alerta?	Padrão
Flex 1	GPIO34	sim, se ativo	modo `auto`
Flex 2	GPIO35	sim, se ativo	modo `off`
FSR (pressão, montagem BioSpine)	GPIO32	**não**: só telemetria	sempre lido

O MPU mede a inclinação do tronco; o flex mede a **curvatura** da coluna (a pessoa pode "corcundar" sem inclinar muito o tronco). Os dois se completam, mas o colete **não depende do flex**: sem ele, segue só com o MPU.

2. Circuito e leitura

O flex é um resistor que aumenta quando dobra (~10 kΩ reto, tolerância ±30%). Ligado num divisor de tensão com um resistor fixo de 10 kΩ:

```
3V3 ── flex ─┬── GPIO34
             └── 10 kΩ ── GND
```

$$V_{34} = 3{,}3\ \text{V} \times \frac{10\ \text{k}\Omega}{R_{flex} + 10\ \text{k}\Omega}$$

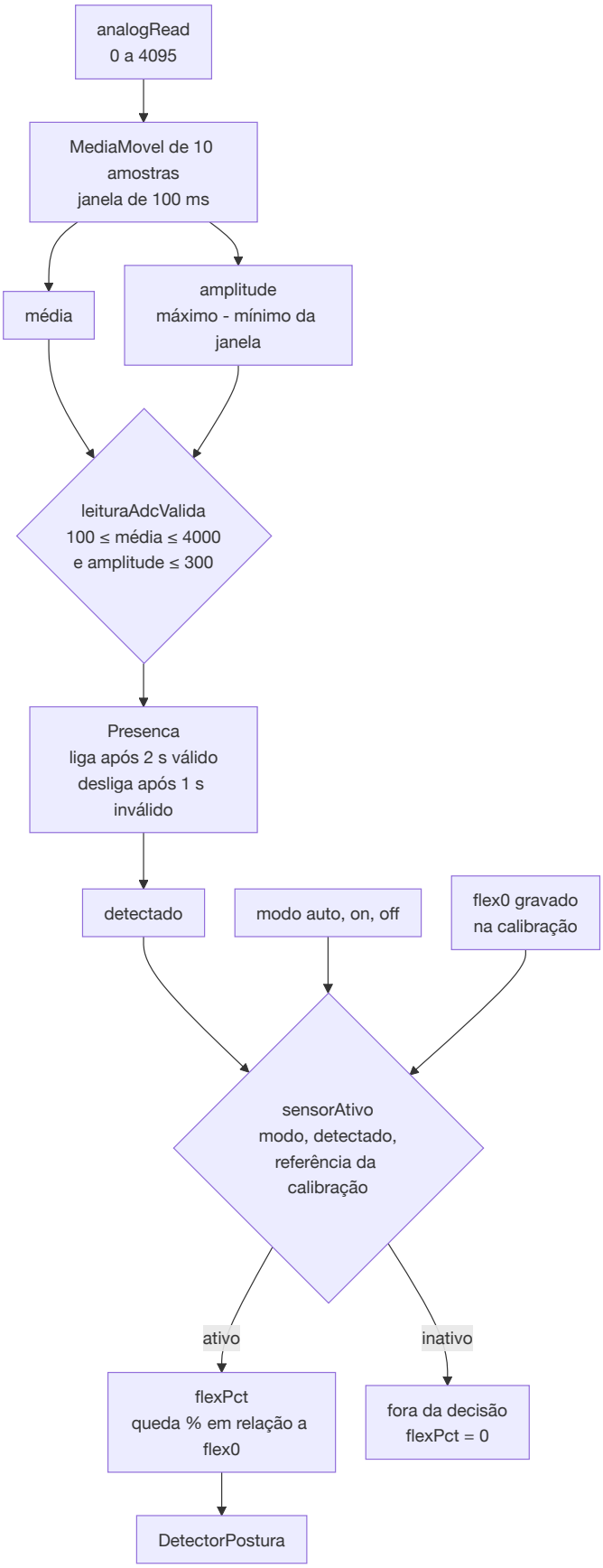
Situação	R_{flex}	V_{34}	Leitura (0–4095)	Curvatura (`flexPct`)
Reto	~10 kΩ	~1,65 V	~2048 (na prática 1800–2300)	0% (é a referência)
Dobrado	~20 kΩ	~1,10 V	~1365	~33%
Falta o resistor de 10 kΩ	—	~3,3 V	~4095	inválido
Flex solto ou quebrado	aberto	~0 V	~0	inválido

Ao dobrar, a leitura cai. Por isso a curvatura é a queda relativa à leitura da calibração.

Configuração do ADC (`sensores_iniciar`): 12 bits (`analogReadResolution(12)`) e atenuação de 11 dB por pino (faixa até ~3,3 V; o ADC do ESP32 é pouco linear nas pontas, o que não atrapalha porque só importa a variação relativa). Todos os sensores ficam no **ADC1**: o ADC2 não funciona com o Wi-Fi ligado.

3. Do pino à decisão

O diagrama mostra o que acontece com cada leitura, a 100 Hz (`PERIODO_ADC_MS` = 10 ms).



Média móvel (`MediaMovel<N>`)

Buffer circular de N = 10 amostras (`ADC_MEDIA_N`) com soma acumulada: cada amostra nova entra e a mais antiga sai, sem somar a janela inteira de novo. Tira o ruído do ADC (que aumenta com o Wi-Fi ligado). Também calcula a **amplitude** da janela (máximo − mínimo), usada para reconhecer pino solto.

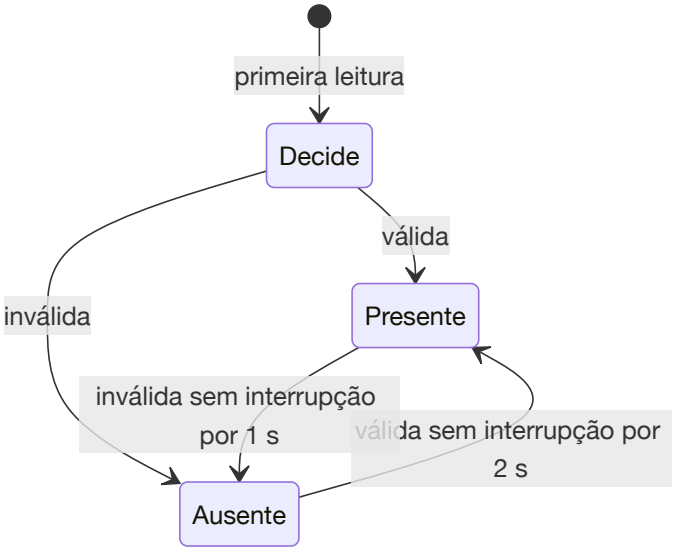
Leitura plausível (`leituraAdcValida`)

Um divisor resistivo de verdade dá leitura **dentro da faixa** e **estável**. Um pino solto ou um flex quebrado falha numa das duas:

- média abaixo de 100 (`ADC_MIN_VALIDO`): flex solto ou quebrado;
- média acima de 4000 (`ADC_MAX_VALIDO`): falta o resistor de 10 kΩ;
- amplitude acima de 300 (`ADC_RUIDO_MAX`): pino flutuando ou mau contato (oscila muito mais que um divisor).

Presença com persistência (`Presenca`)

A validade de uma janela só vira "sensor detectado" se durar. Assim o flex não "pisca" entre ativo e inativo por causa de um ruído.



- Os tempos são `FLEX_LIGA_MS` (2 s) e `FLEX_DESLIGA_MS` (1 s). Qualquer leitura que concorde com o estado atual reinicia a contagem.
- **Flex 1** decide já no boot, depois de encher a média: ligado e bom, fica detectado na hora.
- **Flex 2** e **FSR** começam ausentes e só ficam presentes depois de 2 s de leitura estável.

Curvatura (`flexPct`)

$$\text{flexPct} = \text{clamp}\left(\frac{\text{flex0} - \text{leitura}}{\text{flex0}} \times 100,\ 0,\ 100\right)$$

- `flex0` é a leitura média gravada na calibração (postura correta).
- Leitura maior que a referência (o flex "esticou") dá 0%, nunca negativo.
- Sem referência (`flex0` < 1) dá 0%.
- A curvatura é relativa e não está em graus: o valor de uma "corcunda" típica depende do flex e de onde ele está preso. Por isso o limite (`limiteFlex`, 15% por padrão) é ajustável pelo app.

4. O "fantasma" do ADC e o flex 2

O ADC do ESP32 tem um único conversor com um capacitor de amostragem compartilhado entre os pinos. Um pino **solto** lido logo depois de um pino ligado devolve aproximadamente a tensão do pino anterior, porque o capacitor ainda guarda a carga. Os ADCs são lidos na ordem flex 1 → flex 2 → FSR. Com o GPIO35 solto, o flex 2 "copia" o flex 1: leitura plausível e estável, igual a um sensor de verdade.

Consequências no código:

- **Flex 2 começa desligado** (`configPadrao`: `flex2Modo = MODO_OFF`). Quem tem o segundo flex liga com `set flex2 auto` (ou pelo app) e recalibra (D-11).
- **FSR "fantasma":** uma leitura do FSR a menos de 300 (`ADC_FANTASMA_DIF`) da leitura do flex 1 é tratada como cópia e não conta para a detecção.

5. FSR

O sensor de pressão da montagem BioSpine V3 é só **telemetria** (aparece no app, no `diag` e no JSON `pressure` dos comandos BioSpine). Detecção: leitura acima de 200 (`FSR_LIMIAR`), estável e não fantasma, durante 2 s. Como um FSR sem pressão lê perto de 0 (igual a pino solto), **depois de detectado uma vez ele fica detectado até reiniciar** (`Presenca` com tempo de desligar infinito).

6. Mensagens de diagnóstico

O `diag` usa as mesmas regras para dizer o que está errado:

Leitura do flex 1	Mensagem	Causa provável
~2000, ✅	dobre o sensor e veja o valor mudar	ok
~4095, ❌	falta o resistor de 10 kΩ	resistor fora do divisor
~0, ❌	flex solto ou quebrado	fio solto, flex quebrado na base
outro, ❌	leitura instável: mau contato?	protoboard, jumper frouxo

Nos três casos de ❌ o colete continua só com o MPU, desde que o flex esteja no modo `auto` (padrão). No modo `on` a detecção é ignorada: o flex fica na decisão mesmo solto e a curvatura ~100% dispara o alerta.

7. Evidências (testes)

Teste	O que garante
`test_media_movel`	Média, janela cheia, descarte da amostra mais antiga, amplitude, última amostra
`test_presenca_com_persistencia`	Liga só depois de 2 s válidos sem interrupção; ruído reinicia a contagem; desliga após 1 s
`test_leitura_adc_valida`	Aceita divisor normal; recusa ~4095, ~0 e pino flutuando
`test_flex_pct`	0% na referência, 25% com queda de 25%, 0% esticado, 0% sem referência
`test_flex_inativo_e_ignorado`	Flex inativo não alerta, mesmo em 100%

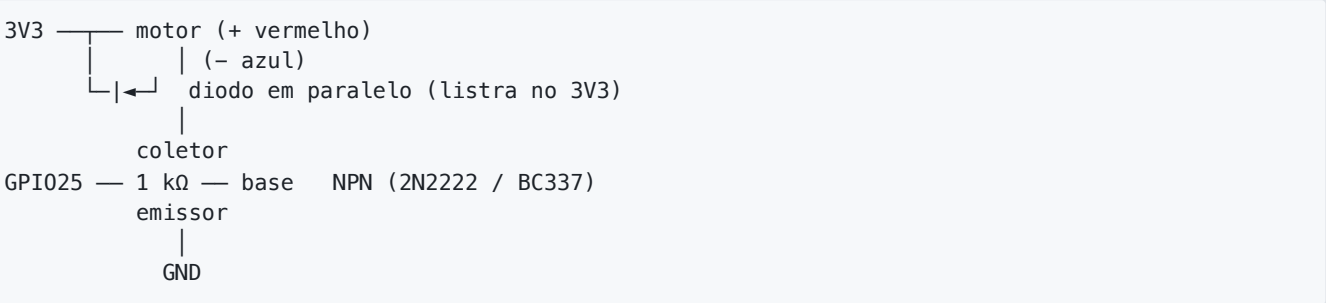
Flex desconectado em uso, flex 2 e FSR também são cobertos pelos roteiros do simulador (09).

06 — Motor vibratório e memória não volátil (NVS)

> Código: `src/motor.h`, `src/motor.cpp`, `src/settings.h`, `src/settings.cpp`, `src/config.h` · Relacionados: referencia/motor.md, referencia/settings.md, D-14, D-15

1. Circuito do motor

O motor tipo moeda consome ~60 mA; um GPIO do ESP32 não fornece isso. O GPIO25 comanda um transistor NPN que funciona como chave do lado do terra:



- Com o GPIO em nível alto, a corrente de base é ~(3,3 – 0,7) V / 1 kΩ ≈ 2,6 mA, suficiente para saturar o transistor com os 60 mA do motor.
- O **diodo** absorve o pico de tensão que a bobina do motor gera ao desligar e que queimaria o transistor.
- O **capacitor de 100 µF** (opcional) perto do motor fornece o pico de corrente da partida, para a tensão não cair e reiniciar o ESP32.

2. PWM

O motor é ligado por PWM do periférico LEDC (`motor_iniciar`), canal 0, **20 kHz** (`MOTOR_LEDC_FREQ`), 8 bits (`MOTOR_LEDC_BITS`, duty de 0 a 255).

- **Intensidade:** duty = 255 × `intensidade` / 100. Muda pela serial (`set motor 50`) ou pelo app; vale para alertas, confirmações e testes.
- **Por que 20 kHz:** PWM na faixa audível faz a bobina do motor "chiar". A 20 kHz o chiado fica acima do que se ouve.
- **LED da placa (GPIO2)** acende e apaga junto com o motor. Serve de diagnóstico: LED acendendo sem o motor vibrar indica problema no circuito do motor, não no firmware.
- O código compila nos cores Arduino-ESP32 2.x (`ledcSetup` + `ledcAttachPin`, escrita por canal) e 3.x (`ledcAttach`, escrita por pino), escolhendo pela macro `ESP_ARDUINO_VERSION_MAJOR`.

3. Padrões de vibração

O motor também é a interface com o usuário quando não há celular: cada situação tem um padrão diferente.

Situação	Padrão	Constantes	Chamado por
Ligou	1 pulso de 120 ms	`MOTOR_CONFIRMA_MS`	`setup`
Calibração começou ("fique parado")	1 pulso de 120 ms	`MOTOR_CONFIRMA_MS`	`iniciarCalibracao`
Calibração concluída	2 pulsos de 120 ms, 120 ms entre eles	`MOTOR_CONFIRMA_MS`	`concluirCalibracao`
Calibração recusada (mexeu)	1 pulso longo de 600 ms	`MOTOR_ERRO_MS`	`concluirCalibracao`

Situação	Padrão	Constantes	Chamado por
Teste (`motor`, `diag`, botão do app)	1 pulso de 1 s	`MOTOR_TESTE_MS`	`processarPedidos`
Alerta de postura	300 ms ligado / 700 ms desligado, enquanto durar	`MOTOR_ALERTA_ON_MS`, `MOTOR_ALERTA_OFF_MS`	`atualizarPostura` → `motor_alerta`

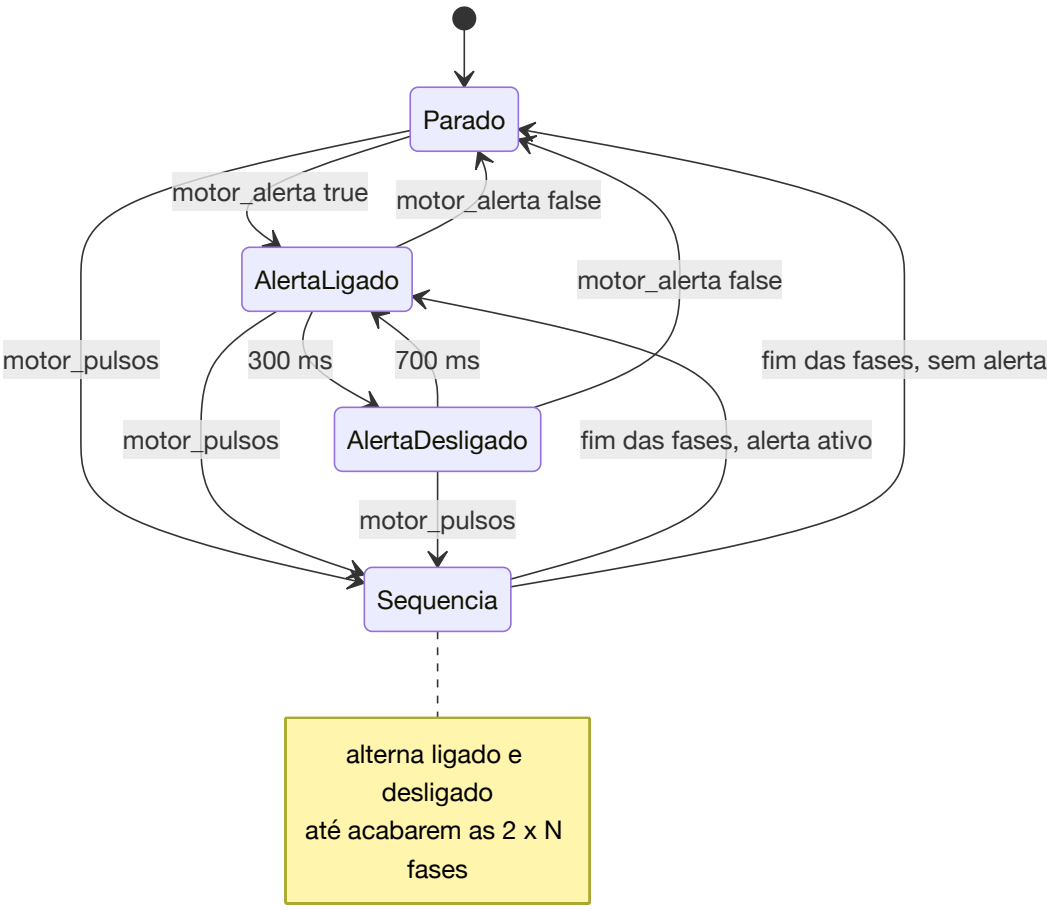
Pulsos incomodam menos que vibração contínua e continuam sendo percebidos (o corpo se acostuma com estímulo constante).

4. Lógica do motor

O módulo não usa `delay()`: guarda a fase atual e o instante em que ela começou (`marco`), e `motor_atualizar` (chamada a cada passo da tarefa de controle) troca de fase quando o tempo dela acaba. Há duas fontes de vibração:

- **Alerta** (`motor_alerta(ativo)`): chamado a cada atualização de postura com `estado == ESTADO_ALERTA`. Só age quando o valor muda. Ao ativar, já começa ligado (resposta imediata).
- **Sequência** (`motor_pulsos(quantidade, ligadoMs, desligadoMs)`): confirmações e testes. Cada pulso tem duas fases (ligado e desligado), então uma sequência tem 2 × `quantidade` fases.

A sequência tem prioridade. Se um alerta começar ou terminar durante uma sequência, o novo valor é guardado e aplicado quando a sequência acabar. Uma sequência nova substitui a anterior.



5. Memória não volátil (NVS)

O ESP32 **não tem EEPROM**. A biblioteca `Preferences` grava pares chave–valor numa partição da flash (NVS, *non-volatile storage*), com distribuição de desgaste feita pelo próprio ESP-IDF. Os parâmetros e a calibração sobrevivem a desligar a placa.

Namespace: `colete`.

Chave	Tipo	Campo	Padrão ao ler
`limite`	float	`Params::limiteGraus`	20
`hist`	float	`Params::histerese`	3
`tempo`	uint32	`Params::tempoMs`	3000
`flexlim`	float	`Params::limiteFlex`	15
`motor`	uint8	`Params::intensidade`	100
`flexmodo`	uint8	`Config::flexModo` (0 auto, 1 on, 2 off)	0 (auto)
`flex2modo`	uint8	`Config::flex2Modo`	2 (off)
`cal_ok`	bool	`Calibracao::valida`	false
`g0x`, `g0y`, `g0z`	float	`Calibracao::g0`	0, 1, 0
`flex0`	float	`Calibracao::flex0`	0
`flex20`	float	`Calibracao::flex20`	0

Leitura (`settings_carregar`, no `setup`)

- Chave ausente (placa nova) usa o padrão de `configPadrao()`.
- Os parâmetros passam por `validarParams`: um valor corrompido não deixa o colete num estado absurdo.
- Modo de sensor fora de 0–2 vira `auto`.
- Calibração com |g0| < 0,5 é considerada inválida (`valida = false`): o colete pede nova calibração.

Gravação

Função	Grava	Quando	Tarefa
`settings_salvarConfig`	as 7 chaves de configuração	`app_definirConfig`: `set` e `reset` da serial, `set` e `reset` do app	loop
`settings_salvarCalibracao`	as 6 chaves de calibração	`concluirCalibracao`, quando a calibração é aceita	controle

- `reset` (serial ou app) volta a configuração ao padrão e **mantém a calibração**.
- Só há gravação por ação do usuário, então o desgaste da flash é desprezível.

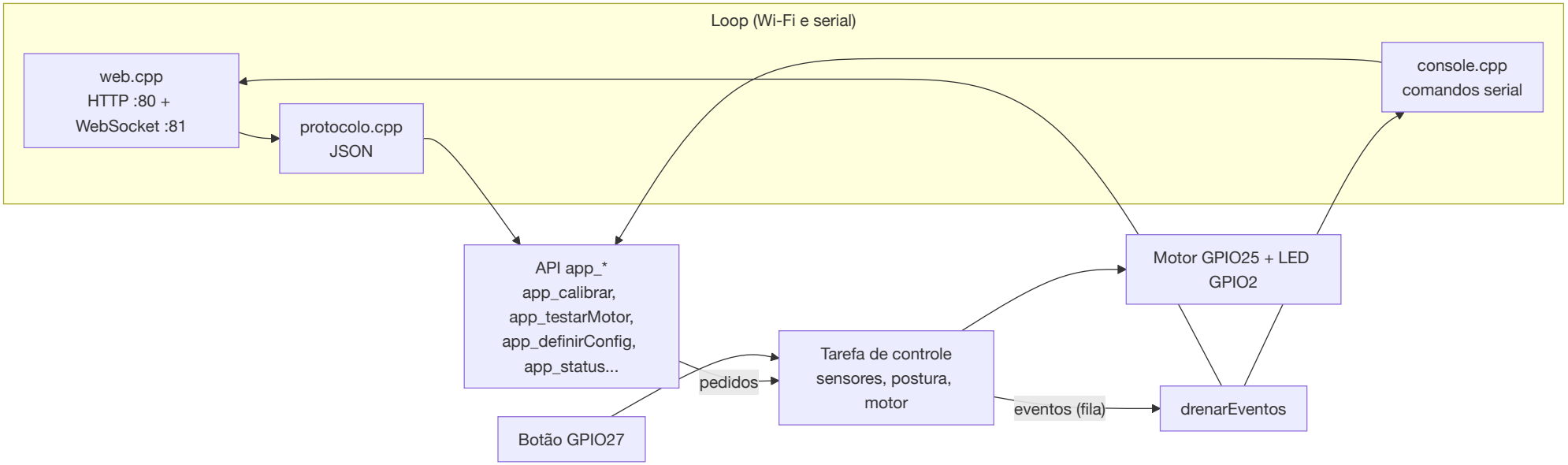
07 — Protocolo e interfaces

> Código: `src/protocolo.h`, `src/protocolo.cpp`, `src/web.h`, `src/web.cpp`, `src/console.h`, `src/console.cpp`, `src/app.h` ·
> Relacionados: referencia/protocolo, referencia/web, referencia/console, 08 — App web, 02 — Arquitetura

O colete tem quatro formas de interação. Só a primeira é necessária: o colete **funciona sozinho**.

Interface	Meio	Para quê
Botão + motor	GPIO27 / GPIO25	calibrar e receber o alerta (vibração) sem nenhum outro aparelho
App web	Wi-Fi (AP próprio) + HTTP + WebSocket	ver o desvio em tempo real, gráfico, 3D, ajustar parâmetros, calibrar
Serial	USB, 115200 baud	diagnóstico da montagem (`diag`), ajustes, gráfico para o TCC (`plot`)
BioSpine V3	USB (mesma serial)	compatibilidade com os comandos e o JSON do projeto BioSpine V3

Todas as interfaces de software rodam no **loop** e chegam ao núcleo pela API `app_*` (`src/app.h`); a exceção é o `diag`, que lê os sensores direto (`sensores()`, `sensores_scanI2C`, `digitalRead` do botão). O diagrama mostra esse caminho.


Loop (Wi-Fi e serial)
web.cpp
HTTP :80 +
WebSocket :81
protocolo.cpp
JSON
console.cpp
comandos serial
API app_*
app_calibrar,
app_testarMotor,
app_definirConfig,
app_status...
Motor GPIO25 + LED
GPIO2
Tarefa de controle
sensores, postura,
motor
pedidos
eventos (fila)
drenarEventos
Botão GPIO27

1. Rede e HTTP

Item	Valor	Constante
Rede (SSID)	`BioSpine`	`WIFI_SSID`
Senha (WPA2)	`colete123`	`WIFI_SENHA`
IP do colete	`192.168.4.1` (padrão do Access Point do ESP32)	—
Página	`http://192.168.4.1` (porta 80)	`HTTP_PORTA`
WebSocket	`ws://192.168.4.1:81/`	`WS_PORTA`
Potência do rádio	11 dBm	`web_iniciar`

Rotas HTTP (as quatro rotas servidas levam `Cache-Control: no-cache`; o redirecionamento 302 não):

Caminho	Conteúdo
`/` e `/index.html`	a página do app (`src/web/index.html`, embutida na flash)
`/manifest.json`	manifest para "Adicionar à tela inicial"
`/icon.svg`	ícone do app
qualquer outro	`302` para `http://192.168.4.1/`

A página calcula a porta do WebSocket como "porta da página + 1" (80 → 81). É isso que permite o `tools/mock_server.py` e o `tools/sim_web.py` servirem a mesma página em 8080 com o WebSocket em 8081.

No celular: conectar no Wi-Fi `BioSpine`, tocar em **Manter conexão** quando o Android avisar "sem internet" e, se a página não carregar, **desligar os dados móveis**.

---

2. Protocolo WebSocket (JSON)

Mensagens de texto, um objeto JSON por mensagem. O ESP32 identifica o tipo pelo campo `t`; o app manda comandos com o campo `cmd`. Definição resumida também no topo de `src/protocolo.h`.

2.1 ESP32 → app: `{"t":"data"}` — 10 Hz

Enviado pelo `loop` a cada `PERIODO_ENVIO_MS` (100 ms) para todos os clientes, montado por `protocolo_dados` a partir do `Status`. Ângulos e porcentagens com 1 casa decimal.

Campo	Tipo	Unidade	Descrição
`t`	texto	—	sempre `"data"`
`pitch`	número	°	frente/trás em relação à referência (positivo = tronco para a frente, com o sensor montado com os componentes para fora)
`roll`	número	°	inclinação lateral em relação à referência
`desvio`	número	°	desvio total (o que decide o alerta)
`flex`	inteiro	ADC 0–4095	média móvel do flex 1
`flexPct`	número	%	curvatura do flex 1 (0 se inativo)
`flexAtivo`	bool	—	flex 1 entra na detecção
`flexDetectado`	bool	—	flex 1 parece ligado

Campo	Tipo	Unidade	Descrição
`flex2`, `flex2Pct`, `flex2Ativo`, `flex2Detectado`	—	—	o mesmo para o flex 2
`fsr`	inteiro	ADC 0–4095	sensor de pressão (só telemetria)
`fsrDetectado`	bool	—	já leu pressão estável desde o boot
`estado`	texto	—	`"ok"`, `"atencao"` ou `"alerta"`
`tempoRuimMs`	inteiro	ms	há quanto tempo a postura está ruim (0 se boa)
`calibrado`	bool	—	existe calibração (sem ela não vibra)
`calibrando`	bool	—	calibração em andamento
`mpuOk`	bool	—	MPU-6050 respondendo
`motor`	bool	—	motor ligado neste instante
`uptime`	inteiro	ms	`millis()` do envio (tempo desde o boot)

Exemplo:

```
{"t":"data","pitch":12.3,"roll":-1.2,"desvio":12.4,"flex":1873,"flexPct":6.5,"flexAtivo":true,"fle
xDetectado":true,"flex2":2051,"flex2Pct":0,"flex2Ativo":false,"flex2Detectado":false,"fsr":0,"fsrD
etectado":false,"estado":"ok","tempoRuimMs":0,"calibrado":true,"calibrando":false,"mpuOk":true,"mo
tor":false,"uptime":81234}
```

2.2 ESP32 → app: `{"t":"config"}`

Enviado **ao conectar** e em resposta a `get` (só para quem pediu), e **depois de toda mudança** (`set`, `reset`, fim da calibração) para todos. Montado por `protocolo_config`.

Campo	Tipo	Unidade	Faixa aceita	Descrição
`t`	texto	—	—	sempre `"config"`
`limiteGraus`	número	°	2–60	limite do desvio
`histerese`	número	° / pontos %	0 até (menor limite − 1)	margem para sair do alerta
`tempoMs`	inteiro	ms	0–60000	tempo mínimo em má postura antes de vibrar
`limiteFlex`	número	%	2–90	limite da curvatura do flex
`intensidade`	inteiro	%	0–100	força do motor no alerta
`flexModo`	texto	—	`auto`, `on`, `off`	modo do flex 1
`flex2Modo`	texto	—	`auto`, `on`, `off`	modo do flex 2
`calibrado`	bool	—	—	existe calibração
`versao`	texto	—	—	`FIRMWARE_VERSAO`

```
{"t":"config","limiteGraus":20,"histerese":3,"tempoMs":3000,"limiteFlex":15,"intensidade":100,"fle
xModo":"auto","flex2Modo":"off","calibrado":true,"versao":"1.0.0"}
```

2.3 ESP32 → app: `{"t":"evt"}` — avisos

Campo	Tipo	Descrição
`t`	texto	sempre `"evt"`
`nivel`	texto	`"info"`, `"ok"` ou `"erro"` (cor do aviso no app)
`msg`	texto	mensagem curta para mostrar como toast

Avisos que o firmware envia:

Situação	`nivel`	`msg`
calibração começou	info	Calibrando... fique parado
calibração concluída	ok	Postura calibrada
calibração recusada (mexeu > 30 °/s)	erro	Mexeu durante a calibração: tente de novo
pedido de calibração sem MPU	erro	MPU-6050 não encontrado
MPU caiu / voltou	erro / ok	MPU-6050 desconectado / MPU-6050 reconectado
`set` aplicado	ok	Parâmetros salvos
`reset` aplicado	ok	Parâmetros padrão restaurados
JSON inválido	erro	comando inválido
`cmd` desconhecido	erro	comando desconhecido

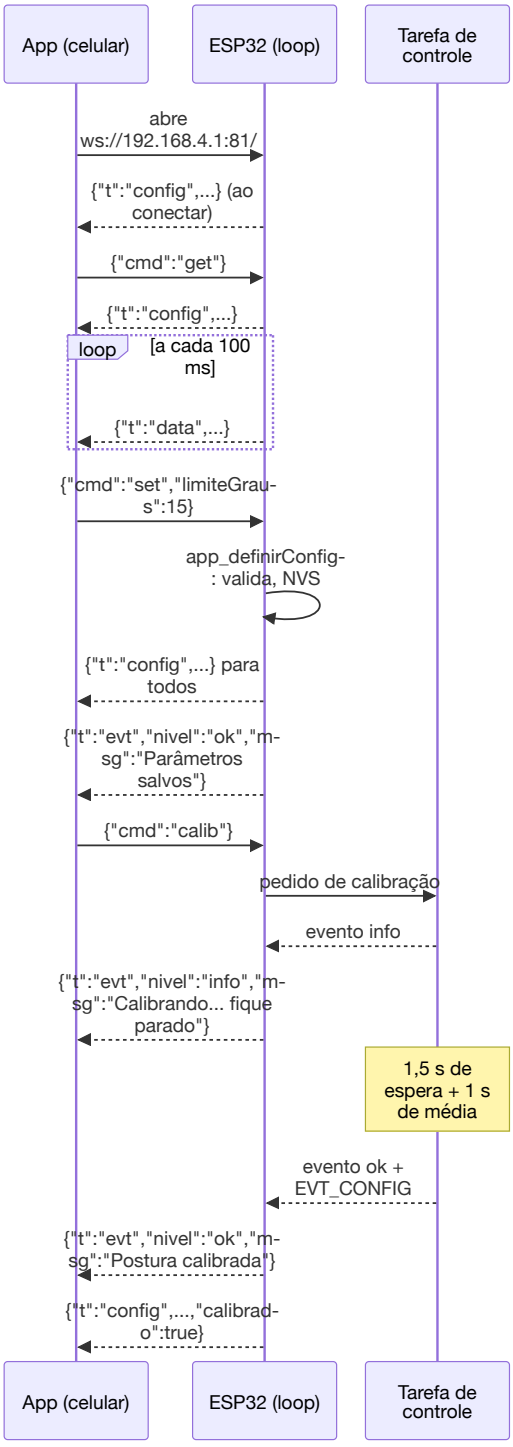
2.4 App → ESP32: comandos

Comando	Campos	Efeito	Resposta
`{"cmd":"get"}`	—	nada	`{"t":"config"}` só para quem pediu
`{"cmd":"set", ...}`	qualquer subconjunto de `limiteGraus`, `histerese`, `tempoMs`, `limiteFlex`, `intensidade` (números) e `flexModo`, `flex2Modo` (texto)	valida (prende nas faixas), aplica e salva na NVS	`{"t":"config"}` para todos + evt "Parâmetros salvos"
`{"cmd":"reset"}`	—	parâmetros e modos de fábrica (a calibração fica)	`{"t":"config"}` para todos + evt
`{"cmd":"calib"}`	—	calibração (igual ao botão)	evts info → ok/erro + `{"t":"config"}`
`{"cmd":"motor_test"}`	—	vibra 1 s	—

Regras do `set`:

- O app manda **só os campos que mudaram** (mensagem curta e não sobrescreve um ajuste feito ao mesmo tempo pela serial ou por outro celular).
- Campo com tipo errado é ignorado; texto inválido em `flexModo`/`flex2Modo` também.
- Valor fora da faixa não dá erro: é preso no limite mais próximo (`validarParams`) e o `config` de volta mostra o valor realmente aplicado.

2.5 Sequência típica



2.6 Conexão e quedas

- **Servidor:** heartbeat do WebSocket (`enableHeartbeat(5000, 2000, 2)`): ping a cada 5 s; dois pongs perdidos (2 s cada) derrubam o cliente. Assim um celular que saiu da rede deixa de receber envios.
- **App:** se o WebSocket fecha, tenta de novo a cada 2 s e mostra "Desconectado". Se ficar 4 s sem receber nada (celular voltando do bloqueio com o socket "zumbi"), fecha e reconecta. Com mais de 1,5 s sem dados, a leitura principal fica marcada como desatualizada. Detalhes em 08 — App web.
- **Colete:** nada disso afeta a detecção, que roda na tarefa de controle (02).

---

3. Serial (115200 baud)

`pio device monitor` (sai com Ctrl+C). Cada linha termina em `\r`, `\n` ou `\r\n`. Os comandos do colete aceitam maiúsculas ou minúsculas.

Comando	Ação
`help` (ou `ajuda`, `?`)	lista os comandos
`diag`	testa cada componente (✅/❌) e pulsa o motor por 1 s
`calib` (ou `calibrar`)	calibra a postura correta (igual ao botão): 1 pulso, fique parado 2,5 s, 2 pulsos = pronto; 1 pulso longo = mexeu, repita
`get`	mostra parâmetros, modo e situação dos flex e a calibração
`set limite 20`	limite do desvio (°, 2–60)
`set histerese 3`	histerese (° / pontos %, menor que os limites)
`set tempo 3000`	tempo mínimo em má postura (ms, 0–60000)
`set flexlim 15`	limite da curvatura do flex (%, 2–90)
`set motor 100`	intensidade do motor (%, 0–100)
`set flex on|off|auto`	modo do flex 1 (`set flex2 ...` para o segundo)
`motor`	testa o motor (1 s)
`plot on` / `plot off`	liga/desliga a saída do Serial Plotter
`status`	uma linha JSON no formato do BioSpine V3
`reset`	parâmetros e modos de fábrica (mantém a calibração)

Tudo que `set` e `reset` alteram é salvo na NVS e avisado ao app.

Mensagens que aparecem sozinhas (eventos da tarefa de controle), no formato `[tag] texto`: `[calib] calibrando...`, `[calib] concluída (N amostras, flex ..., flex2 ...)`, `[calib] recusada: mexeu durante a média (.. °/s)...`, `[mpu] MPU-6050 parou de responder (fio solto?)`, `[mpu] MPU-6050 reconectado`. No boot, `web_iniciar` também imprime direto (sem passar pela fila) `[wifi] AP "BioSpine" senha "colete123" -> abra http://192.168.4.1`.

3.1 `diag`

Saída típica com tudo ligado (montagem do GUIA):

```
I2C scan ........ 0x68 ✅
MPU6050 ......... pitch 2.1° roll -0.4° desvio 2.2° ✅ (0x68, WHO_AM_I 0x68)
Flex (GPIO34) ... 1873 ✅ (dobre o sensor e veja o valor mudar)
Flex 2 (GPIO35) . 1870 desligado (se houver 2º flex: set flex2 auto)
FSR (GPIO32) .... 0 não detectado (normal se não houver FSR; aperte-o para detectar)
Botão (GPIO27) .. solto ✅ (aperte para testar: calibra)
Motor (GPIO25) .. pulsando 1 s — sentiu? (o LED azul da placa acende junto)
Wi-Fi AP ........ BioSpine @ 192.168.4.1 ✅ (1 no app)
Calibração ...... salva ✅
```

O que cada ❌ indica:

Linha	Sintoma	Causa provável
I2C scan	`nada` ❌	SDA/SCL trocados, VCC/GND do MPU soltos
MPU6050	`não responde` ❌	idem; ou MPU fora de 0x68/0x69
MPU6050	`giro sem offset`	placa mexeu no boot; deixe parada 2 s (o offset é recalculado sozinho)

Linha	Sintoma	Causa provável
Flex	`~4095`	falta o resistor de 10 kΩ do divisor
Flex	`~0`	flex solto ou quebrado
Flex	`leitura instável`	mau contato
Botão	`apertado` sem apertar	botão em curto para o GND (pernas erradas)

Com o flex com defeito o colete segue **só com o MPU** (fallback automático no modo `auto`, o padrão; no modo `on` o flex continua na decisão).

3.2 `plot on` (Serial Plotter e gráfico do TCC)

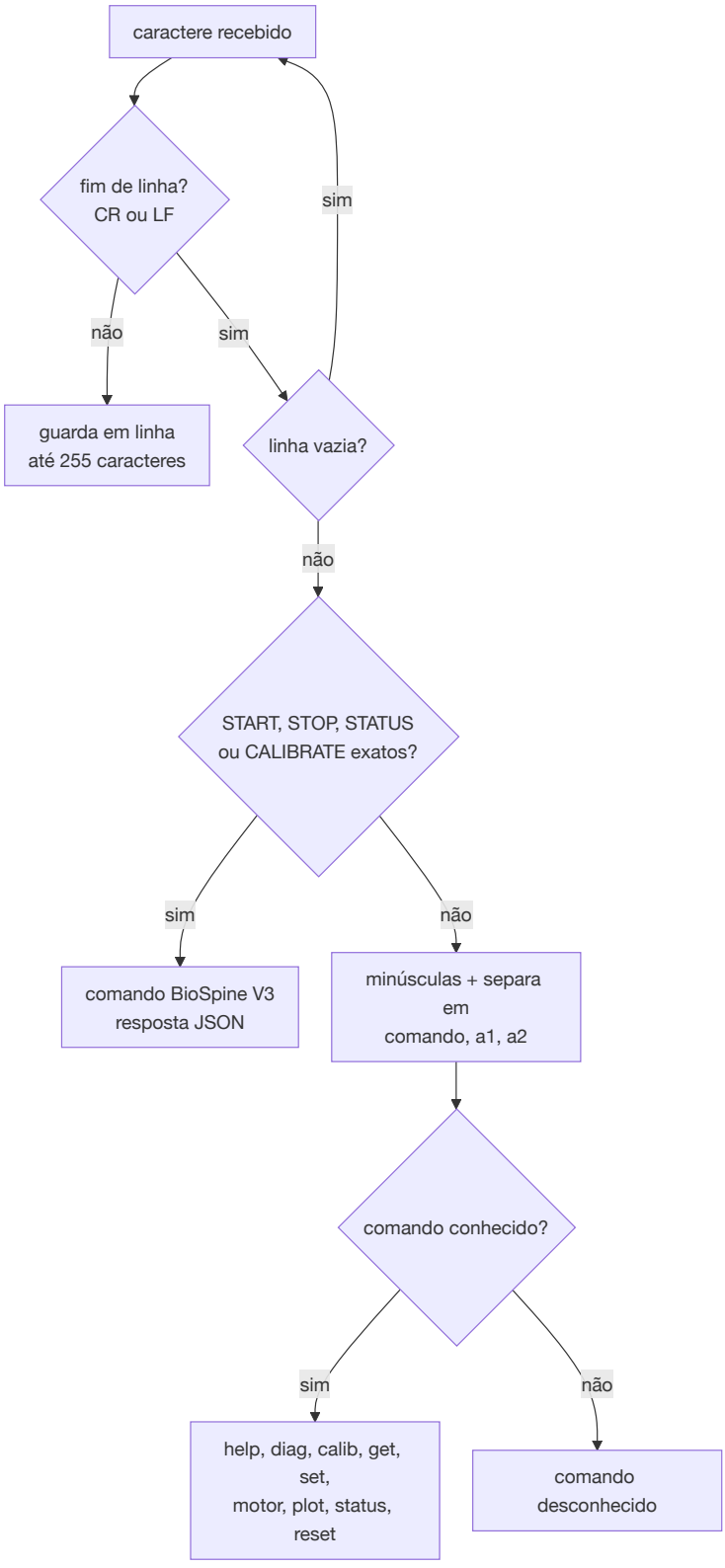
Uma linha a cada 100 ms:

```
desvio:12.4,limite:20.0,pitch:12.3,roll:-1.2,flexPct:6.5,limiteFlex:15.0,alerta:0
```

`alerta` vale 5 durante o alerta e 0 fora dele (só para aparecer como um degrau na mesma escala). É o formato lido pelo Serial Plotter do Arduino e pelo `tools/grafico_serial.py`, que gera CSV + PNG.

3.3 Interpretador

O fluxograma mostra como uma linha recebida é tratada (`console_atualizar` → `executar`).



---

4. Compatibilidade com o BioSpine V3

O firmware aceita os comandos serial do BioSpine V3 (outro projeto, com pinagem compatível) e responde no formato JSON do README dele. Os comandos precisam chegar **em maiúsculas exatas**.

Comando	Resposta
`START`	`{"success":true,"message":"Monitoring started"}` e passa a enviar o JSON de status 1 vez por segundo

Comando	Resposta
`STOP`	`{"success":true,"message":"Monitoring stopped"}` e para o envio contínuo
`STATUS`	uma linha de status (abaixo)
`CALIBRATE`	calibração normal; ao concluir: `{"success":true,"message":"Calibration completed"}`; sem MPU: `{"success":false,"message":"MPU6050 not found"}`

Linha de status:

```
{"angle":12.4,"flex_left":1873,"flex_right":2051,"pressure":0,"status":"GOOD","alert":false,"calibrated":true}
```

Campo	Origem no colete
`angle`	`desvio` (°)
`flex_left`	flex 1 (ADC)
`flex_right`	flex 2 (ADC)
`pressure`	FSR (ADC)
`status`	`GOOD` = ok, `WARNING` = atenção, `BAD` = alerta
`alert`	`true` no estado alerta
`calibrated`	existe calibração

Limitação conhecida: se o `CALIBRATE` for recusado porque a pessoa se mexeu, não sai JSON de falha (só a linha `[calib] recusada...`).

08 — App web embarcado

> Código: `src/web/index.html`, `src/web/manifest.json`, `src/web/icon.svg`, `src/web.cpp` · Relacionados: 02 — Arquitetura, 07 — Protocolo e interfaces, 09 — Simulador, testes e ferramentas, referencia/web.md, referencia/protocolo.md

1. Objetivo

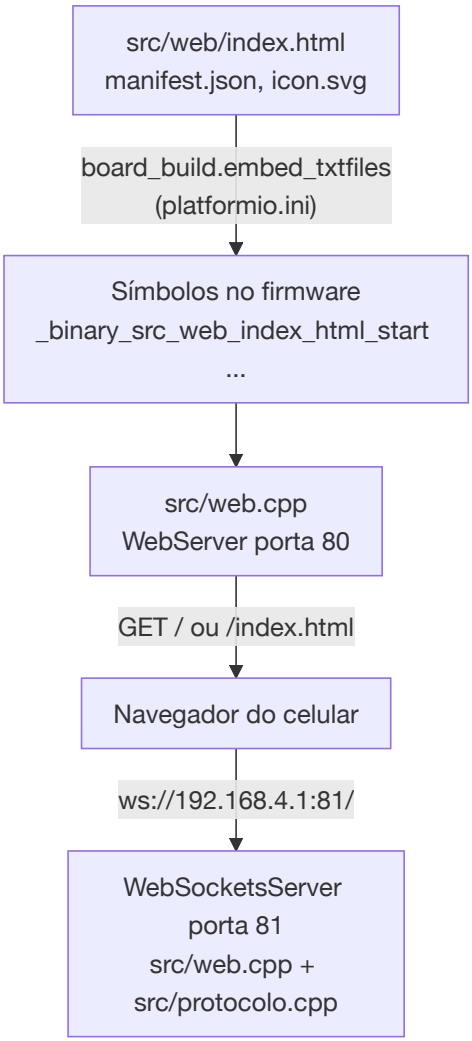
A página é o painel do colete no celular: mostra o desvio em tempo real, o estado (ok, atenção, alerta), os sensores, um gráfico dos últimos 30 s e um manequim 3D; permite calibrar, testar o motor, mudar os ajustes e baixar os dados em CSV para o TCC.

Ela **não decide nada sobre a postura**. Estado, limites, calibração e alerta são calculados no firmware; a página só mostra e manda comandos. Por isso o colete funciona igual com ou sem o celular conectado.

Decisões que moldam o arquivo:

- **Um arquivo só, sem bibliotecas e sem CDN.** Na rede Wi-Fi do colete o celular não tem internet. A fonte de 7 segmentos (subconjunto da DSEG7, licença SIL OFL 1.1) vai embutida em base64 no CSS.
- **Canvas 2D próprio para o gráfico e o 3D.** Nada de WebGL nem Three.js: menos bytes na flash e funciona em qualquer celular.
- **Tudo vai para a flash e para o Wi-Fi.** O arquivo inteiro (com comentários) é gravado no firmware e enviado a cada carga, sem compressão. Comentários curtos; hoje o arquivo tem ~85 KB.

2. Como a página chega ao celular



1. O `platformio.ini` lista os três arquivos em `board_build.embed_txtfiles`. O PlatformIO os grava na flash como texto terminado em `\0` e cria os símbolos `_binary_src_web_index_html_start`, `_binary_src_web_manifest_json_start` e `_binary_src_web_icon_svg_start`.
2. `src/web.cpp` declara esses símbolos (`extern const char index_html[] asm(...)`) e registra as rotas do `WebServer` (porta `HTTP_PORTA` = 80): `/`, `/index.html`, `/manifest.json`, `/icon.svg`. As quatro rotas levam `Cache-Control: no-cache` (o redirecionamento do item 3 não), para o celular não guardar uma página antiga depois de regravar o firmware.
3. Qualquer outro caminho recebe `302` para `http://192.168.4.1/` (útil para caminhos errados no próprio IP, ex.: `/favicon.ico`; não há servidor DNS, então endereços de outros sites não chegam aqui).
4. A página abre o WebSocket na porta `WS_PORTA` = 81. Ao conectar, o firmware manda o `{"t":"config"}` para esse cliente; depois envia `{"t":"data"}` 10 vezes por segundo para todos.

Mudar a página exige **recompilar e regravar o firmware** (`pio run -t upload`). Para desenvolver sem placa, ver a seção 14.

3. Cara de app (PWA)

- `manifest.json`: nome `BioSpine`, `display: standalone` (abre sem a barra do navegador), `start_url: /`, ícone `icon.svg` (`any` e `maskable`), cores de fundo e tema.
- `<meta name="apple-mobile-web-app-capable">` e afins fazem o mesmo no iPhone.

- `<meta name="viewport" ... viewport-fit=cover>` + `env(safe-area-inset-*)` no CSS: a página respeita o entalhe da tela.
- Uso: no navegador, **Adicionar à tela inicial**. Não há service worker: o "app" só abre com o celular conectado ao Wi-Fi do colete (o que é o caso de uso).

4. Estrutura da tela

Em celular, uma coluna; de 920 px para cima, duas colunas (leitura, gráfico e sensores à esquerda, 3D à direita; ajustes embaixo, na largura toda).

Seção (`id`)	O que mostra	Atualizada por
Cabeçalho (`#conn`, `#bTema`)	Marca, botão de tema, indicador de conexão	`conexao`, `aplicarTema`
Leitura principal (`#hero`)	Selo do estado, desvio grande em 7 segmentos, barra de contagem até vibrar, frente/trás, lateral, limite, aviso, botões **Calibrar postura** e **Testar motor**	`hero`, `heroOff`, `aoConfig`
Vista 3D (`#viz`)	Manequim com o colete e os sensores; vistas Costas/Lado/Frente; Expandir; Ocultar 3D; caixas "Inverter frente/trás" e "Inverter lado"	`render` (laço próprio)
Gráfico (`#cg`)	Desvio (°) e flex (%) nos últimos 30 s, limites, faixas de atenção/alerta; botões **Baixar CSV** e **Zerar gravação**	`desenharGrafico`, `gravar`
Sensores (`.sens`)	MPU-6050, flex 1, flex 2 (se habilitado), motor, FSR (se detectado)	`sensores`, `linha`
Ajustes (`#form`)	Sliders de limite, tempo, histerese, limite do flex, intensidade; modos do flex 1 e 2; Salvar, Descartar, Restaurar padrão; versão do firmware	`aoConfig`, `preencher`
Avisos (`#toasts`)	Mensagens temporárias (eventos do firmware e erros locais)	`toast`

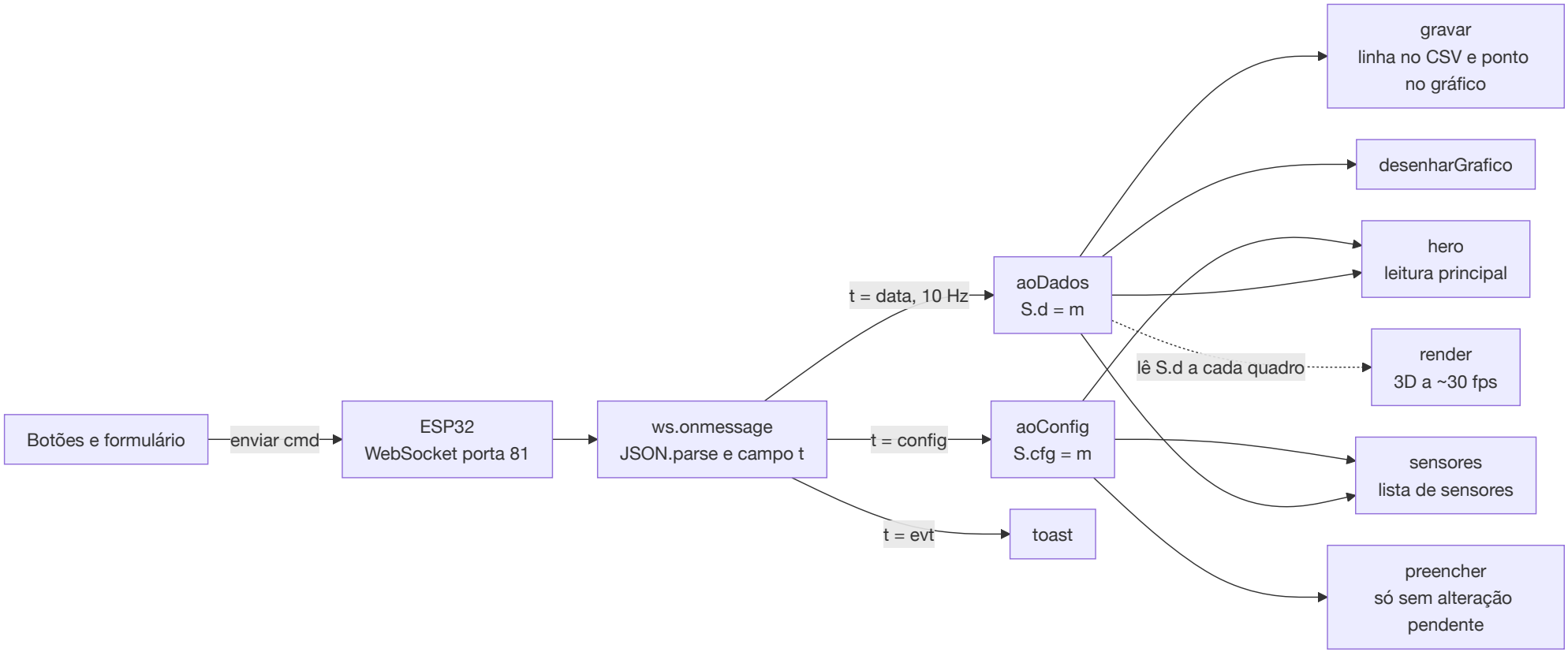
O atributo `data-e` da leitura principal (e do painel do modo expandido) diz o estado mostrado e escolhe a cor: `ok`, `atencao`, `alerta`, `calib` (calibrando), `nocal` (sem calibração), `off` (sem conexão).

5. Estado e fluxo de dados

O estado global fica no objeto `S`:

Campo	Significado
`S.on`	WebSocket aberto
`S.d`	última mensagem `{"t":"data"}` (ou `null` sem conexão)
`S.cfg`	último `{"t":"config"}`
`S.ultimo`	`performance.now()` da última mensagem recebida
`S.invP`, `S.invR`	inverter frente/trás e lado (só exibição)

O diagrama mostra o caminho de cada mensagem dentro da página.

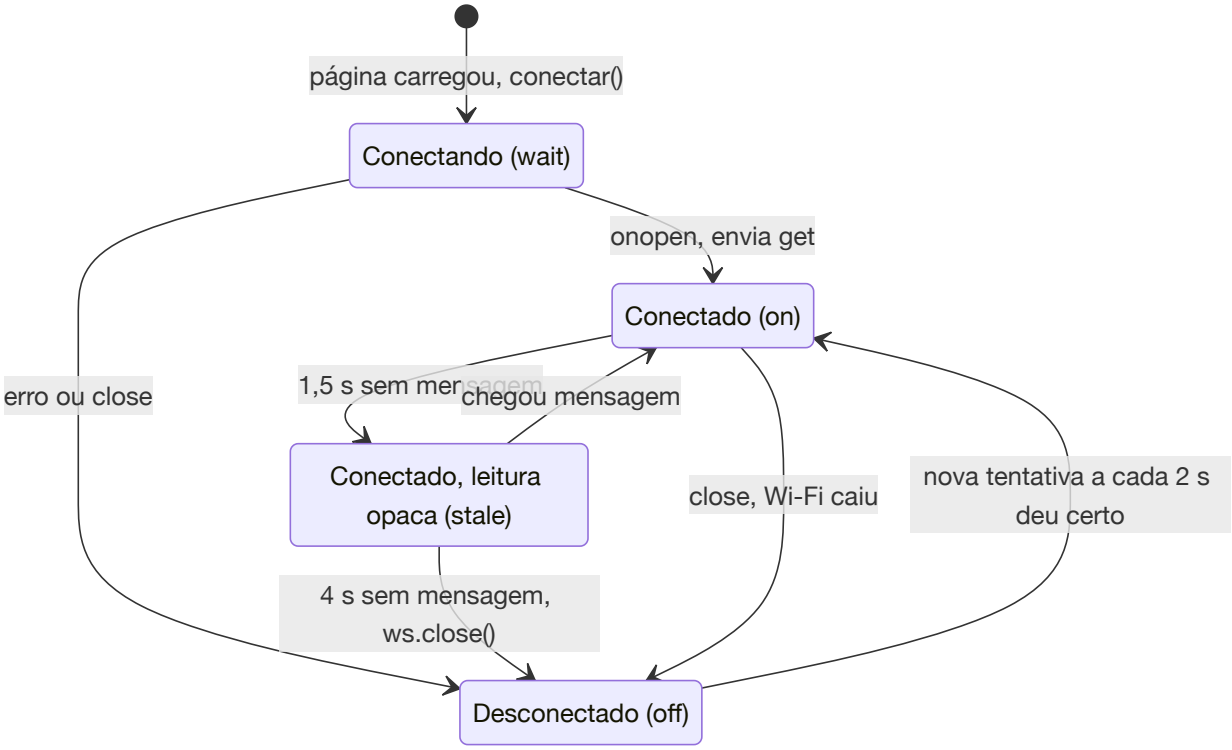

Botões e formulário
enviar cmd
ESP32
WebSocket porta 81
ws.onmessage
JSON.parse e campo t
t = data, 10 Hz
t = config
t = evt
aoDados
S.d = m
aoConfig
S.cfg = m
toast
gravar
linha no CSV e ponto
no gráfico
desenharGrafico
hero
leitura principal
lê S.d a cada quadro
render
3D a ~30 fps
sensores
lista de sensores
preencher
só sem alteração
pendente

Comandos enviados (`enviar`): `{"cmd":"get"}` ao conectar, `{"cmd":"calib"}`, `{"cmd":"motor_test"}`, `{"cmd":"set", ...}` só com os campos alterados e `{"cmd":"reset"}`. O formato completo está em 07 — Protocolo e interfaces e no topo de `src/protocolo.h`.

6. Conexão e reconexão

- **Endereço do WebSocket** (`conectar`): `ws://<host da página>:<porta>/`. Na placa a página vem da porta 80 (`location.port` vazio) e o WebSocket fica na **81**. No simulador ou no mock (página na 8080) o WebSocket fica na **porta seguinte** (8081). Sem host (arquivo aberto do disco), usa `192.168.4.1`.
- **Reconexão:** qualquer erro vira `close` (`onerror` chama `ws.close()`), e `onclose` chama `agendar`, que tenta de novo em **2 s**.
- **Socket "zumbi":** celular que volta do bloqueio pode manter um socket morto. Um temporizador de 500 ms confere `S.ultimo`: com **1,5 s** sem mensagem a leitura fica opaca (classe `stale`); com **4 s**, fecha o socket para reconectar.
- Do lado do ESP32, `ws.enableHeartbeat(5000, 2000, 2)` em `src/web.cpp` derruba clientes que sumiram da rede.

Estados do indicador de conexão:



No estado "leitura opaca" o indicador do topo continua "Conectado": só a leitura principal (`#hero`) recebe a classe `stale` e fica opaca.

Ao desconectar, `conexao(false)` apaga `S.d`, mostra a tela "sem conexão" (`heroOff`) e redesenha o gráfico; os pontos dos últimos 30 s vão saindo e ele fica vazio depois de ~30 s sem conexão. O 3D passa a mostrar os sensores sem estado. Durante as novas tentativas o indicador continua "Desconectado"; o estado "Conectando" só aparece na primeira carga.

7. Leitura principal

- **Selo e frase** (`hero`), em ordem de prioridade: `calibrando` → "CALIBRANDO"; `!calibrado` → "SEM CALIBRAÇÃO" (o colete só vibra depois de calibrado); senão o `estado` do firmware (tabela `EST`: OK, ATENÇÃO, ALERTA).

- **Desvio:** número grande com uma casa e vírgula (`fmt`), `--` sem MPU. Atrás dele, os segmentos apagados (`88,8`), como num instrumento (`mostrarDesvio`).
- **Contagem até vibrar:** no estado `atencao`, a barra mostra `tempoRuimMs / tempoMs` e o texto "Vibra em X s"; no `alerta`, barra cheia e "Vibrando há X s". `tempoRuimMs` vem do firmware (`DetectorPostura::tempoRuimMs`).
- **Frente/trás e lateral:** `pitch` e `roll` em módulo, com a direção em texto ("à frente", "à direita"...; abaixo de 0,5° = "alinhado"). As caixas "Inverter" trocam só o sinal exibido: servem para quando o sensor foi preso virado, sem regravar o firmware. O desvio não tem sinal e não muda.
- **Avisos:** MPU desconectado (borda vermelha) tem prioridade sobre "falta calibrar".
- **Botões:** Calibrar fica desabilitado enquanto `calibrando`; o ícone de Testar motor treme enquanto `motor` é verdadeiro (motor ligado de verdade, inclusive no alerta).

8. Lista de sensores

`estadoFlex` só **descreve** o que o firmware decidiu (o campo `flexAtivo` já vem pronto de `sensorAtivo`, em `src/main.cpp`):

Condição	Led (`data-s`)	Texto
`flexAtivo`	`on` (verde)	"ativo na detecção" ou "ativo, forçado ligado"
modo `off`	`idle` (vazado)	"desligado nos ajustes"
`flexDetectado`, mas não ativo	`idle`	"detectado, calibre (de novo) para usar"
não detectado	`off` (vermelho)	"não detectado"

- O **flex 2** só aparece se estiver ativo, ou se o modo não for `off` e ele for detectado ou forçado (`on`). Motivo: o GPIO35 solto "copia" a tensão do GPIO34, e o flex 2 vem desligado por padrão (ver 05 — Sensores analógicos).
- A barra do flex mostra a curvatura (%) com uma marca vermelha no limite do flex.
- O **FSR** aparece só depois de detectado, como "Sensor extra: pressão" (só telemetria).

9. Gráfico dos últimos 30 s

- Cada `data` vira um ponto em `HIST` (`gravar`): `t` (relógio do celular), `d` = desvio (null sem MPU), `f` = curvatura do flex ativo (flex 1, senão flex 2), `e` = estado. Pontos mais velhos que 30 s (`JANELA`) são descartados.
- `desenharGrafico` roda a cada `data` e no `resize`. Desenha, nesta ordem: faixas amarelas/vermelhas onde o estado foi atenção/alerta; grade com 5 linhas; rótulos de tempo (–30 s ... agora); limites tracejados (vermelho = limite em graus, cinza = limite do flex); curva do flex (cinza) e do desvio (escura, com área).
- Escala vertical automática: no mínimo 30 e 1,4 × o limite, cresce com os dados e arredonda para a dezena.
- `ajustarCanvas` faz o canvas ter a resolução da tela (densidade de pixels limitada a 2×), para as linhas saírem nítidas sem pesar no celular.

10. Gravação em CSV

Desde que a página conecta, cada `data` (10 Hz) vira uma linha em memória (`REG`). **Baixar CSV** gera o arquivo no próprio celular (Blob + link temporário), com o nome `postura_AAAAMMDD_HHMMSS.csv`. O formato é o mesmo lido por `tools/grafico_serial.py --csv` (figura do TCC).

Coluna	Origem	Unidade
`t`	`uptime` do firmware menos o da 1ª amostra	s (2 casas)
`desvio`	`data.desvio`	°

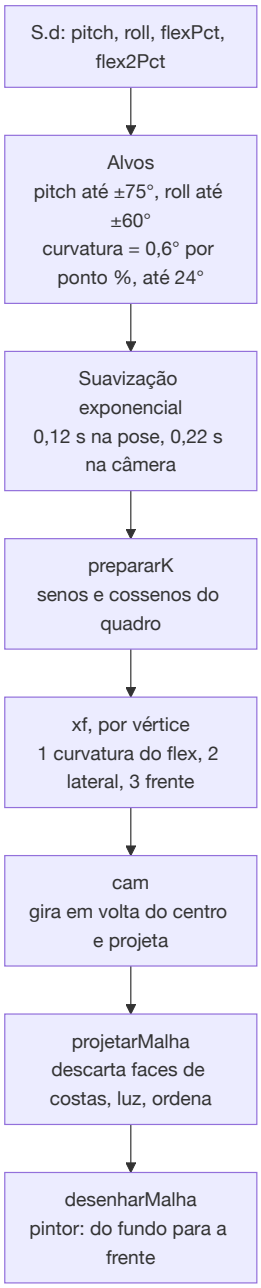
Coluna	Origem	Unidade
`limite`	`config.limiteGraus`	°
`pitch`	`data.pitch` (sem a inversão da página)	°
`roll`	`data.roll` (sem a inversão da página)	°
`flex`	`data.flex`	leitura do ADC (0–4095)
`flexPct`	`data.flexPct`	%
`limiteFlex`	`config.limiteFlex`	%
`estado`	`data.estado`	`ok` / `atencao` / `alerta`
`alerta`	1 se `estado = alerta`, senão 0	—

- A gravação fica **só na memória da aba**: recarregar a página ou fechar o navegador perde os dados. **Zerar gravação** apaga e reinicia o `t`.
- Limite de 200.000 linhas (`MAX_REG`, ~5,5 h a 10 Hz); depois disso as mais antigas saem.

11. Vista 3D

O manequim é desenhado num canvas 2D, sem bibliotecas. É ilustração: os números que valem são os da leitura principal.

Como a inclinação vira pose



- **Coordenadas do corpo** (metros): `x` = direita da pessoa, `y` = altura a partir do quadril, `z` = frente.
- **Malha** (`MESH`, montada uma vez): tronco por seções transversais (`SEC`, superelipse entre elipse e retângulo), cabeça (elipsoide, com cabelo e olhos), braços (tubos). Cada face tem um material: `body`, `vest` (colete, entre as alturas `VEST_Y0` e `VEST_Y1`) ou `hair`.
- **Frente/trás e lateral** (`pitch`, `roll`): giram o tronco inteiro em torno do quadril (`y = 0`). Com o colete calibrado, pose zero = postura de referência.
- **Curvatura do flex:** gira a parte de cima da coluna em torno de um pivô na lombar (`YB` = 0,16 m). Cada vértice tem um peso (`peso(y)`): 0 abaixo de `YB`, 1 acima de `YT` (0,60 m, ombros), com transição suave (smoothstep). Cabeça e braços têm peso 1. A curvatura é **ilustrativa**: o flex mede um número só.
- **Câmera** (`cam`): gira o mundo em yaw e elevação em volta do ponto `(0, ALVO_Y, 0)`, afasta `DIST` e projeta em perspectiva. Botões Costas/Lado/Frente (`VISTAS`) definem a câmera alvo e ela vai até lá pelo caminho mais curto; arrastar gira à mão.
- **Desenho** (`projetarMalha`, `desenharMalha`): faces viradas para trás são descartadas; o brilho é 0,34 de luz ambiente + 0,66 de luz difusa (`LUZ`); as faces são pintadas da mais distante para a mais próxima.

- **Silhueta da postura calibrada:** quando o desvio animado passa de 2°, a malha na pose zero é desenhada translúcida num canvas fora da tela (`fantasma`) e colada atrás do manequim.

Cores e marcadores

Elemento	Aparência	Significado
Colete	cinza (tecido)	sem calibração ou calibrando
Colete	misturado com verde / amarelo / vermelho	estado ok / atenção / alerta; no alerta a cor pulsa
Flex (faixa vertical nas costas)	tira escura com miolo colorido	ativo; miolo verde até 70% do limite, amarelo até o limite, vermelho acima (`corNivel`)
Flex	tracejado cinza	desligado ou ausente
MPU (placa entre as escápulas)	placa escura com LED verde	MPU respondendo
MPU	placa vazada tracejada, LED vermelho	MPU sem resposta
FSR (lombar)	elipse escura, mais forte com mais pressão	só quando detectado
Motor (nuca)	círculo amarelo com ondas e tremido	vibrando agora (campo `motor`)
Rótulos	texto escuro / cinza	sensor ativo / inativo; marcadores atrás do corpo ficam translúcidos e sem rótulo

As posições dos sensores no corpo ficam no objeto `POS`, no topo do script. Se a montagem mudar (por exemplo, dois flex esquerdo e direito no BioSpine), é só ajustar `x` e `y` ali.

Modos e desempenho

- **Expandir:** o painel cobre a tela (e pede tela cheia onde o navegador deixa; o iPhone não deixa). Aparece o painel do canto (`#hud`) com estado e desvio, e os rótulos mostram as leituras de cada sensor. `Esc` ou sair da tela cheia fecha.
- **Ocultar 3D:** esconde o palco; a preferência fica salva.
- **Laço de animação** (`laco`, `atualizarLaco`): `requestAnimationFrame` limitado a ~30 fps (um quadro a cada 32 ms). Para sozinho com o 3D oculto, com o palco fora da tela (`IntersectionObserver`) ou com a aba em segundo plano (`visibilitychange`): poupa bateria do celular.

12. Ajustes

Os `id`s dos campos são os mesmos nomes do JSON de config, e as faixas dos sliders são as mesmas de `validarParams` (`src/posture.cpp`):

Campo (`id`)	Faixa no slider	Passo	Unidade	Regra no firmware
`limiteGraus`	2–60	1	°	2–60
`tempoMs`	0–60000	500	ms (mostrado em s)	até 60000
`histerese`	0 até (menor limite − 1)	0,5	° e pontos %	0 até (menor limite − 1)
`limiteFlex`	2–90	1	%	2–90
`intensidade`	0–100	5	%	até 100
`flexModo`, `flex2Modo`	auto / on / off	—	—	texto inválido é ignorado

- **Validação nos dois lados.** A página limita os sliders e ajusta o máximo da histerese ao mexer nos limites (`ajustarHisterese`), mas o firmware valida tudo de novo (`app_definirConfig` → `validarParams`), porque a

serial e qualquer cliente podem mandar qualquer valor. A histerese precisa ficar abaixo do menor limite, senão a postura nunca voltaria abaixo de `limite − histerese` e o alerta não sairia.

- **Alteração pendente** (`sujo`): ao mexer, Salvar e Descartar ficam habilitados e aparece "Alterações ainda não salvas". Enquanto houver alteração pendente, um `config` que chegar (de outro celular ou da serial) não sobrescreve o formulário.
- **Salvar:** manda `{"cmd":"set"}` **só com os campos que mudaram**, para não desfazer um ajuste feito por outro cliente. O firmware valida, salva na NVS, responde com o evento "Parâmetros salvos" e manda o `config` novo para todos.
- **Descartar:** volta o formulário ao último `config`.
- **Restaurar padrão:** pede confirmação e manda `{"cmd":"reset"}` (parâmetros de `configPadrao`; a calibração é mantida).

13. Tema, acessibilidade e armazenamento local

Tema. Botão alterna automático → claro → escuro. O automático segue o sistema (`prefers-color-scheme`) e acompanha a troca em tempo real. As cores são tokens CSS em `:root`; como os canvas não leem CSS, `lerCores` copia os tokens para o objeto `C` a cada troca. A única cor forte da página é a do estado da postura.

Acessibilidade.

- `role="status"` e `aria-live` no indicador de conexão, no selo e nos avisos; `role="alert"` no aviso de falha; `aria-pressed` e `aria-expanded` nos botões de alternância; `aria-label` nos canvas.
- Alvos de toque com pelo menos 44 px; foco visível (`:focus-visible`); contraste de texto ≥ 4,5:1 e de bordas de controle ≥ 3:1.
- `prefers-reduced-motion`: desliga as animações do CSS.

Armazenamento local (`localStorage`, protegido por `try`, porque o modo privado pode bloquear):

Chave	Valores	Uso
`colete.tema`	`auto`, `light`, `dark`	tema escolhido
`colete.invP`	`0`, `1`	inverter frente/trás
`colete.invR`	`0`, `1`	inverter lado
`colete.3d`	`0`, `1`	3D aberto ou oculto

A gravação do CSV **não** vai para o `localStorage`: fica só na memória.

14. Testar sem a placa

Ferramenta	Quem responde	Uso
`python3 tools/sim_web.py`	**o próprio firmware** compilado para o PC (`sim/`), com MPU falso	conferir a página contra a lógica real. `--demo` mexe o tronco sozinho (bom para gravar a tela). No terminal: `!frente 30`, `!lado -15`, `!flex 1500`, `!botao`, `!mpu off`, `diag`, `set limite 15`…
`python3 tools/mock_server.py`	imitação do ESP32 em Python	desenvolver a página rápido. Opções: `--flex2`, `--fsr`, `--sem-flex`, `--falhas` (um sensor "solta" a cada ~20 s), `--port N`

Os dois servem a página em `http://localhost:8080` e o WebSocket em 8081, a regra "porta da página + 1" de `conectar`. Detalhes em 09 — Simulador, testes e ferramentas.

15. Funções principais do script

Função	O que faz
`conectar`	Abre o WebSocket e liga os tratadores `onopen`, `onclose`, `onerror`, `onmessage`
`agendar`	Nova tentativa de conexão em 2 s
`enviar`	Manda um comando JSON; sem conexão, avisa e retorna `false`
`conexao`	Atualiza o indicador; ao desconectar, limpa a leitura
`aoDados`	Trata `{"t":"data"}`: `gravar`, `hero`, `sensores`, `desenharGrafico`
`aoConfig`	Trata `{"t":"config"}`: guarda, preenche o formulário, atualiza o limite mostrado
`hero` / `heroOff`	Leitura principal com dados / sem conexão
`mostrarDesvio`	Número grande + segmentos apagados
`aviso`	Faixa de aviso da leitura principal
`sensores`, `linha`, `estadoFlex`	Lista de sensores
`preencher`, `ajustarHisterese`, `saidas`, `marcar`	Formulário de ajustes
`toast`	Aviso temporário (3,5 s, no máximo 3)
`gravar`	Linha do CSV e ponto do gráfico
`aplicarTema`, `lerCores`	Tema e cores para os canvas
`ajustarCanvas`	Resolução do canvas conforme a tela
`desenharGrafico`	Gráfico de 30 s
`secAt`, `peso`, `MESH`	Geometria do manequim e peso da curvatura
`prepararK`, `xf`, `cam`	Pose e câmera (corpo → mundo → câmera → tela)
`projetarMalha`, `desenharMalha`, `caminho`	Projeção, descarte de faces, luz e pintura
`costas`, `pontoTela`	Pontos na superfície das costas para os sensores
`desenharMarcadores`, `desenharFlex`, `rotulo`, `corNivel`, `desenharOlhos`	Sensores, rótulos e detalhes do manequim
`corColete`	Cor do colete conforme o estado
`render`	Um quadro do 3D
`laco`, `atualizarLaco`	Laço de animação e pausa automática
`abrir3d`, `expandir`	Ocultar/mostrar e expandir o 3D

16. Armadilhas e cuidados

- **Nomes compartilhados com o firmware.** Os campos do JSON (`desvio`, `flexPct`, `limiteGraus`...) aparecem na página, em `src/protocolo.cpp`, no simulador e no mock. Renomear exige mudar todos (e o comentário do topo de `src/protocolo.h`).
- **`id` dos campos = nome no JSON.** `CAMPOS`, `preencher` e o Salvar dependem disso. Campo novo no config: acrescentar o slider com o mesmo `id`, a entrada em `CAMPOS` e um `<output id="o...">`.
- **Faixas dos sliders** precisam acompanhar `validarParams`; se o firmware mudar uma faixa e a página não, o slider mostra um valor que o firmware vai cortar.
- **Cores em `#rrggbb`.** `lerCores` só entende hexadecimal de 6 dígitos; `#abc` ou `rgb()` nos tokens quebram as cores do canvas.

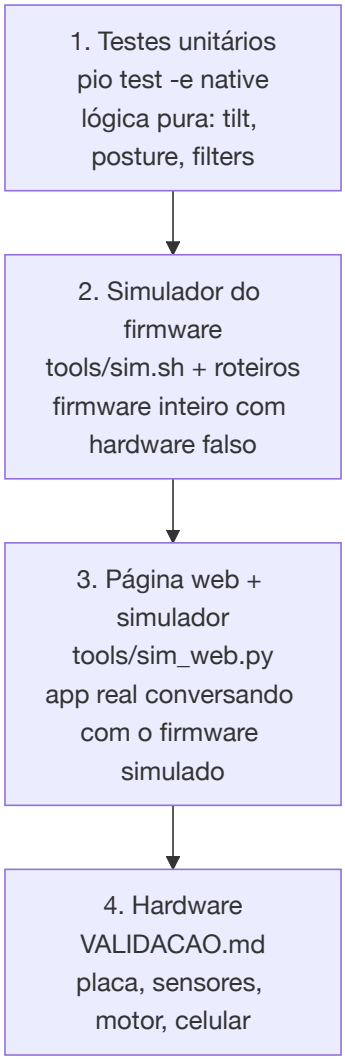
- **Tamanho.** Tudo vai para a flash e para o Wi-Fi a cada carga. Nada de biblioteca externa nem CDN (o celular está sem internet).
- **Comentários.** Nunca escrever `</script>` dentro de comentário JS nem `--` dentro de comentário HTML: quebram a página.
- **Porta do WebSocket.** A regra "porta da página + 1" vale para o simulador e o mock; na placa é sempre 81 (`WS_PORTA`). Mudar a porta no firmware exige mudar `conectar`.
- **Depois de mudar a página:** regravar o firmware; testar com `tools/sim_web.py --demo` antes.

09 — Simulador, testes e ferramentas

> Código: `sim/Arduino.h`, `sim/Wire.h`, `sim/Preferences.h`, `sim/sim_main.cpp`, `test/test_posture/test_main.cpp`, `tools/sim.sh`, `tools/roteiro.txt`, `tools/roteiro_biospine.txt`, `tools/roteiro_sem_mpu.txt`, `tools/sim_web.py`, `tools/mock_server.py`, `tools/grafico_serial.py`, `tools/gerar_bin.sh`, `platformio.ini` · Relacionados: 02 — Arquitetura, 08 — App web, MANUTENCAO, `VALIDACAO.md` (raiz do repositório)

1. Por que existe

O hardware só ficou disponível no último dia, montado por outra pessoa. Todo o software foi escrito e testado **sem a placa**. Para isso o projeto tem quatro camadas de verificação, da mais barata para a mais completa, e só a última precisa do colete:



Cada camada pega um tipo de erro: os testes provam as contas e as regras (ângulo, filtro, histerese, tempo mínimo); o simulador prova a integração (setup/loop, comandos, calibração, motor, NVS, JSON); a página contra o simulador prova o app; o hardware prova o que nada disso consegue simular (seção 9).

Ferramenta	O que faz	Precisa de
`pio test -e native`	roda os 21 testes da lógica pura no PC	PlatformIO
`tools/sim.sh [roteiro]`	compila o firmware para o PC e executa um roteiro em tempo virtual	`g++` e ArduinoJson baixado pelo PlatformIO

Ferramenta	O que faz	Precisa de
`python3 tools/sim_web.py [--demo]`	página real (`src/web/`) + firmware simulado em `http://localhost:8080`	o mesmo do `sim.sh` + Python 3
`python3 tools/mock_server.py`	página real + colete **reescrito em Python** (sem firmware)	só Python 3
`tools/grafico_serial.py`	captura o `plot on` da serial (ou lê um CSV) e gera CSV + PNG para o TCC	pyserial, matplotlib
`tools/gerar_bin.sh`	gera `dist/colete-postural.bin`, imagem única gravável em `0x0`	PlatformIO

---

2. Testes unitários (`test/test_posture/test_main.cpp`)

Como rodam

O ambiente `native` do `platformio.ini` compila para o PC, com o framework de testes **Unity**, só o código que não depende do Arduino:

```
[env:native]
platform = native
test_framework = unity
test_build_src = yes
build_src_filter = -<*> +<tilt.cpp> +<posture.cpp>
build_flags = -std=c++11 -Wall
```

- `build_src_filter` tira tudo de `src/` e põe de volta só `tilt.cpp` e `posture.cpp`. `filters.h` é só cabeçalho e entra pelo `#include` do teste.
- Funciona porque a lógica pura foi separada do hardware de propósito (02 — Arquitetura).
- No ambiente da placa (`esp32dev`), `test_ignore = *` impede que os testes sejam gravados nela.

```
pio test -e native      # esperado: 21 Tests 0 Failures 0 Ignored
```

Sem o PlatformIO, dá para compilar à mão com os fontes do Unity (`unity.c`, `unity.h`, `unity_internals.h`, repositório ThrowTheSwitch/Unity) numa pasta `<unity>`:

```
gcc -c -std=c99 -I<unity> <unity>/unity.c -o unity.o
g++ -std=c++11 -Wall -Isrc -I<unity> test/test_posture/test_main.cpp src/tilt.cpp src/posture.cpp
unity.o -o testes
./testes
```

Modelo usado nos testes

`gravidadeColete(frente, lado)` gera o vetor gravidade que um sensor **em pé nas costas** (eixo Y para cima, componentes para fora) mediria com o tronco inclinado:

```
g = ( sen(lado),  cos(frente)·cos(lado),  sen(frente)·cos(lado) )
```

Inclinar para a frente leva a gravidade para +Z; para o lado, para X. É o mesmo modelo do MPU falso do simulador (seção 3.4). `entrada(desvio)` monta a entrada do detector já calibrada, com os dois flex inativos.

Os 21 testes

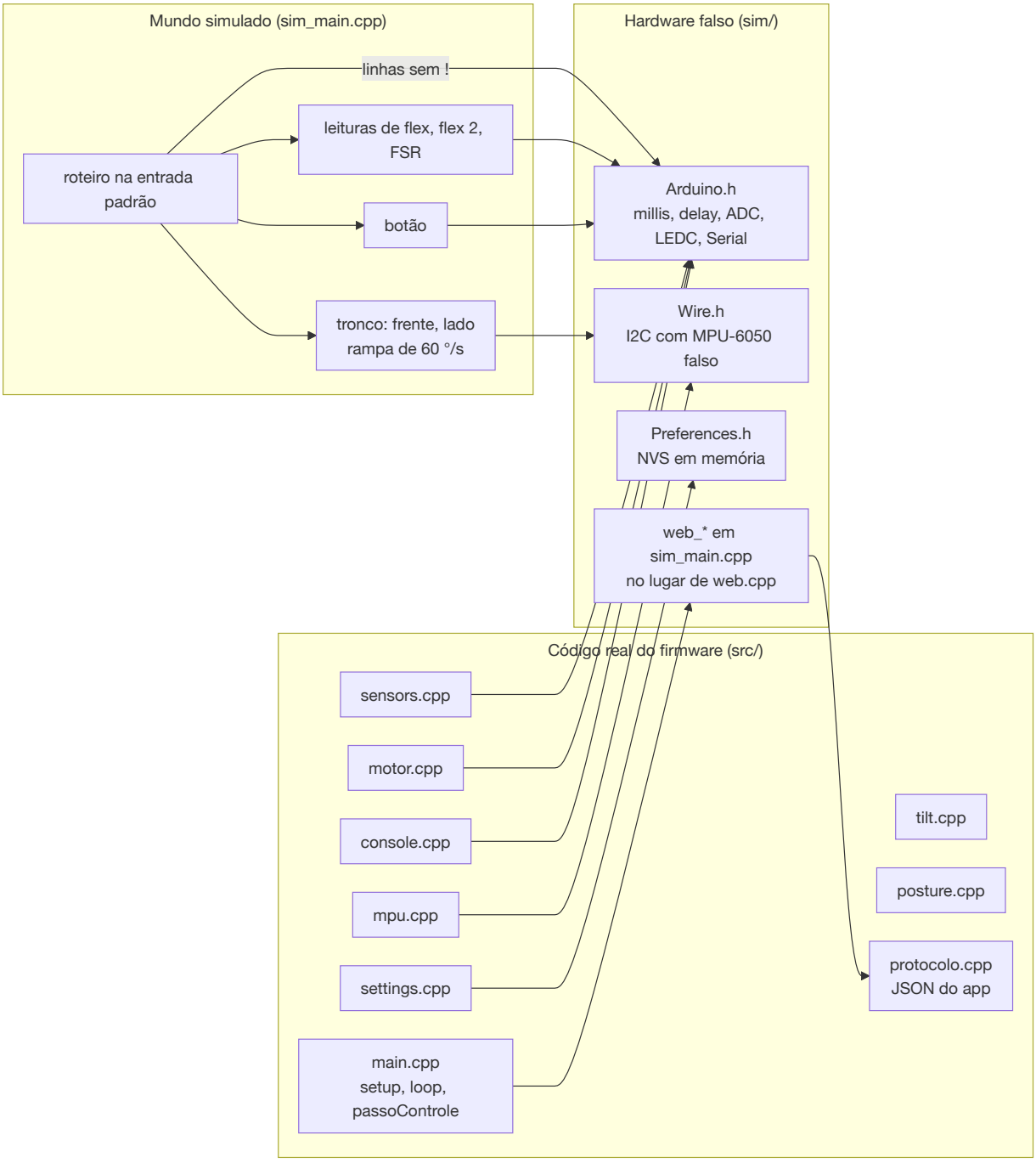
#	Teste	O que prova	Código testado	Requisito / decisão
1	`test_angulo_entre_vetores`	ângulo certo em 0°, 90°, 30° e preciso em 0,5°	`angleBetweenDeg`	base do desvio (03)
2	`test_desvio_sem_salto_perto_da_vertical`	sensor em pé, 10° de lado, passando pela vertical: o desvio varia suave (sem o salto de ~20° da MPU6050_light)	`angleBetweenDeg`	não usar a MPU6050_light (10)
3	`test_componentes_frente_e_lado`	frente e lado separados sem misturar eixos	`tiltComponentsDeg`	exibição frente/trás e lateral no app
4	`test_componentes_com_sensor_de_cabeca_para_baixo`	placa girada 180°: "frente" continua positivo	`tiltComponentsDeg`	independe de como o sensor foi preso
5	`test_componentes_com_placa_deitada`	placa deitada (bancada) não gera NaN	`tiltComponentsDeg`	robustez
6	`test_filtro_converge_para_acelerometro`	giroscópio parado: converge ao acelerômetro (< 0,5° em 10 s)	`TiltFilter::update`	correção de deriva do filtro complementar
7	`test_filtro_segue_giroscopio`	choque (|a| = 2 g) ignora o acelerômetro; -90 °/s em X por 0,2 s = 18° para +Z	`TiltFilter::update`	rejeição de aceleração forte; convenção `dg/dt = g × ω`
8	`test_filtro_reinicia_com_dt_grande`	`dt` > 0,5 s reinicia no acelerômetro	`TiltFilter::update`	robustez (MPU reconectado, atraso)
9	`test_sem_calibracao_nunca_alerta`	80° por 10 s sem calibração: continua "ok"	`DetectorPostura`	sem calibração não vibra
10	`test_atencao_e_depois_alerta`	"atenção" acima do limite; "alerta" só ao completar 3000 ms	`DetectorPostura`	tempo mínimo
11	`test_histerese`	em alerta, 18° mantém; 16,9° (< 20 − 3) volta a "ok"	`DetectorPostura`	histerese
12	`test_pico_curto_nao_alerta`	pico de 0,5 s não vibra; a contagem recomeça do zero	`DetectorPostura`	tempo mínimo
13	`test_flex_dispara_e_respeita_histerese`	flex a 20 % alerta com desvio ok; sai só abaixo de 12 %	`DetectorPostura`	flex entra no "OU" da má postura
14	`test_flex_inativo_e_ignorado`	flex inativo marcando 100 % não alerta	`DetectorPostura`	fallback: colete segue só com o MPU
15	`test_flex2`	só o 2º flex dobrado (30 %) já dá "atenção"	`DetectorPostura`	montagem com dois flex (BioSpine V3)
16	`test_flex_pct`	% de queda (2048 → 1536 = 25 %), nunca negativo, 0 sem referência	`flexPct`	curvatura do flex (05)
17	`test_validar_params`	valores absurdos presos nas faixas; histerese < limites; NaN tratado	`validarParams`	entrada da serial/app não quebra o colete
18	`test_calibrador_media`	espera 1,5 s, coleta 1 s, média do flex, maior giro, gravidade média	`Calibrador`	calibração (04)
19	`test_media_movel`	média com janela incompleta e cheia, amplitude, última amostra	`MediaMovel`	filtro do flex (05)
20	`test_presenca_com_persistencia`	presente só após 2 s válidos seguidos; ausente após 1 s inválido	`Presenca`	flex não "pisca" com ruído
21	`test_leitura_adc_valida`	rejeita sem resistor (~4095), flex solto (~0) e pino flutuando	`leituraAdcValida`	detecção automática do flex

Teste novo: escreva a função em `test_main.cpp`, acrescente o `RUN_TEST` no `main()` e uma linha nesta tabela.

3. Simulador do firmware (`sim/` + `tools/sim.sh`)

3.1 O que é real e o que é falso

O simulador compila o **firmware de verdade** para o PC e troca só o que toca o hardware. A troca é feita pelos caminhos de include: `-Isim` vem antes de `-Isrc`, então `#include <Arduino.h>`, `<Wire.h>` e `<Preferences.h>` caem nos arquivos falsos de `sim/`.



Peça	No ESP32	No simulador
Tempo	`millis()` / `micros()` do relógio da placa	relógio **virtual** (`relogioUs`), só anda quando o simulador manda
Tarefas	tarefa de controle (FreeRTOS) + `loop()`	com `-DSIMULADOR`, o `loop()` chama `passoControle()`: tudo numa linha só (02 — Modo simulador)
MPU-6050	I2C real	`TwoWire` falso: responde no endereço de `!mpuaddr`, gera aceleração e giro a partir de `frente` / `lado`

Peça	No ESP32	No simulador
ADC	`analogRead` real	valor pedido pelo roteiro + ruído pequeno
Botão	GPIO27	`!botao` (apertado por 200 ms)
Motor	PWM LEDC no GPIO25	`ledcWrite` imprime `motor LIGA` / `motor desliga` com o instante
NVS	flash	mapa em memória (vale só durante a execução)
Wi-Fi / app	`web.cpp` (AP, HTTP, WebSocket)	`web_*` de `sim_main.cpp`: o JSON do `protocolo.cpp` sai como linhas `WS>{...}`

3.2 Compilação

`tools/sim.sh` chama o `g++` diretamente:

```
g++ -std=c++17 -O1 -Wall -Wno-unused-parameter -DSIMULADOR -Isim -Isrc -
I.pio/libdeps/esp32dev/ArduinoJson/src \
    src/main.cpp src/console.cpp src/sensors.cpp src/motor.cpp src/settings.cpp \
    src/mpu.cpp src/tilt.cpp src/posture.cpp src/protocolo.cpp sim/sim_main.cpp \
    -o .pio/sim/colete_sim
```

- `-DSIMULADOR`: em `main.cpp` as macros `TRAVAR()`/`DESTRAVAR()` ficam vazias e a tarefa de controle não é criada.
- `src/web.cpp` fica de fora (depende de `WiFi.h`); as funções `web_*` são implementadas em `sim_main.cpp`.
- O ArduinoJson vem de `.pio/libdeps/`, criado pelo PlatformIO no primeiro `pio run`. Sem PlatformIO, baixe o cabeçalho único `ArduinoJson-v7.2.1.h` (versão do `platformio.ini`) e troque o `-I` por a pasta onde ele está, salvo como `ArduinoJson.h`.
- Arquivos novos em `src/` que o firmware precise têm que entrar nesta lista também.
- Função nova do Arduino usada pelo firmware: declarar em `sim/Arduino.h` e implementar em `sim_main.cpp`.

```
tools/sim.sh                              # roteiro padrão (tools/roteiro.txt)
tools/sim.sh tools/roteiro_biospine.txt   # outro roteiro
tools/sim.sh --so-compilar                # só gera .pio/sim/colete_sim
```

3.3 Tempo virtual

- `millis()` e `micros()` leem `relogioUs`. `delay(ms)` não dorme: avança o relógio (e a física) em `ms`. Por isso a calibração do giroscópio no boot (200 leituras com `delay(5)`) consome 1 s virtual.
- `rodar(ms)` executa um `loop()` e avança 1 ms, repetidamente: ~1 kHz, como a tarefa de controle no ESP32 (`vTaskDelay(1)`).
- Um roteiro de vários minutos roda em segundos.
- A semente do ruído é fixa (`srand(42)`): a mesma entrada produz **a mesma saída**, linha por linha. Dá para salvar a saída de um roteiro e comparar depois de uma mudança (`diff`).

3.4 Física do tronco e MPU-6050 falso

Tronco. `!frente 25` não muda o ângulo de uma vez: muda o **alvo**. A cada passo de 1 ms, `passoFisica` anda no máximo `RAMPA_DPS · dt` (60 °/s) até o alvo e guarda a velocidade do passo (`taxaFrente`, `taxaLado`). Assim o giroscópio falso mede uma velocidade realista e o filtro complementar é exercitado como no corpo.

MPU falso (`TwoWire::requestFrom`). A cada leitura monta um mapa de registradores:

Registradores	Conteúdo	Escala
`0x3B..0x40` acelerômetro X, Y, Z	`sen(lado)`, `cos(frente)·cos(lado)`, `sen(frente)·cos(lado)` + ruído ±0,01 g	16384 LSB/g (±2 g)

Registradores	Conteúdo	Escala
`0x43..0x48` giroscópio X, Y, Z	`-taxaFrente + 0,8`, `-0,5`, `taxaLado + 0,3` °/s + ruído ±0,2 °/s	131 LSB/(°/s) (±250 °/s)
`0x75` WHO_AM_I	`0x68` (sempre, mesmo com `!mpuaddr 69`)	—

- Os valores fixos do giroscópio (0,8 / –0,5 / 0,3 °/s) imitam o **offset de fábrica**, que `Mpu6050::calibrarGiro` precisa medir no boot e descontar.
- O sinal do giroscópio segue `dg/dt = g × ω`, a mesma convenção de `TiltFilter::update`. Por isso o simulador **não prova** que o sinal do sensor real está certo (seção 9).
- As escritas de configuração (reset, escalas, filtro passa-baixa) são aceitas e ignoradas.
- `!mpu off` faz o I2C responder NACK: testa a detecção de queda e a reconexão a cada 2 s.

ADC falso (`analogRead`): flex 1 = valor de `!flex` ± 8; FSR = valor de `!fsr` ± 5; flex 2 "solto" (padrão) devolve qualquer valor de 0 a 4095 a cada leitura, imitando o pino flutuando, que o firmware precisa rejeitar pela amplitude da janela (`ADC_RUIDO_MAX`).

3.5 Linguagem dos roteiros

Um roteiro é um arquivo de texto lido pela entrada padrão, uma linha por comando:

Linha	Efeito
vazia, ou `#` **na 1ª coluna**	ignorada (comentário). `# texto` com espaço antes **vai para a serial**
sem `!` (ex.: `diag`, `calib`, `set limite 15`, `STATUS`)	digitada na serial do firmware; o simulador roda 50 ms para o console responder
`!espera 3000`	avança 3000 ms de tempo virtual
`!frente 25` / `!lado -10`	novo alvo de inclinação do tronco (°), alcançado a 60 °/s
`!flex 1800`	leitura do flex 1 (GPIO34), 0–4095
`!flex2 1800` / `!flex2 solto`	liga um 2º flex (GPIO35) com essa leitura / volta ao pino flutuando
`!fsr 300`	leitura do FSR (GPIO32)
`!botao`	aperta o botão de calibração por 200 ms
`!mpu off` / `!mpu on`	desconecta / reconecta o MPU
`!mpuaddr 69`	endereço I2C do MPU em hexadecimal (padrão 68)
`!estado`	imprime uma linha com o `Status` (ver abaixo)
`!eco texto`	imprime `=== texto ===` (título de bloco)
`!ws on` / `!ws off`	mostra / esconde o JSON que iria para o app (linhas `WS>`)
`!ws conectar`	simula um app conectando: recebe o `{"t":"config"}` e liga as linhas `WS>` (como `!ws on`)
`!ws {"cmd":"set","limiteGraus":15}`	comando vindo do app, pelo `protocolo_comando` de verdade
`!boot`	opcional: as linhas **antes** dele montam o hardware antes do `setup()`

Antes do `!boot` use só comandos que mudam o hardware (`!mpu`, `!mpuaddr`, `!flex`, `!flex2`, `!fsr`, `!frente`, `!lado`): `!espera` e `!botao` chamariam `loop()` antes do `setup()`.

Lendo a saída:

Linha	Origem
`> diag`	eco de uma linha mandada para a serial
`[sim 22.410 s] motor LIGA (duty 255)`	transição do motor, com o instante virtual

Linha	Origem
`[sim 21.070 s] desvio 27.0° frente 27.0° lado 0.0° flex 1999 (0.0%) ... atencao cal 1 mpu 1`	`!estado`: desvio, frente, lado, flex (`inativo` se fora da decisão), flex 2, estado, calibrado, MPU ok
`[calib] ...`, `[mpu] ...`	eventos do firmware para o monitor serial (`console_evento`)
`[app:ok] Postura calibrada`	evento que iria para o app (com `!ws off`)
`WS>{"t":"data",...}`	JSON que iria pelo WebSocket (com `!ws on`)

3.6 Os três roteiros

Roteiro	Montagem	O que cobre
`tools/roteiro.txt` (padrão)	GUIA: MPU 0x68, flex 1, sem flex 2 e FSR	boot e `diag`; sem calibração não vibra; calibração pelo botão; pico curto não vibra; atenção → alerta com pulsos 300/700 ms; histerese (18° mantém, 16° solta); alerta só pelo flex (25 %); flex desconectado vira inativo; `set limite`/`set tempo`/`reset`; MPU cai e volta; comandos BioSpine (`STATUS`, `CALIBRATE`, `START`/`STOP`); `plot on`; calibração recusada por movimento (60 °/s)
`tools/roteiro_biospine.txt`	BioSpine V3: MPU 0x69, dois flex, FSR, sem botão	detecção em 0x69; `set flex2 auto`; flex 2 e FSR detectados; calibração pela serial grava as duas referências; alerta só pelo flex 2 (30 %); `STATUS` com `pressure`
`tools/roteiro_sem_mpu.txt`	MPU e flex soltos no boot	boot não trava; `diag` mostra ✗; calibração negada sem MPU; MPU encaixado com a placa ligada é reconectado; calibração passa a funcionar só com o MPU

Trechos da saída esperada do roteiro padrão:

```
=== corcunda por 5 s: atenção e depois alerta com pulsos 300/700 ===
[sim   21.070 s] desvio  27.0° frente  27.0° lado   0.0° ... atencao cal 1 mpu 1
[sim   22.410 s] motor LIGA (duty 255)
[sim   22.710 s] motor desliga
[sim   23.410 s] motor LIGA (duty 255)
...
=== calibração recusada: mexeu durante a média ===
[calib] recusada: mexeu durante a média (60 °/s). Tente de novo, parado.
```

Cada bloco dos roteiros tem, em comentário, o resultado esperado.

3.7 Modo `--tempo-real`

`colete_sim --tempo-real` não lê o roteiro inteiro antes: o relógio virtual acompanha o relógio do PC (`clock_gettime`) e cada linha que chega na entrada é executada na hora. Em cada volta: `select()` sem espera verifica a entrada; roda quantos `loop()` forem precisos para alcançar o relógio real (no máximo 200 por volta) e dorme 1 ms. `!espera` é ignorado nesse modo. É o modo usado pelo `tools/sim_web.py`.

---

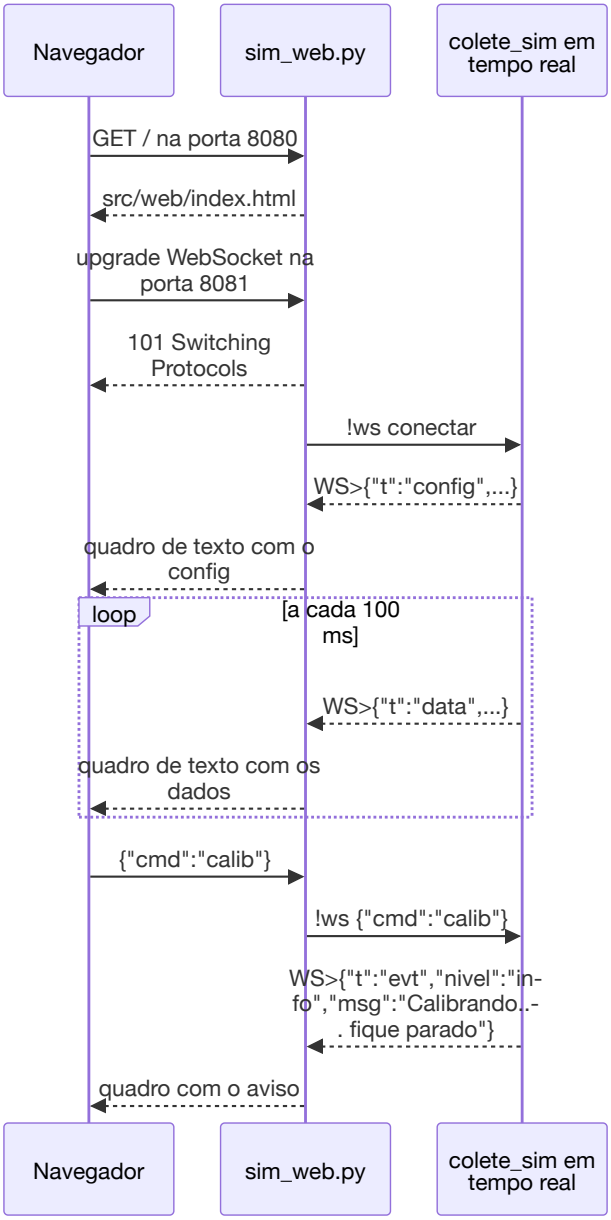
4. Página web sem placa: `sim_web.py` × `mock_server.py`

Os dois servem a página de verdade (`src/web/index.html`, `manifest.json`, `icon.svg`) em `http://localhost:8080` e um WebSocket em `8081`. A página escolhe a porta do WebSocket como "porta da página + 1" (no ESP32: página na 80, WebSocket na 81 — função `conectar()` do `index.html`).

	`tools/sim_web.py`	`tools/mock_server.py`
Quem responde	o **firmware** compilado (`colete_sim --tempo-real`)	uma classe Python (`Colete`) que **reimplementa** a lógica
Corpo	comandos do terminal (`!frente 30` ...) ou `--demo`	inventado: respiração, balanço, episódios de "corcunda"

	`tools/sim_web.py`	`tools/mock_server.py`
Quando usar	conferir app + firmware juntos, gravar vídeo da tela	desenhar a página sem compilar nada
Opções	`--demo`, `--porta`, `--host`	`--flex2`, `--fsr`, `--sem-flex`, `--falhas`, `--port`
Cuidado	precisa do `sim.sh` compilando	se a lógica do firmware mudar, a classe `Colete` fica desatualizada

Ponte do `sim_web.py`, do carregamento da página a um comando:



- Linhas do simulador que **não** começam com `WS>` são a serial do firmware e aparecem no terminal.
- O que for digitado no terminal vai para o simulador: comandos `!` e comandos da serial (`diag`, `get`...).
- `--demo` repete em laço uma sequência de movimentos (calibra, inclina, corcunda, lado, flex).

WebSocket mínimo. Os dois scripts implementam só o necessário da RFC 6455, com a biblioteca padrão:

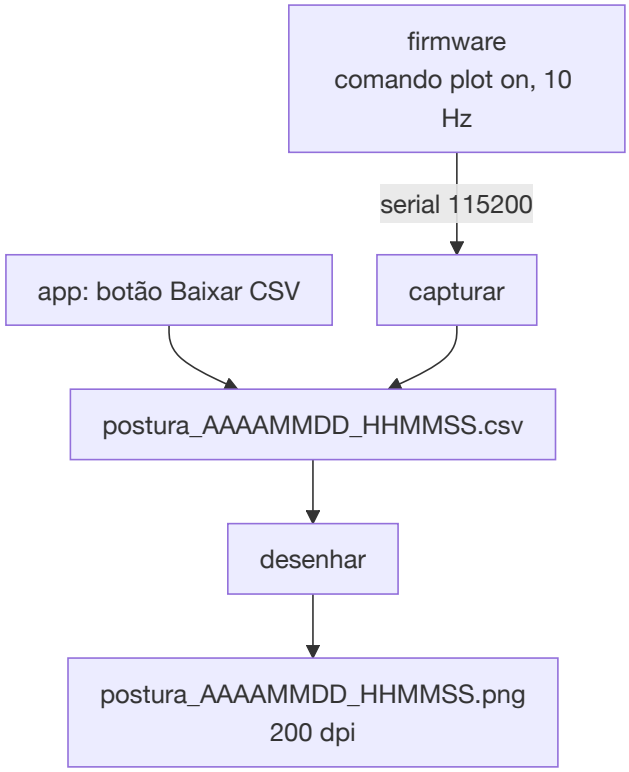
1. *Handshake:* o navegador manda `Sec-WebSocket-Key`; o servidor responde `101` com `Sec-WebSocket-Accept = base64(SHA-1(chave + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"))`.

2. *Quadros:* 1º byte = FIN + opcode (`0x1` texto, `0x8` fechar, `0x9` ping, `0xA` pong); 2º byte = bit de máscara + tamanho (126/127 = tamanho estendido em 2/8 bytes).
3. *Máscara:* quadros do navegador vêm mascarados (XOR com 4 bytes); os do servidor vão sem máscara.

Modelo do `mock_server.py`. A cada 100 ms (`Colete.passo`): calcula o corpo (`corpo`), lê sensores falsos (`adc`), detecta presença (`detectar`, versão simplificada da `Presenca`), aplica a mesma regra do `DetectorPostura` (limite, histerese, tempo mínimo) e monta o `{"t":"data"}` com os mesmos campos do `protocolo_dados`. Diferenças conhecidas: o desvio é aproximado por `hypot(pitch, roll)` em vez do ângulo entre vetores; a detecção de flex não olha a amplitude da janela; a calibração é recusada se houver um episódio de corcunda em andamento (no firmware, se o giro passar de 30 °/s).

---

5. Gráfico para o TCC: `tools/grafico_serial.py`



- **Captura:** abre a porta, manda `plot on`, lê as linhas `desvio:..,limite:..,pitch:..,roll:..,flexPct:..,limiteFlex:..,alerta:..` (geradas por `console_enviarDados`), grava com a coluna `t` (segundos desde o início) e manda `plot off` no fim. Outras linhas (eventos, respostas) só aparecem no terminal.
- **CSV do app:** colunas `t,desvio,limite,pitch,roll,flex,flexPct,limiteFlex,estado,alerta`; a coluna de texto `estado` é ignorada. `alerta` vale 5 no `plot on` (para aparecer no Serial Plotter) e 1 no CSV do app; o gráfico usa só `alerta > 0`.
- **Figura:** desvio × limite, faixa laranja onde o motor estava em alerta e, se o flex marcou alguma curvatura, um 2º gráfico com a curvatura do flex × limite do flex.

```
~/.venvs/colete/bin/python tools/grafico_serial.py -t 60           # captura 60 s
~/.venvs/colete/bin/python tools/grafico_serial.py --csv dados.csv  # só redesenha
~/.venvs/colete/bin/python tools/grafico_serial.py -p /dev/cu.usbserial-0001 -t 60   # macOS
```

A porta automática procura só `/dev/ttyUSB*` e `/dev/ttyACM*` (Linux). Feche o monitor serial antes: a porta só aceita um programa por vez.

---

6. Imagem para gravar: `tools/gerar_bin.sh`

Compila (`pio run -e esp32dev`) e junta, com o `esptool.py merge_bin`, as quatro partes que o ESP32 espera na flash de 4 MB:

Endereço	Arquivo	Conteúdo
`0x1000`	`bootloader.bin`	2º estágio do boot
`0x8000`	`partitions.bin`	tabela de partições
`0xe000`	`boot_app0.bin`	qual partição de app iniciar
`0x10000`	`firmware.bin`	o firmware, com a página web embutida

Saídas em `dist/`:

- `colete-postural.bin`: imagem única, gravada no endereço **`0x0`** (preenche os buracos). Grava de qualquer computador, inclusive pelo navegador em `https://espressif.github.io/esptool-js/` (Chrome/Edge).
- `firmware-app-0x10000.bin`: só o app, para gravar em `0x10000` numa placa que já tem bootloader.

Parâmetros: `--flash_mode dio --flash_freq 40m --flash_size 4MB` (ESP32 DevKit, ESP32-WROOM-32). O script usa o PlatformIO em `~/.local/bin/pio` e `~/.platformio`; outro caminho: `PIO=/caminho/pio tools/gerar_bin.sh`.

> O `dist/` só é atualizado quando o script roda. Mudou a página (`src/web/`) ou o código? Gere de novo, senão o `.bin` não corresponde ao código.

---

7. `platformio.ini`

Ambiente	Chave	Valor	Por quê
`esp32dev`	`platform`	`espressif32@6.9.0`	versão fixa (Arduino-ESP32 core 2.0.17): atualizar pode mudar APIs (ex.: LEDC do motor)
	`monitor_speed`	`115200`	velocidade da serial do firmware
	`monitor_filters`	`esp32_exception_decoder`	traduz o backtrace de um travamento em arquivo e linha
	`upload_speed`	`921600`	se a gravação falhar, baixar para 460800 ou 115200
	`build_flags`	`-DCORE_DEBUG_LEVEL=0`	sem logs do core: a serial fica só para comandos, `diag` e `plot on`
	`lib_deps`	`WebSockets@2.7.3`, `ArduinoJson@7.2.1`	versões fixas do servidor WebSocket e do JSON
	`board_build.embed_txtfiles`	`src/web/index.html`, `manifest.json`, `icon.svg`	página embutida no firmware, vira os símbolos `_binary_src_web_*` (com `'\0'` no fim) usados em `web.cpp`
	`test_ignore`	`*`	os testes não rodam na placa
`native`	`platform` / `test_framework`	`native` / `unity`	testes no PC
	`test_build_src` + `build_src_filter`	só `tilt.cpp` e `posture.cpp`	só a lógica pura compila sem Arduino
	`build_flags`	`-std=c++11 -Wall`	C++11 com todos os avisos: a lógica pura não usa nada mais novo

---

8. Como estender

Mudança	Onde mexer
O firmware passou a usar outra função do Arduino	declarar em `sim/Arduino.h`, implementar em `sim/sim_main.cpp`
Arquivo `.cpp` novo em `src/`	lista do `g++` em `tools/sim.sh`; se for lógica pura testada, `build_src_filter` do `native`
Comando novo de roteiro	`executarLinha` em `sim_main.cpp`, comentário do topo do arquivo e a tabela da seção 3.5
Campo novo no JSON ou regra nova de postura	`src/protocolo.cpp` / `src/posture.cpp` e **também** a classe `Colete` do `mock_server.py`
Teste novo	função + `RUN_TEST` em `test_main.cpp` e a tabela da seção 2
Cenário novo de hardware	roteiro novo em `tools/`, com comentário do resultado esperado em cada bloco, e a tabela da seção 3.6

---

9. Limitações (o que nada disso prova)

O roteiro de teste no hardware é o `VALIDACAO.md`. O que só o hardware valida:

- **Concorrência das duas tarefas.** No simulador tudo roda numa linha só: a trava, a fila de eventos e os pedidos entre tarefas não são exercitados (02).
- **Sinal do giroscópio.** O MPU falso usa a mesma convenção do código (`dg/dt = g × ω`); um sinal invertido no sensor real passaria despercebido.
- **Driver do MPU de verdade:** registradores, escalas, clones (MPU-6500), I2C a 100 kHz com fio longo.
- **ADC real:** ruído com o Wi-Fi ligado e o "fantasma" do GPIO35 solto copiando a tensão do GPIO34.
- **Wi-Fi, HTTP e WebSocket no ESP32** e no celular (o `sim_web.py` usa o servidor do PC).
- **Elétrica:** queda de tensão quando o motor liga, power bank desligando por consumo baixo.
- **NVS na flash** (o simulador guarda em memória e perde tudo ao terminar).

10. Particularidades conhecidas

- `tools/sim.sh -` ("interativo") lê a entrada **até o fim** (Ctrl+D) antes de executar. Para interagir de verdade, use `python3 tools/sim_web.py` (modo `--tempo-real`).
- Com `!mpuaddr 69` o `WHO_AM_I` continua `0x68` (só o endereço muda), como num GY-521 com AD0 em 3V3.
- No simulador, o pulso de 120 ms do boot pode não aparecer como `motor desliga` se um teste de motor (`diag`) começar antes dele terminar: o simulador imprime só as transições.
- Os textos de ajuda do `sim_web.py` e do `grafico_serial.py` (`--help`) são a própria docstring do módulo.

10 — Decisões de projeto

> Código: todo o firmware · Relacionados: 02 — Arquitetura, MANUTENCAO.md

Registro das escolhas que dão forma ao código, com o motivo de cada uma. Antes de reverter uma delas, leia o motivo: quase todas resolvem um problema concreto. Ao mudar uma decisão, não apague o texto antigo: marque "Substituída por D-xx" e acrescente a nova no fim.

Cada decisão tem: **Contexto** (o problema), **Decisão**, **Alternativas** (o que ficou de fora e por quê), **Consequências** e **Código** (onde está).

D-01 — O colete funciona sozinho; o app é opcional

- **Contexto:** o uso real é o dia inteiro, longe do computador; o celular pode estar desligado ou sem rede.
- **Decisão:** toda a detecção e o alerta rodam no ESP32. O app só mostra dados e manda comandos. Calibração e ajustes também funcionam sem app (botão e serial), e os ajustes ficam salvos no colete.
- **Alternativas:** processar no celular ou na nuvem deixaria o colete inútil sem conexão.
- **Consequências:** o firmware precisa de retorno próprio ao usuário (padrões de vibração, 06).
- **Código:** `src/main.cpp`.

D-02 — App web embarcado: Access Point + HTTP + WebSocket

- **Contexto:** era preciso uma interface no celular sem instalar nada, sem internet e sem depender da rede da faculdade.
- **Decisão:** o ESP32 cria a própria rede Wi-Fi (Access Point `BioSpine`, IP 192.168.4.1), serve uma página HTML/CSS/JS de arquivo único (porta 80) e troca JSON por WebSocket (porta 81) a 10 Hz. A página vai embutida na flash (`board_build.embed_txtfiles`) e não usa CDN.
- **Alternativas:** app nativo (Android/iOS) e BLE: mais trabalho e instalação; nuvem/MQTT: precisa de internet. Ficaram para uma versão 2.
- **Consequências:** o Android avisa "sem internet" e pode desviar o tráfego para os dados móveis: é preciso tocar em "Manter conexão" ou desligar os dados móveis. Para parecer app, a página tem manifest e ícone ("Adicionar à tela inicial").
- **Código:** `src/web.cpp`, `src/web/`, `platformio.ini`.

D-03 — `WebServer` do core + `WebSocketsServer` (links2004)

- **Contexto:** escolher a pilha de servidor.
- **Decisão:** `WebServer` (já vem no core Arduino-ESP32) para a página e `links2004/WebSockets` para o WebSocket.
- **Alternativas:** `ESPAsyncWebServer`, que tem vários forks e versões incompatíveis entre si.
- **Consequências:** os dois servidores são síncronos e rodam dentro do `loop()` (`web_atualizar`); um envio lento bloqueia o loop. Isso motivou a D-12.
- **Código:** `src/web.cpp`.

D-04 — Driver próprio do MPU-6050, sem a biblioteca `MPU6050_light`

- **Contexto:** o plano inicial usava a `MPU6050_light`. Com o sensor em pé nas costas, a fórmula de ângulo dela salta ~20° perto da vertical quando há inclinação lateral; o `calcOffsets()` dela supõe o sensor deitado; e o endereço 0x68 é fixo (a montagem BioSpine usa 0x69).

- **Decisão:** driver mínimo por registradores: procura em 0x68 e 0x69, configura ±250 °/s, ±2 g e DLPF de ~44 Hz, lê 14 bytes de uma vez, trata clones da família MPU-6500.
- **Alternativas:** outras bibliotecas também entregam ângulos de Euler e herdariam o problema da D-05.
- **Consequências:** cerca de 150 linhas a manter; os valores de registrador seguem o datasheet (rev. 4.2).
- **Código:** `src/mpu.cpp` — 03.

D-05 — Desvio = ângulo entre vetores gravidade; a calibração grava `g0`

- **Contexto:** ângulos de Euler (pitch/roll) têm singularidade justamente na posição em que o sensor fica no colete.
- **Decisão:** a medida de postura é o ângulo entre a gravidade atual e a gravidade gravada na calibração (`atan2(|a×b|, a·b)`). A calibração grava o vetor `g0`, e não `pitch0/roll0`. Frente/trás e lateral são calculados só para exibir.
- **Alternativas:** pitch/roll com troca de eixos conforme a montagem: frágil, depende de como o sensor foi preso.
- **Consequências:** funciona em qualquer orientação de montagem; o desvio não tem sinal (o alerta não sabe se foi para a frente ou para o lado, e não precisa saber).
- **Código:** `src/tilt.cpp` (`angleBetweenDeg`, `tiltComponentsDeg`), `src/settings.h` (`Calibracao::g0`).

D-06 — Filtro complementar sobre o vetor gravidade, com rejeição de aceleração

- **Contexto:** acelerômetro é ruidoso e sente o movimento; giroscópio deriva.
- **Decisão:** filtro complementar aplicado ao vetor (previsão `g × ω` pelo giroscópio + 2% do acelerômetro a cada 5 ms; τ ≈ 0,25 s). Quando |a| sai de [0,7; 1,3] g, o acelerômetro é ignorado naquele instante.
- **Alternativas:** Kalman ou Madgwick resolvem também, com mais parâmetros e mais custo; para inclinação lenta do tronco o complementar basta.
- **Consequências:** um único ajuste (`MPU_FILTRO_GIRO`); o sinal da convenção do giroscópio precisa ser conferido no hardware (`VALIDACAO.md`, V2.6).
- **Código:** `src/tilt.cpp` (`TiltFilter`).

D-07 — Offset do giroscópio no boot e recalculado sozinho quando parado

- **Contexto:** o colete pode ser ligado já vestido, com a pessoa se mexendo, e o boot não consegue medir o offset.
- **Decisão:** mede no boot (~1 s) só se a placa estiver parada; depois recalcula a cada ~1 s parado.
- **Consequências:** sem intervenção do usuário; o filtro complementar cobre o tempo sem offset.
- **Código:** `src/mpu.cpp` (`calibrarGiro`, `acumularOffset`).

D-08 — Detecção com limite, histerese e tempo mínimo (ok → atenção → alerta)

- **Contexto:** um limite simples liga e desliga o motor quando o desvio oscila perto dele, e vibra a cada movimento rápido.
- **Decisão:** entra em "ruim" acima do limite, sai só abaixo de limite − histerese; o motor só liga depois de `tempoMs` em "ruim". MPU e flex combinados com OU para entrar e E para sair.
- **Consequências:** três parâmetros ajustáveis; um estado intermediário (atenção) que o app mostra com contagem regressiva.
- **Código:** `src/posture.cpp` (`DetectorPostura`) — 04.

D-09 — Sem calibração, o colete não vibra

- **Contexto:** sem referência não existe "postura correta"; a postura de cada pessoa e a posição do sensor variam.
- **Decisão:** o detector força o estado ok enquanto não houver calibração válida. O app e a serial pedem a calibração.
- **Consequências:** primeiro uso exige apertar o botão; a calibração fica na NVS e vale nos próximos usos.
- **Código:** `DetectorPostura::atualizar`, `console_iniciar`.

D-10 — Calibração com espera, média e recusa se mexer; retorno pela vibração

- **Contexto:** o botão fica no colete; ao apertar, a pessoa mexe o tronco. Uma referência gravada em movimento faria o colete vibrar na postura correta.
- **Decisão:** 1 pulso ao começar, espera de 1,5 s, média de 1 s; recusa com pulso longo se o giroscópio passar de 30 °/s; 2 pulsos quando conclui. Calibração recusada mantém a anterior.
- **Consequências:** dá para calibrar sem olhar nenhuma tela.
- **Código:** `src/main.cpp` (`iniciarCalibracao`, `concluirCalibracao`), `src/posture.cpp` (`Calibrador`).

D-11 — Sensores analógicos opcionais e detectados sozinhos; flex 2 desligado por padrão; FSR só telemetria

- **Contexto:** a montagem pode vir sem flex, com flex quebrado ou sem resistor, com um ou dois flex, com ou sem FSR. O colete não pode depender do flex.
- **Decisão:** cada canal só entra na decisão se a leitura for plausível e estável por 2 s e se a calibração foi feita com ele ligado; no modo `auto` (padrão) sai sozinho após 1 s inválido (no modo `on` fica, mesmo solto). O flex 2 começa em `off` porque um GPIO35 solto copia a tensão do GPIO34 (capacitor do ADC) e pareceria um sensor de verdade. O FSR não alerta.
- **Consequências:** quem tem o segundo flex precisa de `set flex2 auto` e recalibrar.
- **Código:** `src/sensors.cpp`, `src/filters.h`, `src/settings.cpp` (`configPadrao`), `src/main.cpp` (`sensorAtivo`) — 05.

D-12 — Duas tarefas FreeRTOS: controle (prioridade 3) e loop (Wi-Fi e serial)

- **Contexto:** quando o celular sai do alcance, o envio TCP do WebSocket pode bloquear o `loop()` por segundos. Com tudo no loop, o colete pararia de detectar e vibrar nesse tempo.
- **Decisão:** a tarefa de controle (sensores, postura, motor, botão, calibração) roda a cada 1 ms com prioridade maior que o loop. As tarefas só conversam por pedidos, fila de eventos e cópias protegidas por trava. A tarefa de controle nunca chama `Serial` nem `web_*`.
- **Consequências:** regra rígida para quem mexer no código (usar `emitir()`); concorrência só validável no hardware.
- **Código:** `src/main.cpp` — 02.

D-13 — Agendamento por `millis()`, sem `delay()`

- **Contexto:** vários períodos diferentes (5, 10, 20, 100 ms) no mesmo laço.
- **Decisão:** cada etapa guarda o instante da última execução e roda quando passou o seu período. `delay()` só no boot (offset do giroscópio, reset do MPU), no `Mpu6050::iniciar` (~150 ms, também usado na reconexão do MPU, dentro da tarefa de controle) e no simulador.
- **Código:** `sensores_atualizar`, `passoControle`, `motor_atualizar`, `loop`.

D-14 — Parâmetros ajustáveis sem recompilar, validados no firmware e salvos na NVS

- **Contexto:** os limites certos só aparecem com o colete no corpo da pessoa, no dia do teste.
- **Decisão:** limite, histerese, tempo, limite do flex, intensidade e modos dos sensores mudam pela serial e pelo app. O firmware valida (`validarParams`) qualquer valor recebido e grava na NVS.
- **Consequências:** a validação fica num lugar só; a página pode limitar os controles, mas quem garante é o firmware.
- **Código:** `src/posture.cpp`, `src/settings.cpp`, `src/console.cpp`, `src/protocolo.cpp` — 06.

D-15 — Motor por PWM a 20 kHz, alerta em pulsos de 300/700 ms, LED junto

- **Contexto:** vibração contínua incomoda e o corpo se acostuma; PWM audível faz o motor chiar.
- **Decisão:** PWM fora da faixa audível, alerta em pulsos, LED da placa como espelho do motor para diagnóstico.
- **Código:** `src/motor.cpp`.

D-16 — Wi-Fi com potência reduzida (11 dBm), sem economia de energia, heartbeat no WebSocket

- **Contexto:** pico de corrente do Wi-Fi somado ao motor pode derrubar a tensão e reiniciar o ESP32; celulares somem da rede sem avisar.
- **Decisão:** `WiFi.setTxPower(WIFI_POWER_11dBm)` (alcance de sobra para uma sala), `WiFi.setSleep(false)` (latência estável), heartbeat de 5 s que derruba clientes que sumiram.
- **Código:** `src/web.cpp` (`web_iniciar`).

D-17 — I2C a 100 kHz, dois endereços e reconexão automática do MPU

- **Contexto:** fios longos no colete, mau contato, duas montagens com AD0 diferente.
- **Decisão:** 100 kHz (mais tolerante que 400 kHz), timeout de 20 ms, endereços 0x68 e 0x69, 10 falhas seguidas = sensor perdido, nova tentativa a cada 2 s, avisos na serial e no app.
- **Código:** `src/sensors.cpp`, `src/mpu.cpp`.

D-18 — Um firmware para as duas montagens (GUIA e BioSpine V3)

- **Contexto:** existem duas pinagens em uso (a do `montagem.pdf` e a do README do BioSpine V3), e nenhum pino conflita.
- **Decisão:** atender as duas ao mesmo tempo, com detecção automática do que está ligado, e aceitar os comandos serial do BioSpine V3 (`START`, `STOP`, `STATUS`, `CALIBRATE`) com as respostas JSON no formato dele.
- **Código:** `src/config.h`, `src/console.cpp` — 07.

D-19 — Lógica pura separada do hardware; testes no PC e simulador do firmware inteiro

- **Contexto:** o hardware só ficou disponível no último dia.
- **Decisão:** `tilt`, `posture` e `filters` não dependem do Arduino e têm 21 testes nativos. Um simulador compila o firmware real no PC, trocando só o hardware (MPU falso, ADC, botão, NVS em memória, Wi-Fi desligado).
- **Consequências:** quase todos os fluxos foram exercitados antes da placa; continuam sem cobertura a concorrência, o ADC real e o Wi-Fi real.
- **Código:** `test/`, `sim/`, `tools/sim.sh` — 09.

D-20 — Protocolo JSON num módulo separado do Wi-Fi

- **Contexto:** o simulador não tem Wi-Fi, mas o JSON do app precisava ser testado.
- **Decisão:** `protocolo.cpp` monta e interpreta as mensagens (ArduinoJson) sem conhecer o Wi-Fi; `web.cpp` só transporta. O simulador e o `tools/sim_web.py` usam o mesmo `protocolo.cpp` com a página real.
- **Código:** `src/protocolo.cpp`, `src/web.cpp`, `sim/sim_main.cpp`.

11 — Mapa entre o texto do TCC e o código

> Código: `src/mpu.cpp`, `src/sensors.cpp`, `src/motor.cpp`, `src/settings.cpp` · Relacionados: 01 — Visão geral, 10 — Decisões

Liga cada afirmação técnica do texto do TCC ("Colete Ortopédico Inteligente para Monitoramento Postural em Tempo Real", versão de 19/10/2026) ao ponto do código que a implementa, lista o que precisa ser corrigido no texto e sugere o que falta escrever sobre o software.

1. Afirmações do texto e onde estão no código

Seção do texto	Afirmação	No código	Situação
Resumo	MPU-6050 com acelerômetro e giroscópio num módulo	`src/mpu.cpp` (driver), `src/sensors.cpp`	ok
Resumo	ESP32 faz aquisição, tratamento e transmissão	`sensores_atualizar`, `TiltFilter`, `DetectorPostura`, `web_enviarDados`	ok
1	Sensores de flexão (plural) para a curvatura	Flex 1 (GPIO34) e flex 2 opcional (GPIO35, desligado por padrão)	ok; ver 05
1	Motor vibratório acionado quando há desvio	`motor_alerta` em `atualizarPostura`	ok
1.1	Resistência do flex aumenta ao flexionar; leitura por divisor de tensão	Divisor 3V3–flex–GPIO34–10 kΩ–GND; leitura **cai** ao dobrar; `flexPct`	ok
1.1	Resistência inicial 10 kΩ, tolerância ±30%	Curvatura relativa à calibração; faixa válida ampla (100–4000)	ok: a tolerância é o motivo da medida relativa
1.2	Sensibilidade 131 LSB/°/s e 16384 LSB/g	`LSB_POR_DPS = 131`, `LSB_POR_G = 16384` em `src/mpu.cpp`	ok
1.2	Escalas ±250 °/s e ±2 g	`GYRO_CONFIG = 0x00`, `ACCEL_CONFIG = 0x00` em `Mpu6050::iniciar`	ok
1.2	"fator variando de um a quatro"	—	**corrigir** (item 2)
1.3	ADC de 12 bits para o flex	`analogReadResolution(12)`, só ADC1	ok
1.3	I2C com o MPU-6050	`Wire` a 100 kHz, GPIO21/22	ok
1.3	Wi-Fi e Bluetooth	Wi-Fi em Access Point; Bluetooth **não usado**	complementar
1.3	Flash 0.512KB, EEPROM 428KB	—	**corrigir** (item 1)
1.4	Motor 3 V, 60 mA, vibração contínua (DC)	Motor DC acionado por **PWM 20 kHz** via transistor, alerta em **pulsos** 300/700 ms	complementar

2. Correções sugeridas no texto

1. **Memória do ESP32 (1.3).** "Flash 0.512KB / EEPROM 428KB" está errado. O ESP32-WROOM-32 tem **4 MB de flash** e **não tem EEPROM**: dados permanentes ficam na NVS, uma área da flash (é onde o colete guarda limites e calibração). SRAM ~520 kB está certo.
2. **Faixas do MPU-6050 (1.2).** São quatro faixas: ±250, ±500, ±1000, ±2000 °/s e ±2, ±4, ±8, ±16 g. O fator vai de **1 a 8** e a sensibilidade **divide** por ele (131 → 65,5 → 32,8 → 16,4 LSB/°/s; 16384 → 8192 → 4096 → 2048 LSB/g). O projeto usa a faixa mais sensível (fator 1).
3. **Motor (1.4).** Incluir que o GPIO não fornece 60 mA e por isso há um transistor NPN com resistor de base de 1 kΩ e diodo de proteção; e que o firmware usa PWM e pulsos (ver 06).
4. **Numeração das figuras.** Há duas "Figura 2" e duas "Figura 3".

5. **Itens que faltam no hardware:** botão de calibração, transistor + diodo, capacitor de 100 µF, power bank, esquema de ligação (figura do `montagem.pdf`).

3. O que escrever sobre o software (roteiro sugerido)

Seção sugerida	Conteúdo	Base nesta documentação
Arquitetura do sistema	Colete autônomo + app web embarcado; módulos; duas tarefas	01, 02
Aquisição e filtragem	Configuração do MPU, offset do giroscópio, filtro complementar sobre o vetor gravidade, desvio	03
Sensores de flexão	Divisor, média móvel, detecção automática, curvatura relativa	05
Calibração e detecção	Máquina de estados ok/atenção/alerta, histerese, tempo mínimo, calibração com recusa	04
Alerta e persistência	Circuito do motor, PWM, padrões de vibração, NVS	06
Interface com o usuário	Botão, serial, app web (Access Point, WebSocket, gráfico, 3D)	07, 08
Testes e validação	Testes unitários, simulador, roteiro no hardware	09, `VALIDACAO.md`
Resultados	Gráfico desvio × tempo com limite e alerta, vídeo, tabela de validação	`tools/grafico_serial.py`, CSV do app

Frase de resumo para o texto (do plano original): "Aplicação web embarcada, servida pelo próprio microcontrolador, com comunicação bidirecional via WebSocket sobre Wi-Fi em modo Access Point."

Figuras sugeridas

Figura	Diagrama
Contexto do sistema	01, seção 2
Ligações elétricas	01, seção 3 ou o esquema do `montagem.pdf`
Arquitetura em camadas	02, seção 2
Tarefas e comunicação entre elas	02, seção 3
Fluxo de dados do sensor ao alerta	02, seção 5
Cálculo da inclinação	03, seção 1
Máquina de estados da postura	04, seção 2
Sequência da calibração	04, seção 3
Protocolo app ↔ colete	07
Resultado: desvio × tempo	PNG gerado por `tools/grafico_serial.py`

Como exportar os diagramas em imagem: README.md.

4. Perguntas prováveis da banca

Como o colete sabe qual é a postura correta? Pela calibração: a pessoa veste, fica na postura correta e aperta o botão. O colete grava a direção da gravidade naquele instante (média de 1 s) e passa a medir o desvio em relação a ela. Sem calibração ele não vibra.

Por que usar giroscópio e acelerômetro juntos? O acelerômetro dá a direção absoluta da gravidade, mas é ruidoso e sente os movimentos; o giroscópio é suave, mas acumula erro. O filtro complementar usa o giroscópio para os movimentos rápidos e o acelerômetro para corrigir a deriva (98% / 2% a cada 5 ms, constante de tempo

de ~0,25 s).

Por que não calcular pitch e roll, como é comum? As fórmulas de pitch/roll supõem o sensor deitado. Em pé, nas costas, uma delas passa por uma singularidade e o ângulo salta ~20° com 4° de movimento real. Medir o ângulo entre o vetor gravidade atual e o da calibração não tem esse problema e não depende de como o sensor foi preso.

O que é a histerese e por que usar? É uma margem para sair do alerta: entra acima de 20° e só sai abaixo de 17°. Sem ela, um desvio oscilando em torno de 20° ligaria e desligaria o motor sem parar.

Por que esperar 3 segundos para vibrar? Para não vibrar em movimentos rápidos e normais (pegar algo no chão). Só a postura ruim que se mantém gera alerta. O tempo é ajustável.

E se o sensor flex quebrar ou soltar? No modo padrão (`auto`), o firmware percebe (leitura fora da faixa ou instável por 1 s), tira o flex da decisão e segue só com o MPU. Se o flex tiver sido forçado ligado (`set flex on`), ele continua na decisão e um flex solto dispara o alerta; por isso o modo recomendado é `auto`.

E se o celular desconectar? Nada muda no colete: a detecção e o motor rodam numa tarefa separada, de prioridade maior que a do `loop()` (onde rodam a página, o WebSocket e a serial), e não dependem do app.

Os ajustes se perdem ao desligar? Não. Ficam na NVS, uma área da flash do ESP32.

Como foi testado sem o hardware? A lógica (inclinação, filtro, detecção, calibração, filtros dos sensores) tem 21 testes automáticos que rodam no PC. O firmware inteiro também roda no PC com um MPU-6050 simulado, seguindo roteiros (inclinar, dobrar o flex, apertar o botão, desconectar o sensor) e ligado à página web real.

Qual a frequência de amostragem? MPU a 200 Hz, flex a 100 Hz, decisão de postura a 50 Hz, envio ao app a 10 Hz.

Qual a precisão da medida? A resolução do acelerômetro é 1/16384 g. A estabilidade do desvio parado e a resposta a movimentos rápidos são medidas no teste com o hardware (`VALIDACAO.md`, V2.4 e V2.6); use os números medidos no texto.

A rede do colete é segura? É uma rede local com senha (WPA2), sem internet. Quem tiver a senha pode ver os dados e mudar os ajustes. Para um protótipo de TCC é suficiente; uma versão comercial precisaria de senha por aparelho.

Módulo `config`

> Código: `src/config.h` · Relacionados: 01 — Visão geral, 03 — Inclinação, 04 — Detecção e calibração, 05 — Sensores analógicos, 06 — Motor e NVS

Papel no sistema

Reúne em um lugar só tudo que é número fixo do firmware: pinos, endereços I2C, tempos, limiares dos sensores, padrões de vibração, nome e senha do Wi-Fi e os valores de fábrica dos parâmetros ajustáveis. Só tem `#define`, sem lógica. A ideia é que um ajuste de bancada (ex.: ruído do ADC maior que o esperado) seja uma troca de número aqui, sem procurar o valor espalhado pelo código.

Os `PADRAO_*` são só o ponto de partida: o que o usuário ajusta pela serial ou pelo app fica salvo na NVS (settings) e vale mais que estes valores.

Dependências

- Usa: `<stdint.h>`.
- É usado por: praticamente todo o firmware (`main.cpp`, `sensors.cpp`, `mpu.cpp`, `motor.cpp`, `posture.cpp`, `console.cpp`, `protocolo.cpp`, `web.cpp`) e pelo simulador (`sim/sim_main.cpp`).
- Roda em: compilação (não há código executável).

Pinagem dupla

O firmware atende as duas montagens existentes ao mesmo tempo, porque nenhum pino conflita:

Função	Constante	GPIO	montagem.pdf / GUIA	BioSpine V3
I2C SDA (MPU-6050)	`PIN_I2C_SDA`	21	sim	sim
I2C SCL (MPU-6050)	`PIN_I2C_SCL`	22	sim	sim
Flex 1	`PIN_FLEX`	34	sim	flex esquerdo
Flex 2	`PIN_FLEX2`	35	reserva	flex direito
FSR (pressão)	`PIN_FSR`	32	—	sim
Motor (via NPN)	`PIN_MOTOR`	25	sim	—
Botão de calibração	`PIN_BUTTON`	27	sim	—
LED azul da placa	`PIN_LED`	2	—	sim

Por que esses pinos:

- **Todo sensor analógico fica no ADC1 (GPIO 32–39).** O ADC2 do ESP32 não funciona com o Wi-Fi ligado.
- GPIO 34 e 35 são só entrada (servem para ADC, não para saída).
- Saídas evitam os pinos de boot (0, 2, 12, 15). O GPIO2 é usado só para o LED, que já existe na placa.
- O motor puxa ~60 mA: o GPIO não aguenta, por isso vai por um transistor NPN (resistor de 1 kΩ na base).
- Botão para o GND com `INPUT_PULLUP`: solto = `HIGH`, apertado = `LOW`. Dispensa resistor externo.

Tipos e constantes

MPU-6050

Constante	Valor	Unidade	Usada em	Descrição
`MPU_ADDR_A`	`0x68`	—	`mpu.cpp`, `console.cpp`	endereço com AD0 solto ou em GND (padrão do GY-521)
`MPU_ADDR_B`	`0x69`	—	`mpu.cpp`, `console.cpp`	endereço com AD0 em 3V3
`MPU_I2C_HZ`	100000	Hz	`sensors.cpp`	100 kHz: mais tolerante a fio longo no colete do que 400 kHz
`MPU_FILTRO_GIRO`	0.98	—	`sensors.cpp`	peso do giroscópio no filtro complementar (o acelerômetro fica com 0,02)
`MPU_REINICIO_MS`	2000	ms	`sensors.cpp`	intervalo entre tentativas de reconectar o MPU
`GIRO_PARADO_DPS`	5.0	°/s	`mpu.cpp`	variação máxima do giroscópio numa janela para considerar o sensor parado
`GIRO_OFFSET_MAX`	25.0	°/s	`mpu.cpp`	offset de giroscópio plausível; acima disso é movimento, não offset

Sensores analógicos (ADC de 12 bits, 0–4095)

Constante	Valor	Unidade	Usada em	Descrição
`ADC_MIN_VALIDO`	100	ADC	`sensors.cpp`, `main.cpp`, `console.cpp`	abaixo disso: flex solto ou quebrado
`ADC_MAX_VALIDO`	4000	ADC	`sensors.cpp`, `main.cpp`, `console.cpp`	acima disso: falta o resistor de 10 kΩ do divisor
`ADC_RUIDO_MAX`	300	ADC	`sensors.cpp`	amplitude (máx − mín) máxima na janela da média; pino solto oscila mais
`ADC_MEDIA_N`	10	amostras	`sensors.cpp`	tamanho da média móvel (10 amostras a 100 Hz = janela de 100 ms)
`FLEX_LIGA_MS`	2000	ms	`sensors.cpp`	tempo de leitura válida contínua para considerar um sensor presente (flex 1, flex 2 e FSR)
`FLEX_DESLIGA_MS`	1000	ms	`sensors.cpp`	tempo de leitura inválida contínua para considerar o flex ausente
`FSR_LIMIAR`	200	ADC	`sensors.cpp`	FSR com leitura estável acima disso por `FLEX_LIGA_MS` = sensor existe
`ADC_FANTASMA_DIF`	300	ADC	`sensors.cpp`	FSR com leitura a menos disso do flex 1 é tratado como "fantasma" (pino solto copiando o canal vizinho)

Motor (LEDC = periférico de PWM do ESP32)

Constante	Valor	Unidade	Usada em	Descrição
`MOTOR_LEDC_CANAL`	0	—	`motor.cpp`	canal LEDC (core Arduino 2.x; no 3.x o canal é escolhido pelo pino)
`MOTOR_LEDC_FREQ`	20000	Hz	`motor.cpp`	20 kHz: acima da faixa audível (o motor não "chia")
`MOTOR_LEDC_BITS`	8	bits	`motor.cpp`	resolução do duty: 0–255
`MOTOR_ALERTA_ON_MS`	300	ms	`motor.cpp`	alerta: tempo ligado de cada pulso

Constante	Valor	Unidade	Usada em	Descrição
`MOTOR_ALERTA_OFF_MS`	700	ms	`motor.cpp`	alerta: pausa entre pulsos (menos incômodo que vibração contínua)
`MOTOR_TESTE_MS`	1000	ms	`main.cpp`	pulso do comando `motor`, do botão "Testar motor" e do `diag`
`MOTOR_CONFIRMA_MS`	120	ms	`main.cpp`	pulso curto de aviso: boot (1), início da calibração (1), fim da calibração (2)
`MOTOR_ERRO_MS`	600	ms	`main.cpp`	pulso longo: calibração recusada

Tempos (agendamento por `millis()`, sem `delay()`)

Constante	Valor	Frequência	Usada em	Descrição
`PERIODO_MPU_MS`	5 ms	200 Hz	`sensors.cpp`	leitura do MPU e passo do filtro complementar
`PERIODO_ADC_MS`	10 ms	100 Hz	`sensors.cpp`	leitura dos três canais analógicos
`PERIODO_POSTURA_MS`	20 ms	50 Hz	`main.cpp` (`passoControle`)	cálculo do desvio e detecção
`PERIODO_ENVIO_MS`	100 ms	10 Hz	`main.cpp` (`loop`)	envio para o app e para o plotter
`CALIBRACAO_ESPERA_MS`	1500 ms	—	`main.cpp`	depois do clique: tempo para a pessoa se ajeitar
`CALIBRACAO_MS`	1000 ms	—	`main.cpp`	duração da média da calibração
`CALIBRACAO_GIRO_MAX`	30 °/s	—	`main.cpp`	se o giroscópio passar disso durante a média, a calibração é recusada
`BOTAO_DEBOUNCE_MS`	50 ms	—	`main.cpp`	tempo que o botão precisa ficar estável para contar

Wi-Fi

Constante	Valor	Usada em	Descrição
`WIFI_SSID`	`"BioSpine"`	`web.cpp`, `console.cpp`	nome da rede criada pelo colete (Access Point)
`WIFI_SENHA`	`"colete123"`	`web.cpp`	senha WPA2 (mínimo de 8 caracteres, senão o AP não sobe)
`HTTP_PORTA`	80	`web.cpp`	página do app: `http://192.168.4.1`
`WS_PORTA`	81	`web.cpp`	WebSocket: `ws://192.168.4.1:81/`

Valores de fábrica dos parâmetros ajustáveis

Usados no primeiro boot, no comando `reset` e no botão "Restaurar padrão" do app. São folgados de propósito: funcionam sem calibração fina. As faixas aceitas são impostas por `validarParams` (posture).

Constante	Valor	Unidade	Campo em `Params`	Faixa aceita
`PADRAO_LIMITE_GRAUS`	20	°	`limiteGraus`	2–60
`PADRAO_HISTERESE`	3	° (MPU) / pontos % (flex)	`histerese`	0 até (menor limite − 1)
`PADRAO_TEMPO_MS`	3000	ms	`tempoMs`	0–60000
`PADRAO_LIMITE_FLEX`	15	%	`limiteFlex`	2–90
`PADRAO_INTENSIDADE`	100	% do duty	`intensidade`	0–100

Constante	Valor	Usada em	Descrição
`FIRMWARE_VERSAO`	`"1.0.0"`	`main.cpp`, `protocolo.cpp`	mostrada no boot da serial e enviada no `{"t":"config"}`

Armadilhas e cuidados

- **Números de linha citados por fora.** O `VALIDACAO.md` (roteiro do teste no hardware) cita linhas deste arquivo (`src/config.h:25`, `:27`, `:31`, `:39`, `:59`, `:65`). Não inserir nem remover linhas antes da seção do Wi-Fi; comentário novo vai no fim de uma linha existente ou depois da seção do Wi-Fi. Duas dessas citações já estavam deslocadas antes da documentação: `:31` aponta para `ADC_MIN_VALIDO` (o texto fala de `ADC_RUIDO_MAX`, linha 33) e `:39` aponta para um comentário (o texto fala de `MOTOR_LEDC_FREQ`, linha 44).
- **Pinos:** trocar um pino analógico para fora do ADC1 faz o sensor parar de funcionar com o Wi-Fi ligado.
- **`WIFI_SENHA` com menos de 8 caracteres:** `WiFi.softAP` falha e o colete fica sem app (continua vibrando).
- **`PERIODO_ADC_MS` × `ADC_MEDIA_N`:** juntos definem a janela da média (100 ms). Aumentar qualquer um deixa o flex mais lento para reagir.
- **`MPU_FILTRO_GIRO`:** mais perto de 1 = mais suave e mais lento para corrigir deriva; mais perto de 0 = mais rápido e mais sensível a vibração (inclusive a do próprio motor).
- **`FLEX_LIGA_MS` vale para os três sensores analógicos** (flex 1, flex 2 e FSR), apesar do nome.

Módulo `main` (núcleo e API `app_*`)

> Código: `src/main.cpp`, `src/app.h` · Relacionados: 02 — Arquitetura, 04 — Detecção e calibração, console, protocolo

Papel no sistema

`main.cpp` liga todos os módulos: faz o boot (`setup`), roda a **tarefa de controle** (sensores → desvio → detecção → motor, botão e calibração) e o **loop** (Wi-Fi e serial). Também implementa a API `app_*` declarada em `app.h`, que é a porta principal pela qual a serial (console) e o app web (protocolo, web) conversam com o núcleo (as exceções, leituras diretas de `sensores()`, estão em 02, seção 6).

A divisão em duas tarefas existe porque o Wi-Fi pode travar o `loop()` por vários segundos: quando o celular sai do alcance, o envio TCP fica esperando dentro da biblioteca. Com a detecção numa tarefa à parte, de prioridade maior, o colete continua detectando e vibrando nessa hora. Ele funciona sozinho, sem celular e sem computador.

Dependências

- Usa: config, sensors, posture (`DetectorPostura`, `Calibrador`, `validarParams`), tilt (`angleBetweenDeg`, `tiltComponentsDeg`), motor, settings, console, web.
- É usado por: `console.cpp` e `protocolo.cpp` (pela API `app_*`); `sim/sim_main.cpp` chama `setup()` e `loop()` e também usa `app_status()`.
- Roda em: as duas tarefas (cada função diz qual, abaixo). No simulador (`-DSIMULADOR`) tudo roda numa linha só: o `loop()` chama `passoControle()` e a trava vira nada.

Duas tarefas e como elas conversam

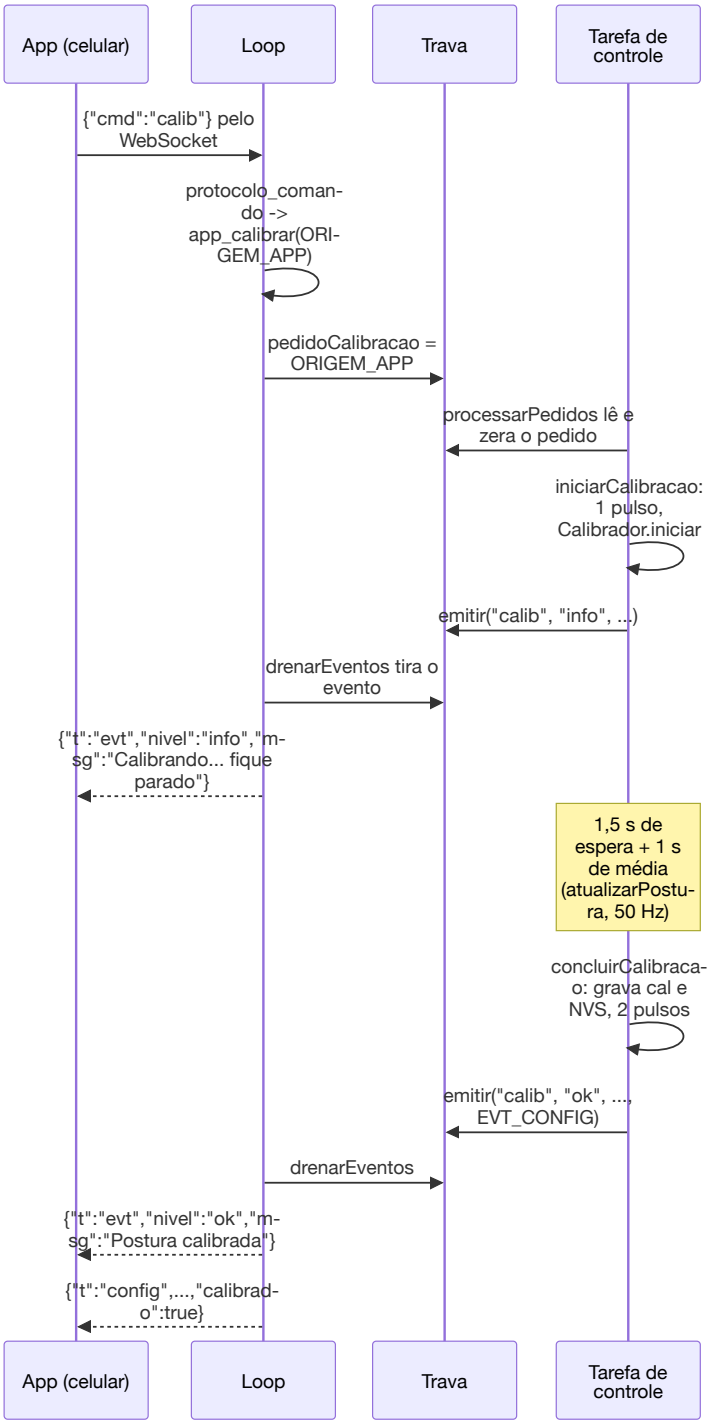
	Tarefa de controle (`tarefaControle`)	**Loop (`loop()` do Arduino)**
Criada por	`xTaskCreatePinnedToCore` no fim do `setup`	core do Arduino
Núcleo / prioridade	núcleo 1, prioridade 3	núcleo 1, prioridade 1
Ritmo	um `passoControle` a cada 1 ms (`vTaskDelay(1)`)	o mais rápido possível
Faz	sensores, botão, desvio, detecção, calibração, motor	serial, HTTP, WebSocket, envio a 10 Hz
Pode bloquear?	não em Wi-Fi nem serial (nunca chama `Serial` nem `web_*`); esperas curtas e conhecidas: timeout do I2C, ~150 ms na reconexão do MPU, gravação da calibração na NVS	sim (Wi-Fi, serial)

As duas tarefas trocam informação de três formas, todas protegidas pela mesma trava (fora delas, só leituras de exibição de `sensores()` pelo loop, ver 02, seção 6):

1. **Pedidos** (loop → controle): `pedidoCalibracao` (a `Origem`, ou –1) e `pedidoMotorTeste`. O loop só marca; a tarefa de controle executa no próximo `processarPedidos`.
2. **Eventos** (controle → loop): fila circular `fila[FILA_N]` de `Evento`, preenchida por `emitir()` e esvaziada por `drenarEventos()`. É assim que a tarefa de controle "escreve" na serial e no app sem nunca esperar por eles.
3. **Cópias**: `cfg` (Config), `cal` (Calibracao) e `stPublico` (Status) são lidos sempre como cópia inteira (`app_config`, `app_calibracao`, `app_status`), para nunca ler uma struct pela metade.

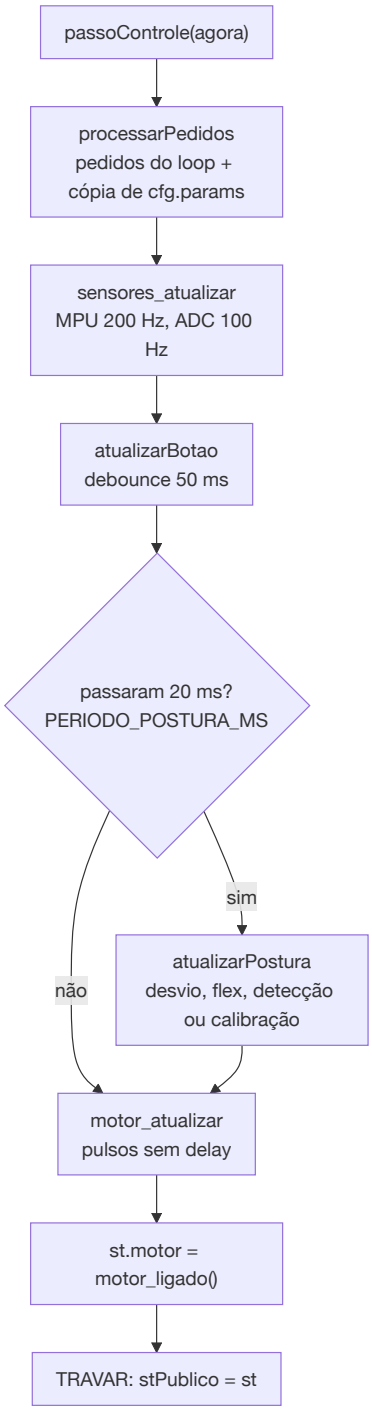
A trava (`TRAVAR`/`DESTRAVAR`) é um `portMUX` do FreeRTOS: enquanto fechada, as interrupções do núcleo ficam desligadas. Por isso dentro dela só se copia memória (microssegundos); nunca `Serial`, NVS ou Wi-Fi.

O diagrama mostra um pedido de calibração vindo do app, até a resposta voltar como evento.



Um passo da tarefa de controle

Cada etapa tem o próprio período, controlado por `millis()`; o passo em si roda a cada 1 ms.



Tipos e constantes

`struct Status` (`app.h`)

Foto do colete num instante. Montada pela tarefa de controle (`st`) e publicada a cada passo em `stPublico`. É a origem de tudo que o app e a serial mostram.

Campo	Tipo	Unidade	Descrição
`pitch`	float	°	componente frente/trás do desvio (só exibição)
`roll`	float	°	componente lateral do desvio (só exibição)
`desvio`	float	°	ângulo total entre a gravidade atual e a referência
`flex`	int	ADC 0–4095	média móvel do flex 1

Campo	Tipo	Unidade	Descrição
`flexPct`	float	%	curvatura do flex 1 (0 se inativo)
`flexAtivo`	bool	—	flex 1 entra na detecção (`sensorAtivo`)
`flexDetectado`	bool	—	flex 1 parece ligado
`flex2`, `flex2Pct`, `flex2Ativo`, `flex2Detectado`	—	—	o mesmo para o flex 2
`fsr`	int	ADC 0–4095	média do FSR (só telemetria)
`fsrDetectado`	bool	—	já leu pressão estável desde o boot
`estado`	`Estado`	—	ok / atenção / alerta
`tempoRuimMs`	uint32_t	ms	há quanto tempo a postura está ruim (0 se boa)
`calibrado`	bool	—	existe referência gravada
`calibrando`	bool	—	calibração em andamento
`mpuOk`	bool	—	MPU-6050 respondendo
`motor`	bool	—	saída do motor ligada neste instante

`enum Origem` (`app.h`)

Quem pediu a calibração: `ORIGEM_BOTAO`, `ORIGEM_SERIAL` (comando `calib`), `ORIGEM_SERIAL_JSON` (`CALIBRATE` do BioSpine V3) e `ORIGEM_APP`. Só muda o fim: com `ORIGEM_SERIAL_JSON`, a conclusão também imprime `{"success":true,"message":"Calibration completed"}`.

`struct Evento` e a fila (`main.cpp`)

Campo	Tamanho	Descrição
`tag`	8	prefixo da serial (`calib`, `mpu`) → `[calib] ...`
`nivel`	6	nível do aviso no app: `info`, `ok`, `erro`
`console`	96	texto para a serial (`""` = não imprime)
`app`	64	texto curto para o toast do app (`""` = não envia)
`flags`	1	`EVT_CONFIG` (reenviar `{"t":"config"}`) e/ou `EVT_JSON_CALIB_OK` (resposta JSON do BioSpine)

- Tamanho fixo: nada de alocação dinâmica dentro da tarefa de controle. Texto maior é cortado (`strlcpy`).
- `FILA_N = 8`. **Com a fila cheia, o evento novo é descartado** (perder um aviso é melhor que travar o controle). Na prática o loop esvazia a fila a cada volta; só enche se o loop ficar travado no Wi-Fi.

Variáveis de estado

Variável	Dona	Descrição
`cfg`, `cal`, `stPublico`	compartilhadas (trava)	config em vigor, calibração, última foto publicada
`pedidoCalibracao`, `pedidoMotorTeste`	compartilhadas (trava)	pedidos do loop
`detector`, `calibrador`	controle	máquina de estados da postura e média da calibração
`origemCalibracao`	controle	quem pediu a calibração em andamento
`st`	controle	foto em construção
`params`	controle	cópia de `cfg.params` tirada a cada passo
`intensidadeAplicada`	controle	última intensidade passada ao motor (255 = nenhuma)
`gBoot`, `temGBoot`	controle	referência provisória antes da 1ª calibração
`mpuAvisado`	controle	último estado do MPU já avisado

Variável	Dona	Descrição
`ultimoPostura`	controle	agendamento dos 50 Hz
`botaoEstavel`, `botaoLido`, `botaoMudou`	controle	debounce do botão
`ultimoEnvio`	loop	agendamento dos 10 Hz

Funções

API pública (`app.h`) — chamada pelo loop

`bool app_calibrar(Origem origem)`

- **O que faz:** registra um pedido de calibração para a tarefa de controle.
- **Parâmetros:** `origem` — quem pediu.
- **Retorno:** `false` (e avisa serial e app na hora) se o MPU não estiver respondendo; `true` se o pedido foi registrado. O resultado da calibração chega depois, como evento.
- **Chamada por:** `console.cpp` (`calib`, `CALIBRATE`), `protocolo.cpp` (`{"cmd":"calib"}`).
- **Observações:** retorna `true` mesmo se já houver uma calibração em andamento; nesse caso o pedido é ignorado pela tarefa de controle, sem aviso.

`void app_testarMotor()`

- **O que faz:** pede um pulso de `MOTOR_TESTE_MS` (1 s). Quem liga o motor é a tarefa de controle.
- **Chamada por:** `console.cpp` (`motor`, `diag`), `protocolo.cpp` (`motor_test`).

`Config app_config()` · `Calibracao app_calibracao()` · `Status app_status()`

- **O que fazem:** devolvem uma cópia (struct inteira, com a trava) da configuração, da calibração e da última foto publicada.
- **Chamadas por:** console, protocolo; `app_config`/`app_calibracao` também pela própria tarefa de controle (`atualizarPostura`, `concluirCalibracao`).

`void app_definirConfig(const Config& c)`

- **O que faz:** aplica uma configuração nova: `validarParams` (prende cada valor na faixa) → troca `cfg` → grava na NVS (`settings_salvarConfig`) → envia `{"t":"config"}` a todos os clientes do app.
- **Chamada por:** `console.cpp` (`set`, `reset`), `protocolo.cpp` (`set`, `reset`).
- **Observações:** só pode ser chamada do loop (envia pelo WebSocket). A tarefa de controle pega os parâmetros novos no próximo `processarPedidos`, em até ~1 ms.

Eventos

`static void emitir(tag, nivel, console, app, flags = 0)`

- **O que faz:** monta um `Evento` e põe na fila. Fila cheia = descarta.
- **Roda em:** tarefa de controle.

`static void drenarEventos()`

- **O que faz:** tira os eventos um a um (com a trava) e entrega cada um com a trava aberta: `console_evento` (serial), `console_respostaJson` (se `EVT_JSON_CALIB_OK`), `web_evento` (app) e `web_enviarConfig` (se `EVT_CONFIG`).
- **Roda em:** loop, a cada volta.

Tarefa de controle

`static void passoControle(uint32_t agora)`

- **O que faz:** um passo completo (diagrama acima). Termina publicando `st` em `stPublico`.
- **Chamada por:** `tarefaControle` (ESP32) ou `loop` (simulador).

`static void processarPedidos(uint32_t agora)`

- **O que faz:** numa trava só, lê e zera os pedidos e copia `cfg.params` para `params`. Depois executa: calibração, pulso de teste e, se a intensidade mudou, `motor_intensidade`.

`static void atualizarBotao(uint32_t agora)`

- **O que faz:** debounce. A leitura crua (`botaoLido`) só vira valor aceito (`botaoEstavel`) depois de ficar igual por `BOTAO_DEBOUNCE_MS`. Na transição para `LOW` (apertado) chama `iniciarCalibracao(ORIGEM_BOTAO)`. Segurar o botão não repete.

`static void atualizarPostura(uint32_t agora)` — 50 Hz

- **O que faz**, em ordem:
  1. Avisa (evento) se o MPU caiu ou voltou, só na mudança.
  2. Escolhe a referência: `cal.g0` se calibrado; senão `gBoot` (gravidade do primeiro instante com o MPU ok). Com `gBoot` o app já mostra o movimento, mas o detector não alerta (`Entrada::calibrado = false`).
  3. Calcula `desvio = angleBetweenDeg(g, ref)` e a decomposição `pitch`/`roll` (`tiltComponentsDeg`). Sem MPU, os três ficam 0 e só o flex pode alertar.
  4. Calcula `flexAtivo`/`flexPct` (e o mesmo do flex 2) com `sensorAtivo` e `flexPct`.
  5. Copia FSR, `calibrado`, `calibrando`.
  6. **Calibrando:** entrega amostras ao `Calibrador` (gravidade, flex 1 e 2, norma do giroscópio) e, ao terminar, chama `concluirCalibracao`. Estado fica `ok` (sem alerta durante a calibração). **Senão:** monta a `Entrada` e roda `detector.atualizar(e, params, agora)`.
  7. `motor_alerta(estado == ESTADO_ALERTA)`: só o alerta vibra; "atenção" é a contagem do tempo mínimo.

`static bool sensorAtivo(ModoSensor modo, bool detectado, float referencia)`

- **O que faz:** decide se um flex entra na detecção.

Modo	Resultado
`off`	nunca
`on`	se a referência da calibração for válida (`ADC_MIN_VALIDO`–`ADC_MAX_VALIDO`)
`auto`	se o sensor estiver detectado agora **e** a referência for válida

- **Por quê:** sem uma referência gravada com o sensor ligado, `flexPct` não tem sentido. Por isso um flex ligado depois da calibração aparece no `get` como "detectado, recalibre para usar".

`static void iniciarCalibracao(Origem origem, uint32_t agora)`

- **O que faz:** se o MPU estiver ok e não houver calibração em andamento: guarda a origem, inicia o `Calibrador` (espera `CALIBRACAO_ESPERA_MS` + média de `CALIBRACAO_MS`), zera o detector, dá 1 pulso curto e emite "calibrando...". Sem MPU, emite erro.

`static void concluirCalibracao()`

- **O que faz:** encerra o `Calibrador` e:
  - **recusa** se `giroMax() > CALIBRACAO_GIRO_MAX` (30 °/s): 1 pulso longo (`MOTOR_ERRO_MS`), evento de erro, **mantém a calibração anterior**;

- **aceita** caso contrário: `g0 = calibrador.gravidade()`; `flex0`/`flex20` = média do sensor se ele estiver detectado ou em modo `on`, senão 0 (fica fora da detecção); grava em `cal` e na NVS; zera o detector; 2 pulsos curtos; evento "Postura calibrada" com `EVT_CONFIG` (e `EVT_JSON_CALIB_OK` se veio do `CALIBRATE`).

`static void tarefaControle(void*)`

- **O que faz:** laço infinito `passoControle(millis()); vTaskDelay(1);`. O `vTaskDelay(1)` devolve a CPU por 1 tick (1 ms) para o loop e o watchdog.
- **Existe só no ESP32** (`#ifndef SIMULADOR`).

Arduino

`void setup()`

Ordem do boot: serial 115200 → botão `INPUT_PULLUP` → `motor_iniciar` (motor desligado já no início) → `settings_carregar` → banner → `sensores_iniciar` (bloqueia ~1 s medindo o offset do giroscópio: o colete deve estar parado) → aviso se o MPU não respondeu → `web_iniciar` → `console_iniciar` → 1 pulso curto ("liguei") → cria a tarefa de controle.

`void loop()`

`drenarEventos` → `console_atualizar` → `web_atualizar` → a cada `PERIODO_ENVIO_MS` (100 ms): `app_status()` e envio para o app (`web_enviarDados`) e a serial (`console_enviarDados`). No simulador, chama `passoControle` antes de tudo.

Armadilhas e cuidados

- **Nunca chamar `Serial`, `web_*`, `console_*` da tarefa de controle.** Use `emitir()`. Exceção já existente e segura: `settings_salvarCalibracao` (NVS) em `concluirCalibracao`.
- **Nada demorado dentro de `TRAVAR()`/`DESTRAVAR()`**: as interrupções ficam desligadas.
- **Leituras sem trava:** `app_calibrar` e `console.cpp` (`diag`) leem `sensores()` direto, sem trava. É aceitável para um booleano e para diagnóstico, mas não use esse caminho para lógica de decisão no loop.
- **Fila de eventos com 8 posições:** se uma mudança futura emitir muitos eventos seguidos, aumente `FILA_N` ou eles serão descartados em silêncio.
- **Sem calibração o colete não vibra** (decisão de projeto: sem referência não existe "postura correta").
- **Simulador:** qualquer código novo na tarefa de controle precisa compilar também com `-DSIMULADOR` (lá não existe FreeRTOS).

Módulo `tilt`

> Código: `src/tilt.h`, `src/tilt.cpp` · Relacionados: 03 — Inclinação, sensors, mpu, main

Papel no sistema

É a matemática da postura. Transforma as leituras do MPU-6050 (aceleração em g e velocidade angular em °/s) numa estimativa estável da **direção da gravidade** vista pela placa, e mede o **desvio**: o ângulo entre essa gravidade e a gravidade gravada na calibração (referência `g0`).

O módulo existe porque a abordagem clássica (ângulos de Euler pitch/roll, como na biblioteca `MPU6050_light`) falha no colete: o sensor fica **em pé** nas costas, perto da singularidade das fórmulas de pitch/roll, e o ângulo salta ~20° ao passar pela vertical com um pouco de inclinação lateral. Trabalhar com vetores evita isso e deixa o resultado independente de como o sensor foi preso (ver 10 — Decisões).

É lógica pura (só `<math.h>`): roda igual no ESP32, nos testes nativos e no simulador.

Dependências

- Usa: `<math.h>` (`sqrtf`, `atan2f`, `fabsf`).
- É usado por: `sensors.cpp` (`TiltFilter`), `main.cpp` (`angleBetweenDeg`, `tiltComponentsDeg`, `norm`), `posture.h` (`Vec3` no `Calibrador`), `mpu.h`/`mpu.cpp` (`Vec3`), `settings.h` (`Vec3` em `Calibracao`), `test/test_posture/test_main.cpp`.
- Roda em: tarefa de controle (ESP32); PC (testes e simulador).

Convenção de eixos e de "gravidade"

- Os vetores estão nos **eixos da placa** do MPU (X, Y, Z).
- "Gravidade" é o que o acelerômetro mede parado: **+1 g apontando para cima** (a reação do apoio). Só a direção importa, então os vetores de gravidade são sempre normalizados (unitários).
- Montagem esperada no colete: placa em pé entre as escápulas, eixo **Y para cima** e eixo **Z saindo das costas** (componentes virados para fora). Inclinar o tronco para a frente leva a gravidade para +Z; para o lado, para X. Essa é a mesma convenção do MPU falso do simulador e da função `gravidadeColete` dos testes.

Tipos e constantes

Nome	Tipo	Descrição	Unidade
`Vec3`	`struct { float x, y, z; }`	vetor 3D nos eixos da placa	g, °/s ou sem unidade (unitário)
`v3(x, y, z)`	função inline	monta um `Vec3` (o struct fica agregado simples, sem construtor)	—
`+`, `-`, `* k`	operadores inline	soma, subtração, multiplicação por escalar	—
`dot(a, b)`	inline	produto escalar = |a|·|b|·cos θ	—
`cross(a, b)`	inline	produto vetorial (regra da mão direita), módulo |a|·|b|·sen θ	—
`norm(a)`	inline	módulo do vetor	—
`RAD_PARA_GRAU`, `GRAU_PARA_RAD`	`static const float` (tilt.cpp)	conversões	—
`TiltFilter`	classe	filtro complementar sobre o vetor gravidade	—

Funções

`Vec3 normalize(Vec3 a)`

- **O que faz:** devolve `a / |a|`.
- **Parâmetros:** `a` — qualquer vetor.
- **Retorno:** vetor unitário na direção de `a`. Se `|a| < 1e-6`, devolve `a` sem mudança (evita divisão por zero).
- **Chamada por:** `tiltComponentsDeg`, `TiltFilter`, `Calibrador::gravidade` (posture.h), testes.
- **Observações:** quem chama decide o que fazer com vetor nulo (ex.: `TiltFilter::reset` não se considera inicializado).

`float angleBetweenDeg(Vec3 a, Vec3 b)`

- **O que faz:** ângulo entre dois vetores, em graus (0–180).
- **Parâmetros:** `a`, `b` — não precisam ser unitários.
- **Retorno:** `atan2(|a × b|, a · b)` convertido para graus.
- **Chamada por:** `atualizarPostura` em `main.cpp` (o **desvio** = `angleBetweenDeg(L.g, ref)`), testes.
- **Observações:** a forma "óbvia" `acos(a · b)` perde precisão justamente perto de 0°, onde o colete passa a maior parte do tempo: o cosseno é quase constante ali (cos 0,5° = 0,99996), então o erro de arredondamento do `float` vira alguns centésimos de grau (até ~0,04°); e um `a · b` de 1,0000001 por arredondamento daria `NaN`. Com `atan2` as duas partes (seno e cosseno) entram juntas e a precisão é boa em toda a faixa. O teste `test_angulo_entre_vetores` confere 0,5° com erro < 0,01°.

`void tiltComponentsDeg(Vec3 g, Vec3 ref, float* frente, float* lado)`

- **O que faz:** decompõe a inclinação de `g` em relação a `ref` em dois ângulos com sinal: frente/trás e lateral.
- **Parâmetros:** `g` — gravidade atual; `ref` — gravidade de referência; `frente`, `lado` — saídas em graus (podem ser `NULL`).
- **Retorno:** nada (escreve nas saídas). `frente` positivo = tronco para a frente (com os componentes do sensor virados para fora).
- **Chamada por:** `atualizarPostura` em `main.cpp` (campos `pitch` e `roll` do `Status`), testes.
- **Observações: só para exibição** (app, serial, plotter). A detecção usa o desvio total, que não tem sinal e não depende de rótulos. Como funciona:
  1. Monta uma base ortonormal presa à postura calibrada: `ref` (cima), `f` (frente) e `l` (lado).
  2. `f` = eixo Z da placa sem a parte paralela a `ref` (Gram-Schmidt: `Z − ref·(Z·ref)`), normalizado.
  3. Se Z for quase paralelo a `ref` (`|Z·ref| > 0,9`, menos de ~26° entre eles), a projeção fica curta e instável: usa o eixo Y no lugar. É o caso da placa deitada na bancada.
  4. `l = ref × f`.
  5. `frente = atan2(g·f, g·ref)` e `lado = atan2(g·l, g·ref)`.

  Se o sensor for montado com os componentes virados para o corpo, `frente` sai com sinal trocado; o desvio e o alerta não mudam. O app tem as caixas "Inverter frente/trás" e "Inverter lado" para isso.

`TiltFilter::TiltFilter(float gyroCoef = 0.98f)`

- **O que faz:** cria o filtro não inicializado.
- **Parâmetros:** `gyroCoef` — peso do giroscópio na mistura (0–1). O firmware passa `MPU_FILTRO_GIRO` (0,98).

`void TiltFilter::reset(Vec3 acc)`

- **O que faz:** recomeça a estimativa pela direção do acelerômetro.
- **Parâmetros:** `acc` — aceleração (g).
- **Retorno:** nada. Fica inicializado só se `|acc| > 1e-3` (leitura não nula).

- **Chamada por:** `iniciarMpu` em `sensors.cpp` (boot e reconexão) e pelo próprio `update`.

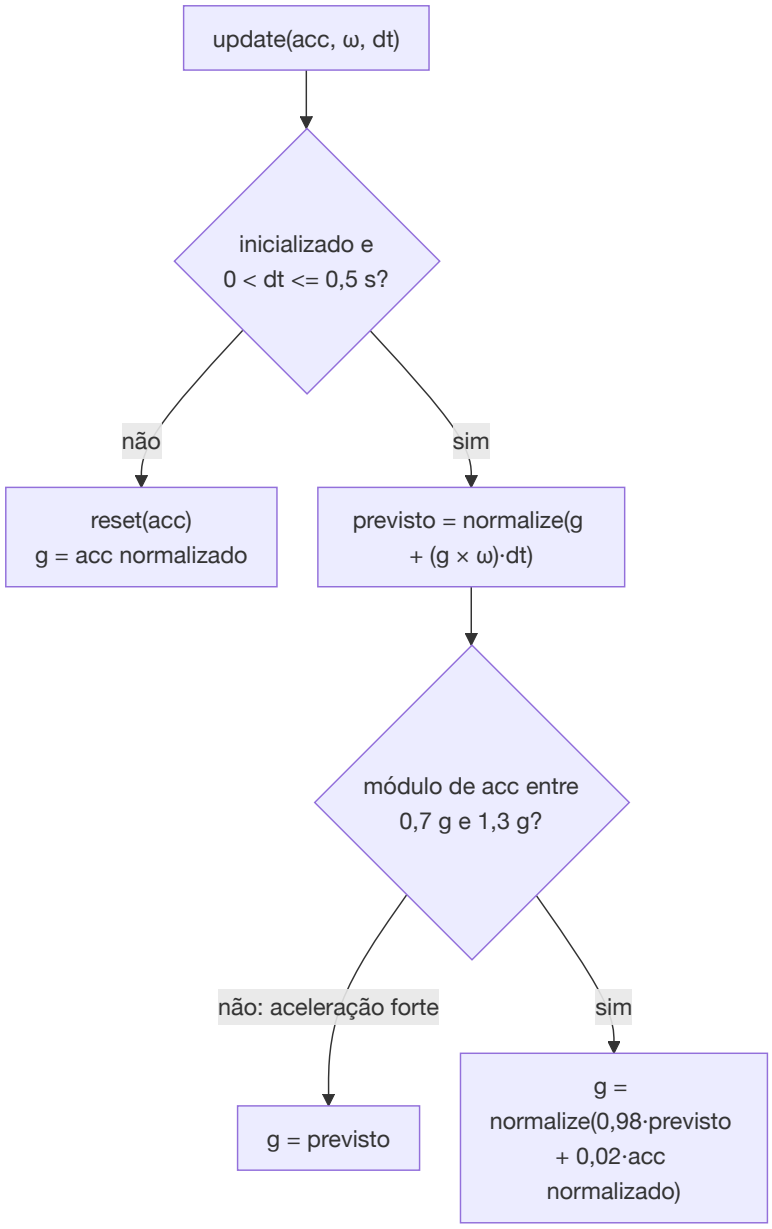
`void TiltFilter::update(Vec3 acc, Vec3 gyroDps, float dt)`

- **O que faz:** um passo do filtro complementar (200 Hz).
- **Parâmetros:** `acc` em g; `gyroDps` em °/s (já sem offset); `dt` em segundos (medido com `micros()`).
- **Retorno:** nada; a estimativa nova fica em `gravity()`.
- **Chamada por:** `sensores_atualizar` em `sensors.cpp`.
- **Observações:** passos, na ordem:
  1. **Proteção:** se não inicializado, ou `dt <= 0`, ou `dt > 0,5 s`, chama `reset(acc)` e sai. Integrar o giroscópio com um intervalo absurdo daria lixo.
  2. **Previsão pelo giroscópio:** `previsto = normalize(g + (g × ω)·dt)`, com ω em rad/s. A gravidade é fixa no mundo; quando a placa gira com velocidade angular ω, um vetor fixo no mundo, visto nos eixos da placa, gira com −ω: `dg/dt = −ω × g = g × ω`. É um passo de integração de Euler; ele aumenta um pouco o módulo a cada iteração, e a normalização mantém o vetor unitário.
  3. **Rejeição de aceleração:** se `|acc|` estiver fora de [0,7; 1,3] g, há aceleração do corpo somada (passo, batida) e a direção de `acc` não é a vertical. Nesse instante usa só a previsão.
  4. **Mistura:** `g = normalize(0,98·previsto + 0,02·normalize(acc))`. O giroscópio é bom no curto prazo e deriva; o acelerômetro é ruidoso e não deriva. Com `dt` = 5 ms, a constante de tempo é `τ = coef·dt / (1 − coef) ≈ 0,25 s`.

`Vec3 TiltFilter::gravity() const` / `bool TiltFilter::initialized() const`

- Gravidade estimada (unitária) / se já recebeu uma aceleração válida.

O fluxo do `update` em um passo:



Armadilhas e cuidados

- **Sinal do giroscópio:** a convenção `dg/dt = g × ω` foi conferida contra a da `MPU6050_light`, mas o simulador usa a mesma convenção do código, então ele não prova o sinal. Se no hardware um movimento rápido "repicar" (ir para o lado errado e voltar devagar), trocar `cross(g_, w)` por `cross(w, g_)` em `TiltFilter::update` e inverter o sinal do giro em `test_filtro_segue_giroscopio` (ver `VALIDACAO.md`, item V2.6).
- **Não trocar `atan2` por `acos`** em `angleBetweenDeg` (precisão perto de 0°, `NaN`).
- **Limiar 0,9 em `tiltComponentsDeg`:** é o que separa "placa em pé" de "placa deitada". Mudar o eixo de referência (Z) muda o significado de frente/lado no app.
- **`MPU_FILTRO_GIRO`** (config.h): subir (0,99) suaviza mais e corrige a deriva mais devagar; descer deixa o desvio mais ruidoso com a vibração do motor.
- Testes que cobrem o módulo (`test/test_posture/test_main.cpp`): `test_angulo_entre_vetores`, `test_desvio_sem_salto_perto_da_vertical`, `test_componentes_frente_e_lado`, `test_componentes_com_sensor_de_cabeca_para_baixo`, `test_componentes_com_placa_deitada`, `test_filtro_converge_para_acelerometro`, `test_filtro_segue_giroscopio`, `test_filtro_reinicia_com_dt_grande`.

Módulo `posture`

> Código: `src/posture.h`, `src/posture.cpp` · Relacionados: 04 — Detecção e calibração, main, tilt, settings

Papel no sistema

Decide **quando o colete vibra** e guarda a lógica da **calibração**. Recebe o desvio (°) do MPU e a curvatura (%) dos flex, aplica limite, histerese e tempo mínimo e devolve o estado: ok, atenção ou alerta. O `Calibrador` faz a média da postura correta (gravidade e flex) que vira a referência salva na NVS.

Fica separado de `main.cpp` e sem Arduino para ser testado no PC: é o coração do comportamento do colete e tem o maior grupo dos 21 testes nativos (10; `tilt` tem 8 e `filters`, 3).

Dependências

- Usa: `tilt.h` (`Vec3`, `normalize`), `config.h` (valores `PADRAO_*`).
- É usado por: `main.cpp` (detector, calibrador, `validarParams`, `flexPct`), `settings.cpp` (`paramsPadrao`, `validarParams`), `protocolo.cpp` (`estadoNome`), `console.cpp` (`Estado`), `app.h` (`Estado` no `Status`), testes.
- Roda em: tarefa de controle (detector e calibrador, 50 Hz); `validarParams` também no loop (via `app_definirConfig`); PC (testes e simulador).

Tipos e constantes

`struct Params` — parâmetros ajustáveis (serial, app, NVS)

Campo	Tipo	Descrição	Unidade	Padrão (`config.h`)	Faixa (`validarParams`)
`limiteGraus`	`float`	desvio máximo aceito	°	20 (`PADRAO_LIMITE_GRAUS`)	2–60
`histerese`	`float`	margem abaixo do limite para sair da má postura (vale para o MPU e para o flex)	° ou pontos %	3 (`PADRAO_HISTERESE`)	0 até (menor limite − 1)
`tempoMs`	`uint32_t`	tempo mínimo em má postura antes de vibrar	ms	3000 (`PADRAO_TEMPO_MS`)	0–60 000
`limiteFlex`	`float`	curvatura máxima do flex	%	15 (`PADRAO_LIMITE_FLEX`)	2–90
`intensidade`	`uint8_t`	potência do motor (duty do PWM)	%	100 (`PADRAO_INTENSIDADE`)	0–100

`enum Estado`

Valor	Nome no app (`estadoNome`)	Nome no JSON BioSpine (`console.cpp`)	Significado
`ESTADO_OK = 0`	`ok`	`GOOD`	postura boa (ou sem calibração)
`ESTADO_ATENCAO = 1`	`atencao`	`WARNING`	passou do limite, contando o tempo mínimo; não vibra
`ESTADO_ALERTA = 2`	`alerta`	`BAD`	passou do limite há `tempoMs` ou mais; motor pulsando

Os valores numéricos indexam a tabela `{"GOOD", "WARNING", "BAD"}` de `console.cpp`: não reordenar.

`struct Entrada` — o que o detector recebe a cada atualização

Campo	Tipo	Descrição
`calibrado`	`bool`	existe referência salva; sem ela o detector fica sempre em ok
`desvio`	`float`	° em relação à postura calibrada
`flexAtivo`, `flexPct`	`bool`, `float`	flex 1 entra na decisão? curvatura em %
`flex2Ativo`, `flex2Pct`	`bool`, `float`	idem para o flex 2

Montada em `atualizarPostura` (`main.cpp`). "Ativo" já considera o modo (auto/on/off), a detecção e a referência (função `sensorAtivo` de `main.cpp`).

Funções

`Params paramsPadrao()`

- **O que faz:** devolve os valores de fábrica (`PADRAO_*` de `config.h`).
- **Chamada por:** `configPadrao` (`settings.cpp`), testes.

`Params validarParams(Params p)`

- **O que faz:** prende cada campo numa faixa segura (tabela acima).
- **Parâmetros:** `p` — valores vindos da serial, do app ou da NVS (podem ser qualquer coisa).
- **Retorno:** cópia corrigida.
- **Chamada por:** `app_definirConfig` (`main.cpp`), `settings_carregar` (`settings.cpp`), testes.
- **Observações:**
  - `NaN` vira o mínimo da faixa (a função interna `prender` testa `v != v`).
  - **Por que histerese < menor limite – 1:** para sair da má postura o valor precisa ficar abaixo de `limite − histerese`. Se a histerese fosse maior ou igual ao limite, esse valor seria ≤ 0, e desvio e `flexPct` nunca são negativos: o colete **nunca sairia do alerta**. Como a mesma histerese vale para o MPU (°) e para o flex (%), ela é limitada pelo **menor** dos dois limites. O "– 1" garante pelo menos 1 unidade de faixa para a saída. Na página web o controle deslizante da histerese também é limitado (`ajustarHisterese`).

`const char* estadoNome(Estado e)`

- **O que faz:** nome do estado no protocolo do app (`"ok"`, `"atencao"`, `"alerta"`).
- **Chamada por:** `protocolo_dados` (`protocolo.cpp`), simulador.

`float flexPct(float leitura, float referencia)`

- **O que faz:** curvatura do flex como % de queda da leitura em relação à calibração.
- **Parâmetros:** `leitura` — média atual do ADC; `referencia` — média gravada na calibração (`flex0` ou `flex20`).
- **Retorno:** `(referencia − leitura) / referencia × 100`, preso em 0–100. `0` se `referencia < 1` (sem calibração do flex).
- **Chamada por:** `atualizarPostura` (`main.cpp`), testes.
- **Observações:** o divisor é 3V3 → flex → pino → 10 kΩ → GND. Ao dobrar, a resistência do flex sobe e a tensão no pino cai; por isso a curvatura é a **queda** relativa. Ex.: referência 2048 e leitura 1536 → 25 %. Leitura acima da referência (flex dobrado para o outro lado) dá 0, não negativo.

`DetectorPostura::reset()`

- **O que faz:** volta para ok e esquece o tempo acumulado.

- **Chamada por:** construtor; `iniciarCalibracao` e `concluirCalibracao` (`main.cpp`), para a referência nova começar do zero.

`Estado DetectorPostura::atualizar(const Entrada& e, const Params& p, uint32_t agoraMs)`

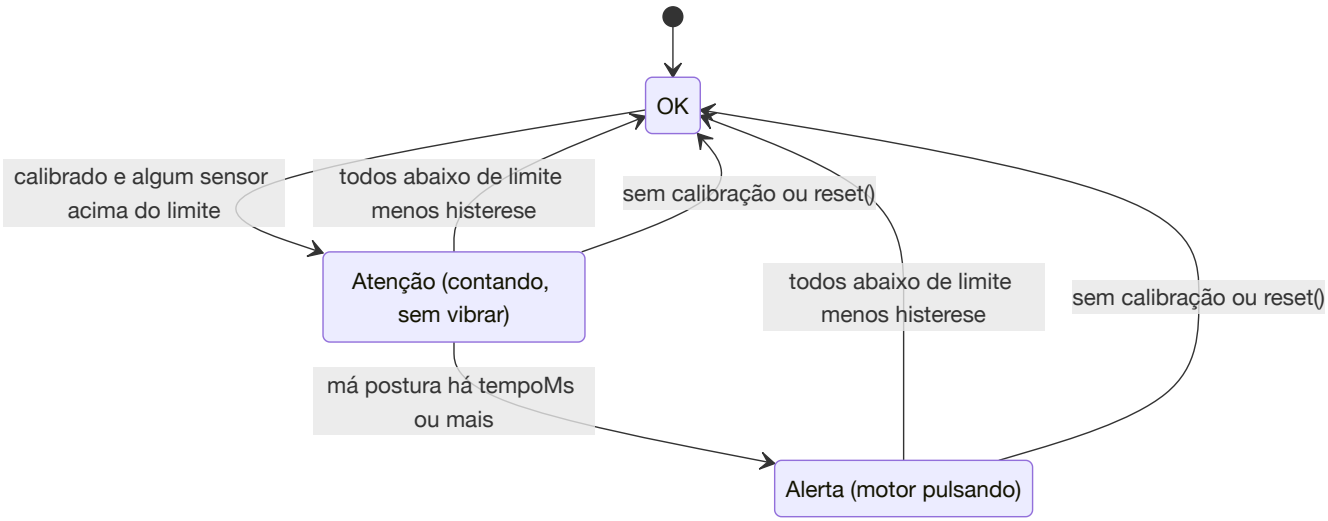
- **O que faz:** avalia uma nova entrada e devolve o estado.
- **Parâmetros:** `e` — entrada; `p` — parâmetros já validados; `agoraMs` — `millis()`.
- **Retorno:** novo estado.
- **Chamada por:** `atualizarPostura` (`main.cpp`) a 50 Hz (`PERIODO_POSTURA_MS`), só quando não está calibrando.
- **Observações:** duas camadas.
  1. **Má postura com histerese** (`ruim_`):
     - **entra** quando *algum* sensor passa do limite: `desvio > limiteGraus` **OU** `flexPct > limiteFlex` (de cada flex ativo);
     - **sai** quando *todos* os sensores ativos ficam abaixo de `limite − histerese`: `desvio < limiteGraus − histerese` **E** `flexPct < limiteFlex − histerese`;
     - entre os dois valores (faixa de histerese), mantém o que era. Isso impede o motor de ficar ligando e desligando quando o desvio oscila em cima do limite.
  2. **Tempo mínimo:** com `ruim_` há menos de `tempoMs` → **atenção** (não vibra); há `tempoMs` ou mais → **alerta**. Um pico curto (pegar algo no chão) não vibra. O tempo é contado a partir do instante em que `ruim_` ligou e recomeça do zero se a postura voltar a ser boa.

  - Sem calibração (`e.calibrado == false`), `ruim_` é forçado a `false`: sem referência não existe "postura correta". Por isso o colete **não vibra antes da primeira calibração**.
  - A conta `agoraMs − inicio_` é em `uint32_t`: continua certa quando `millis()` dá a volta (~49 dias).

`Estado estado() const` / `bool ruim() const` / `uint32_t tempoRuimMs(uint32_t agoraMs) const`

- Último estado / se está em má postura (atenção ou alerta) / há quantos ms está em má postura (0 se não está; vai para o app como `tempoRuimMs`, que desenha a barra de contagem até vibrar).

Máquina de estados do detector:



Exemplo com os padrões (limite 20°, histerese 3°, tempo 3000 ms), combinando os testes `test_atencao_e_depois_alerta` e `test_histerese` (os valores 18° e 16,9° vêm do segundo, que os testa em outros instantes):

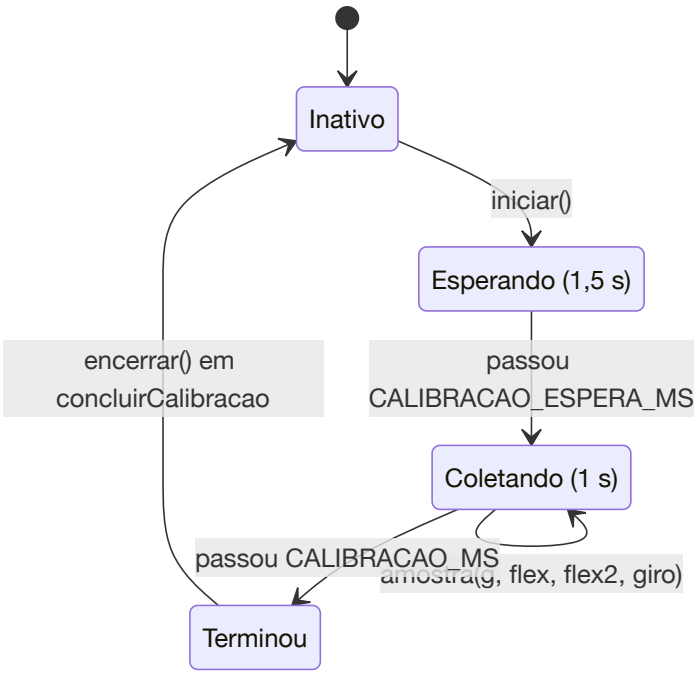
t (ms)	desvio	Estado	Por quê
0	10°	ok	abaixo do limite
1000	25°	atenção	passou de 20°; começa a contar
3999	25°	atenção	2999 ms < 3000 ms
4000	25°	alerta	3000 ms em má postura
4100	18°	alerta	abaixo de 20°, mas acima de 17° (20 – 3): faixa de histerese
4200	16,9°	ok	abaixo de 17°

Classe `Calibrador`

Faz a média da postura correta. A linha do tempo é: `iniciar()` → espera `CALIBRACAO_ESPERA_MS` (1,5 s, para a pessoa se ajeitar depois de apertar o botão) → coleta durante `CALIBRACAO_MS` (1 s) → `terminou()`. Quem decide aceitar ou recusar é `concluirCalibracao` em `main.cpp`, olhando `giroMax()` contra `CALIBRACAO_GIRO_MAX` (30 °/s).

Método	O que faz
`iniciar(agoraMs, esperaMs, duracaoMs)`	zera as somas, marca o início e fica ativo
`coletando(agoraMs)`	`true` depois da espera (as amostras contam)
`amostra(g, flex, flex2, giroDps)`	soma uma amostra; guarda o maior `giroDps` (módulo do giroscópio, °/s). Ignorada se inativo
`terminou(agoraMs)`	`true` quando espera + coleta já passaram
`encerrar()`	fica inativo; as médias continuam legíveis
`ativo()`, `amostras()`	em andamento? quantas amostras entraram
`gravidade()`	soma das gravidades normalizada = direção média (vira `Calibracao::g0`)
`flex()`, `flex2()`	média das leituras (viram `flex0` e `flex20`; 0 se não houve amostras)
`giroMax()`	maior velocidade angular durante a coleta (°/s)

`main.cpp` chama `amostra` a 50 Hz enquanto `coletando()`: cerca de 50 amostras por calibração.



Armadilhas e cuidados

- **OR para entrar, AND para sair.** Trocar a lógica faria o colete sair do alerta com um sensor ainda ruim, ou nunca entrar com o flex.
- **Uma histerese só** para graus e para %. Se um dia precisarem de histereses diferentes, `validarParams` e a página web (`ajustarHisterese`) também mudam.
- **Sem calibração não alerta** (decisão de projeto, ver 10). Não "consertar" usando a gravidade do boot: ela serve só para mostrar movimento antes da calibração.
- **Ordem do `enum Estado`** é usada como índice em `console.cpp`.
- Testes que cobrem o módulo: `test_sem_calibracao_nunca_alerta`, `test_atencao_e_depois_alerta`, `test_histerese`, `test_pico_curto_nao_alerta`, `test_flex_dispara_e_respeita_histerese`, `test_flex_inativo_e_ignorado`, `test_flex2`, `test_flex_pct`, `test_validar_params`, `test_calibrador_media`.

Módulo `filters`

> Código: `src/filters.h` · Relacionados: 05 — Sensores analógicos, sensors

Papel no sistema

Ferramentas pequenas para os sensores analógicos (flex 1, flex 2, FSR), usadas por `sensors.cpp`:

- `MediaMovel` suaviza a leitura do ADC e mede quanto ela oscila na janela;
- `leituraAdcValida` diz se a janela parece um divisor resistivo de verdade;
- `Presenca` transforma esse "parece válido agora" num "sensor presente" estável, com tempo mínimo para ligar e para desligar.

Juntas, elas implementam o **fallback do flex**: flex solto, quebrado ou sem resistor é desativado sozinho e o colete segue só com o MPU. Tudo é header-only e sem Arduino, testado no PC.

Dependências

- Usa: `<stdint.h>`.
- É usado por: `sensors.cpp`, testes.
- Roda em: tarefa de controle (ESP32, a 100 Hz, `PERIODO_ADC_MS`); PC (testes).

Tipos e constantes

Nome	Tipo	Descrição
`MediaMovel<N>`	template de classe	média móvel de `N` inteiros (buffer circular + soma corrente)
`Presenca`	classe	liga/desliga com persistência
`leituraAdcValida`	função inline	critério de leitura plausível de divisor resistivo

Constantes de `config.h` usadas com estas ferramentas (em `sensors.cpp`):

Constante	Valor	Uso
`ADC_MEDIA_N`	10 amostras	tamanho da média móvel (a 100 Hz, janela de 100 ms)
`ADC_MIN_VALIDO`	100	abaixo: flex solto ou quebrado
`ADC_MAX_VALIDO`	4000	acima: falta o resistor de 10 kΩ
`ADC_RUIDO_MAX`	300	amplitude máxima da janela (pino solto oscila mais)
`FLEX_LIGA_MS`	2000 ms	tempo válido contínuo para "presente"
`FLEX_DESLIGA_MS`	1000 ms	tempo inválido contínuo para "ausente"

Funções

`MediaMovel<N>::push(int v)`

- **O que faz:** acrescenta uma amostra. Com o buffer cheio, a mais antiga sai da soma antes de ser sobrescrita.
- **Observações:** custo O(1); a soma é `long` para não estourar com valores de 12 bits.

`float MediaMovel<N>::media() const`

- **Retorno:** média das amostras presentes; antes de encher, divide só pelas que existem; 0 se vazia.

`bool MediaMovel<N>::cheio() const` / `int MediaMovel<N>::ultimo() const`

- Já tem `N` amostras? / amostra mais recente (0 se vazia).

`int MediaMovel<N>::amplitude() const`

- **O que faz:** `máximo − mínimo` da janela.
- **Por quê:** um pino solto (sem sensor) flutua e oscila muito mais que um divisor resistivo. A amplitude separa os dois casos mesmo quando a média cai na faixa válida. Custo O(N), com N = 10.

`Presenca(uint32_t ligaMs, uint32_t desligaMs)`

- **Parâmetros:** `ligaMs` — tempo contínuo com a condição verdadeira para passar a presente; `desligaMs` — tempo contínuo com a condição falsa para passar a ausente.
- **Uso no firmware:** flex 1 e 2 com 2000/1000 ms; FSR com 2000/`UINT32_MAX` (depois de detectado, **nunca** volta a ausente até reiniciar, porque um FSR sem pressão lê ~0, igual a pino solto).

`void Presenca::atualizar(bool okAgora, uint32_t agora)`

- **O que faz:** atualiza o estado com a leitura do instante.
- **Parâmetros:** `okAgora` — a leitura deste instante parece válida; `agora` — `millis()`.
- **Observações:**
  - A primeira chamada adota o valor recebido na hora, sem esperar (por isso um flex bem ligado já sai do `setup` como presente).
  - Enquanto a condição concorda com o estado, o cronômetro é reiniciado. Quando discorda, o estado só troca depois de `ligaMs`/`desligaMs` **contados desde a última amostra que concordava**. Uma única amostra concordante no meio reinicia a contagem (teste `test_presenca_com_persistencia`).

`bool Presenca::presente() const`

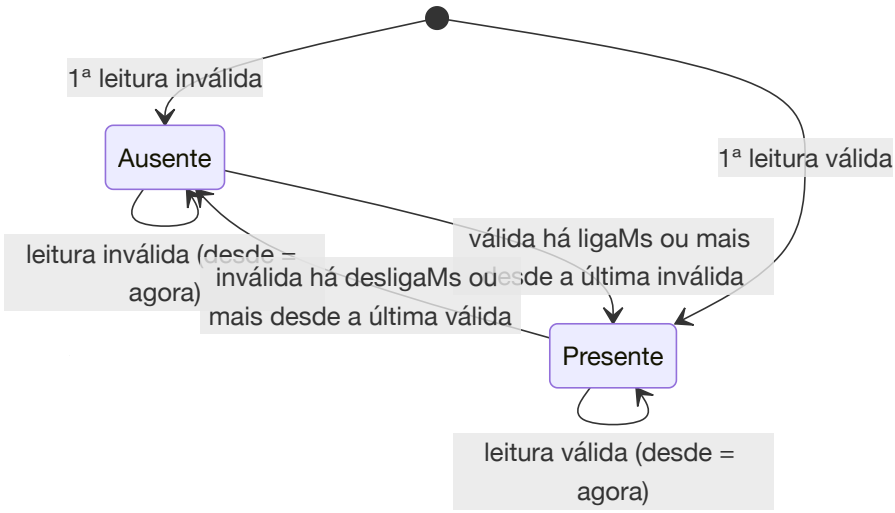
- Estado atual (com persistência).

`bool leituraAdcValida(float media, int amplitude, int minimo, int maximo, int ruidoMax)`

- **O que faz:** `minimo <= media <= maximo` **e** `amplitude <= ruidoMax`.
- **Chamada por:** `sensores_iniciar` e `sensores_atualizar` (`sensors.cpp`) com 100, 4000 e 300.
- **Observações:** o que cada falha indica, no divisor 3V3 → flex → pino → 10 kΩ → GND:

Leitura	Causa provável
~0	flex solto ou quebrado na base (pino puxado para GND pelo resistor)
~4095	falta o resistor de 10 kΩ (pino puxado para 3V3 pelo flex)
média ok, amplitude alta	mau contato ou pino flutuando

Máquina de estados da `Presenca` (`desde` = instante da última amostra que concordou com o estado):



Armadilhas e cuidados

- **Detectado não é o mesmo que ativo.** `Presenca` só diz se o sensor parece ligado. Para entrar na detecção de postura, o flex também precisa de modo `auto`/`on` e de uma referência válida gravada na calibração (`sensorAtivo` em `main.cpp`). Ver 05.
- **`ADC_RUIDO_MAX` e Wi-Fi:** o ruído real do ADC com o Wi-Fi ligado ainda precisa ser medido no hardware (`VALIDACAO.md`, V3.2). Se o flex alternar entre ativo e inativo, subir o limite.
- **`amplitude()` percorre a janela inteira:** barato com N = 10; se `ADC_MEDIA_N` crescer muito, considerar outra estrutura.
- Testes que cobrem o módulo: `test_media_movel`, `test_presenca_com_persistencia`, `test_leitura_adc_valida`.

Módulo `mpu`

> Código: `src/mpu.h`, `src/mpu.cpp` · Relacionados: 03 — Inclinação, sensores, tilt

Papel no sistema

Driver mínimo do MPU-6050 (acelerômetro + giroscópio de 3 eixos, I2C), escrito direto nos registradores. É a única parte do firmware que fala I2C com o sensor. Entrega aceleração em **g** e velocidade angular em **°/s** já sem o offset do giroscópio.

Por que um driver próprio em vez da biblioteca `MPU6050_light` (decisão registrada em 10):

- o `calcOffsets()` da biblioteca supõe o sensor deitado; no colete ele fica em pé;
- o endereço 0x68 é fixo; aqui o sensor é procurado em 0x68 e 0x69;
- o ângulo da biblioteca salta perto da vertical (o firmware usa o vetor gravidade, ver tilt);
- o texto do TCC fixa as escalas ±250 °/s e ±2 g: aqui elas são configuradas explicitamente.

Dependências

- Usa: `Wire.h` (I2C), `Arduino.h` (`delay`, `min`, `max`), `config.h` (`MPU_ADDR_A/B`, `GIRO_PARADO_DPS`, `GIRO_OFFSET_MAX`), `tilt.h` (`Vec3`).
- É usado por: `sensors.cpp` (único usuário).
- Roda em: tarefa de controle (ESP32). `calibrarGiro` roda só no `setup()`. No simulador, `sim/Wire.h` + `sim/sim_main.cpp` imitam o MPU nos mesmos registradores.

Tipos e constantes

Registradores usados (datasheet / mapa de registradores MPU-6000/6050, rev. 4.2)

Registrador	Endereço	Valor escrito	Efeito
`PWR_MGMT_1`	0x6B	`0x80`	DEVICE_RESET: volta tudo ao padrão (estado conhecido também na reconexão); o chip fica dormindo
`PWR_MGMT_1`	0x6B	`0x01`	SLEEP = 0 (acorda) e CLKSEL = 1 (PLL do giroscópio X, mais estável que o oscilador interno)
`SMPLRT_DIV`	0x19	`0x00`	taxa interna = 1 kHz / (1 + 0) = 1 kHz (com o DLPF ligado)
`CONFIG`	0x1A	`0x03`	DLPF_CFG = 3: banda de ~44 Hz (acelerômetro) e ~42 Hz (giroscópio), ~5 ms de atraso
`GYRO_CONFIG`	0x1B	`0x00`	FS_SEL = 0: ±250 °/s
`ACCEL_CONFIG`	0x1C	`0x00`	AFS_SEL = 0: ±2 g
`ACCEL_CONFIG2`	0x1D	`0x03`	só família MPU-6500: filtro do acelerômetro (~44 Hz) fica neste registrador
`ACCEL_XOUT_H`	0x3B	— (leitura)	início dos 14 bytes de medida
`WHO_AM_I`	0x75	— (leitura)	identificação do chip

Escalas e sensibilidade

Grandeza	Escala	Sensibilidade (`mpu.cpp`)	Por quê
Aceleração	±2 g	16384 LSB/g (`LSB_POR_G`)	mede só a direção de 1 g: a escala mais sensível dá a melhor resolução

Grandeza	Escala	Sensibilidade (`mpu.cpp`)	Por quê
Velocidade angular	±250 °/s	131 LSB/(°/s) (`LSB_POR_DPS`)	movimento de tronco fica bem abaixo de 250 °/s

O filtro de ~44 Hz corta a vibração do motor e o ruído acima do que o corpo consegue fazer, e fica abaixo da metade da taxa de leitura do firmware (200 Hz), o que evita aliasing.

Campos da classe `Mpu6050`

Campo	Descrição
`ok_`	sensor respondendo (iniciado e sem 10 falhas seguidas)
`addr_`	endereço I2C encontrado (0x68, 0x69 ou 0)
`whoAmI_`	valor de `WHO_AM_I`
`falhas_`	leituras seguidas com erro
`offsetGiro_`	offset descontado do giroscópio (°/s)
`giroCalibrado_`	o offset já foi medido alguma vez
`janSoma_`, `janMin_`, `janMax_`, `janN_`	janela do recálculo automático do offset

Funções

`bool Mpu6050::iniciar()`

- **O que faz:** procura o sensor em 0x68 (`MPU_ADDR_A`, AD0 solto/GND) e depois em 0x69 (`MPU_ADDR_B`, AD0 em 3V3), reseta, configura os registradores da tabela e faz uma leitura de teste.
- **Retorno:** `true` se achou, configurou e leu.
- **Chamada por:** `iniciarMpu` em `sensors.cpp` (boot e reconexão a cada `MPU_REINICIO_MS` = 2 s).
- **Observações:** bloqueia ~150 ms (100 ms depois do reset + 50 ms para estabilizar). O `WHO_AM_I` é só informativo (aparece no `diag`): 0x68 = MPU-6050 original; 0x70, 0x71, 0x73, 0x74 = família MPU-6500 (6500, 9250, 9255...), que é compatível nesses registradores mas tem o filtro do acelerômetro no 0x1D. Outros valores = clones; o firmware segue usando.

`bool Mpu6050::ler(Vec3* acc, Vec3* gyro)`

- **O que faz:** leitura normal (200 Hz): lê, conta falhas, alimenta o recálculo do offset e desconta o offset.
- **Parâmetros:** saídas `acc` (g) e `gyro` (°/s, sem offset).
- **Retorno:** `false` se a leitura falhou ou se o sensor já está marcado como caído.
- **Chamada por:** `sensores_atualizar` e `iniciarMpu` (`sensors.cpp`).
- **Observações:** uma falha isolada é tolerada; **10 seguidas** marcam `ok_ = false` (fio solto). Daí em diante `sensors.cpp` tenta `iniciar()` a cada 2 s. O recálculo do offset recebe o valor **bruto** (antes do desconto).

`bool Mpu6050::calibrarGiro()`

- **O que faz:** mede o offset do giroscópio no boot: 200 leituras a cada 5 ms (~1 s, bloqueante).
- **Retorno:** `true` se gravou o offset. Recusa se mais da metade das leituras falhar ou se algum eixo variar mais que `GIRO_PARADO_DPS` (5 °/s) — a placa mexeu e o offset sairia errado. Nesse caso fica sem offset; o filtro complementar corrige a deriva de qualquer jeito e o recálculo automático mede depois.
- **Chamada por:** `iniciarMpu(true)` em `sensores_iniciar` (só no `setup()`, antes da tarefa de controle existir). Na reconexão não é chamada, para não travar a tarefa de controle por 1 s.

`bool ok() const` / `bool giroCalibrado() const` / `uint8_t endereco() const` / `uint8_t whoAmI() const`

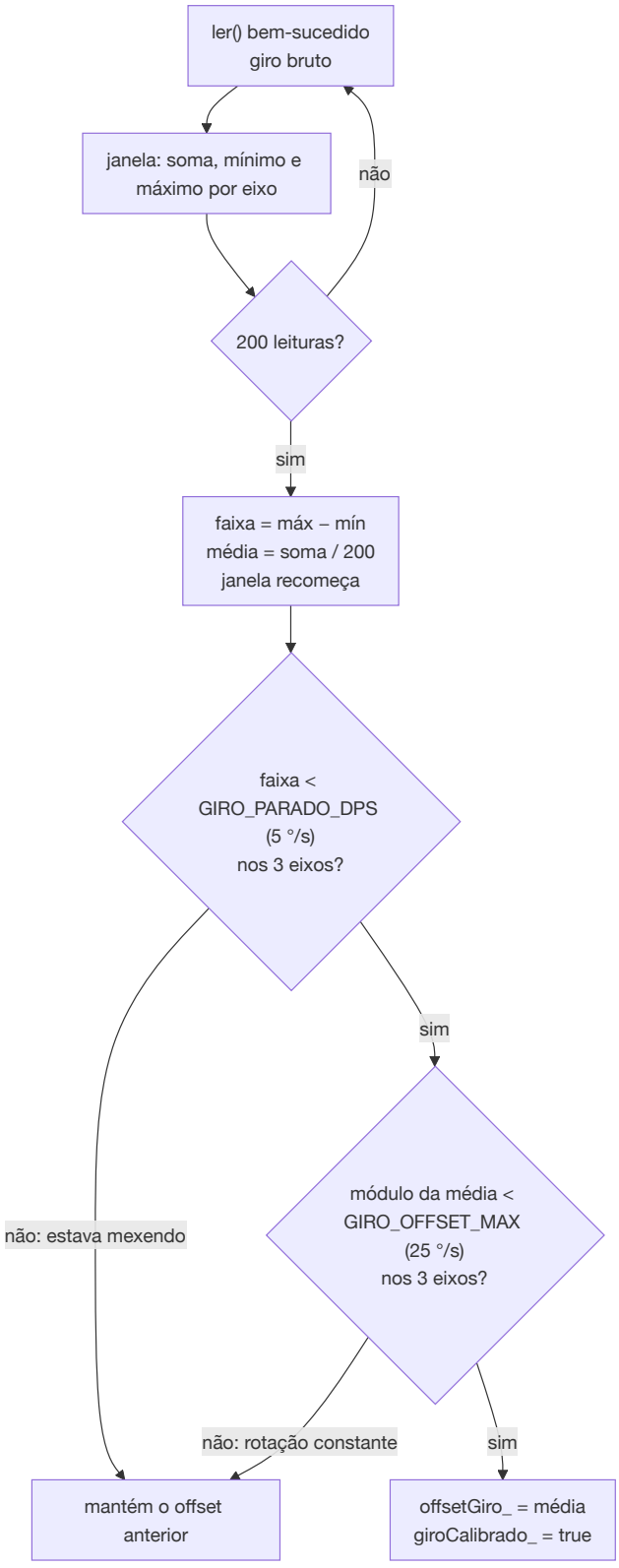
- Acesso aos campos `ok_`, `giroCalibrado_`, `addr_`, `whoAmI_`.

Privadas

Função	O que faz
`escrever(reg, valor)`	escreve um registrador (`[endereço][reg][valor]`); `true` com ACK
`lerBytes(reg, buf, n)`	escolhe o registrador e lê `n` bytes seguidos com *repeated start* (`endTransmission(false)`), sem soltar o barramento; o MPU incrementa o ponteiro sozinho
`lerBruto(acc, gyro)`	lê 14 bytes a partir de 0x3B: acelerômetro X/Y/Z, temperatura (ignorada), giroscópio X/Y/Z. Cada medida é `int16_t` big-endian (byte alto primeiro, complemento de 2); divide pela sensibilidade
`acumularOffset(giroBruto)`	janela do recálculo automático (abaixo)
`responde(addr)` (estática do .cpp)	transmissão vazia: `true` se algum dispositivo dá ACK no endereço

Recálculo automático do offset (`acumularOffset`)

O giroscópio parado não lê zero: cada eixo tem um offset de alguns °/s que muda com a temperatura. Integrado pelo filtro, ele viraria deriva do ângulo. A medição do boot falha se o colete for ligado já vestido (a pessoa se mexe). Por isso o offset é recalculado sozinho a cada janela de 200 leituras (~1 s a 200 Hz) em que o sensor ficou parado:



Armadilhas e cuidados

- **Nunca rodou no hardware antes da validação** (`VALIDACAO.md`, F2): registradores, escala, sinal dos eixos e I2C a 100 kHz são os maiores riscos. Se o desvio de uma inclinação de 30° der 15° ou 60°, conferir `LSB_POR_G`, `LSB_POR_DPS` e os valores de `GYRO_CONFIG`/`ACCEL_CONFIG`.
- **Escalas e sensibilidades andam juntas.** Mudar `GYRO_CONFIG`/`ACCEL_CONFIG` exige mudar `LSB_POR_DPS`/`LSB_POR_G` (e o texto do TCC, ver 11).
- **`calibrarGiro` é bloqueante** (~1 s): não chamar da tarefa de controle em funcionamento.

- **Recálculo absorve rotação lenta e constante:** uma rotação contínua abaixo de 5 °/s de variação e 25 °/s de média durante 1 s seria tomada como offset naquela janela. O acelerômetro corrige essa deriva em menos de um segundo (filtro complementar), então o efeito é pequeno.
- **I2C compartilhado:** o comando `diag` faz a varredura do barramento (`sensores_scanI2C`) a partir do loop, enquanto a tarefa de controle lê o MPU. Isso depende da trava interna do `Wire` do core Arduino-ESP32.
- **Simulador:** o MPU falso devolve `WHO_AM_I` 0x68 e usa a mesma convenção de eixos do código; ele não valida sinais nem escalas do chip real.

Módulo `sensors`

> Código: `src/sensors.h`, `src/sensors.cpp` · Relacionados: 03 — Inclinação, 05 — Sensores analógicos, mpu, tilt, filters, main

Papel no sistema

Junta todo o hardware de entrada num único struct `Leituras`: a gravidade filtrada do MPU-6050, os dois sensores flex e o FSR (sensor de pressão). Cada sensor é lido no seu próprio período, sem `delay()`, e cada um é **detectado** (presente ou não), para o colete seguir funcionando com o que estiver ligado:

- MPU caído → tenta reconectar a cada 2 s; enquanto isso, só o flex pode alertar;
- flex solto, quebrado ou sem resistor → deixa de ser detectado; no modo `auto` (padrão) sai da decisão e o colete segue só com o MPU;
- flex 2 e FSR → opcionais (atendem a montagem BioSpine V3, ver 01).

Dependências

- Usa: `mpu.h` (`Mpu6050`), `tilt.h` (`TiltFilter`, `Vec3`), `filters.h` (`MediaMovel`, `Presenca`, `leituraAdcValida`), `config.h` (pinos, períodos, limiares), `Wire.h`, `Arduino.h` (`analogRead`, `millis`, `micros`).
- É usado por: `main.cpp` (postura, calibração, `app_calibrar`), `console.cpp` (`diag`), simulador.
- Roda em: tarefa de controle. `sensores_iniciar` roda no `setup()`.

Tipos e constantes

`struct CanalAdc` — um canal analógico

Campo	Tipo	Descrição	Unidade
`bruto`	`int`	última leitura	ADC 0–4095
`media`	`float`	média móvel de `ADC_MEDIA_N` (10) amostras	ADC 0–4095
`amplitude`	`int`	máximo − mínimo da janela	ADC
`detectado`	`bool`	leitura plausível há tempo suficiente (`Presenca`)	—

`struct Leituras` — tudo que os sensores informam

Campo	Tipo	Descrição	Unidade
`mpuOk`	`bool`	MPU respondendo	—
`mpuEndereco`	`uint8_t`	0x68, 0x69 ou 0	—
`mpuWhoAmI`	`uint8_t`	identificação do chip	—
`giroCalibrado`	`bool`	offset do giroscópio já medido	—
`acc`	`Vec3`	aceleração (última leitura boa)	g
`gyro`	`Vec3`	velocidade angular sem offset	°/s
`g`	`Vec3`	gravidade filtrada (`TiltFilter`)	unitário
`flex`	`CanalAdc`	flex 1 (GPIO34, `PIN_FLEX`)	—
`flex2`	`CanalAdc`	flex 2 (GPIO35, `PIN_FLEX2`), opcional	—
`fsr`	`CanalAdc`	FSR (GPIO32, `PIN_FSR`), opcional, só telemetria	—

Estado interno (`sensors.cpp`)

Nome	Descrição
`mpu`, `filtro`, `L`	driver, filtro complementar (peso `MPU_FILTRO_GIRO` = 0,98) e as leituras
`mFlex`, `mFlex2`, `mFsr`	médias móveis
`pFlex`, `pFlex2`	`Presenca(FLEX_LIGA_MS, FLEX_DESLIGA_MS)` = 2 s para ligar, 1 s para desligar
`pFsr`	`Presenca(FLEX_LIGA_MS, UINT32_MAX)`: detectado uma vez, fica detectado até reiniciar
`ultimoMpu`, `ultimoAdc`, `ultimaTentativaMpu`	agendamento por `millis()`
`ultimoMpuUs`	`micros()` da última leitura do MPU: dá o `dt` real do filtro

Funções

`void sensores_iniciar()`

- **O que faz:** configura o I2C (`PIN_I2C_SDA`/`PIN_I2C_SCL`, 100 kHz = `MPU_I2C_HZ`, timeout de 20 ms) e o ADC (12 bits, atenuação de 11 dB = faixa até ~3,3 V); inicia o MPU **medindo o offset do giroscópio** (~1 s bloqueante); enche as médias móveis com 10 leituras; faz a primeira detecção.
- **Chamada por:** `setup()` em `main.cpp`, com o colete parado (o firmware avisa na serial: "Mantenha o colete parado por 2 s").
- **Observações:**
  - o flex 1 já sai do `setup` como presente se a leitura for plausível (a primeira chamada da `Presenca` adota o valor na hora);
  - o flex 2 e o FSR começam **ausentes**: o GPIO35 fica solto na montagem padrão e só passa a presente depois de `FLEX_LIGA_MS` de leitura estável.

`void sensores_atualizar(uint32_t agoraMs)`

- **O que faz:** um passo de leitura; cada sensor só é lido quando o seu período venceu.
- **Parâmetros:** `agoraMs` — `millis()`.
- **Chamada por:** `passoControle` em `main.cpp` (a cada passo da tarefa de controle, ~1 ms).
- **Observações:**
  - **MPU (200 Hz, `PERIODO_MPU_MS` = 5 ms):** `Mpu6050::ler` → `dt` medido com `micros()` → `TiltFilter::update` → `L.g`. Leitura com falha mantém os valores anteriores; depois de 10 falhas seguidas `mpu.ok()` vira `false`.
  - **MPU caído:** nova tentativa de `iniciar()` a cada `MPU_REINICIO_MS` (2 s), **sem** medir o giro (não bloqueia a tarefa de controle; o offset anterior continua valendo e o recálculo automático do `mpu.cpp` cuida do resto). O filtro recomeça pela direção do acelerômetro.
  - **ADC (100 Hz, `PERIODO_ADC_MS` = 10 ms):** lê flex 1, flex 2 e FSR, nessa ordem; atualiza as médias e a detecção.
  - **Flex:** presente se `leituraAdcValida(média, amplitude, 100, 4000, 300)` valer por 2 s seguidos; ausente depois de 1 s inválido.
  - **FSR:** critério próprio, porque sem pressão ele lê ~0 (igual a pino solto): presente quando lê pressão estável (`media > FSR_LIMIAR` = 200 e amplitude ≤ 300) que **não** seja o fantasma do ADC.
  - **Fantasma do ADC:** um pino analógico solto fica com a tensão que sobrou no capacitor de amostragem do canal lido antes. O FSR é lido depois do flex 2, que solto também copia o flex 1; por isso uma leitura do FSR a menos de `ADC_FANTASMA_DIF` (300) da do flex 1 é descartada. É o mesmo motivo de o flex 2 vir desligado por padrão (`configPadrao` em `settings.cpp`).

`const Leituras& sensores()`

- **Retorno:** referência para as últimas leituras. Não guardar entre passos.
- **Chamada por:** `main.cpp` (postura, calibração, `app_calibrar`), `console.cpp` (`diag`).

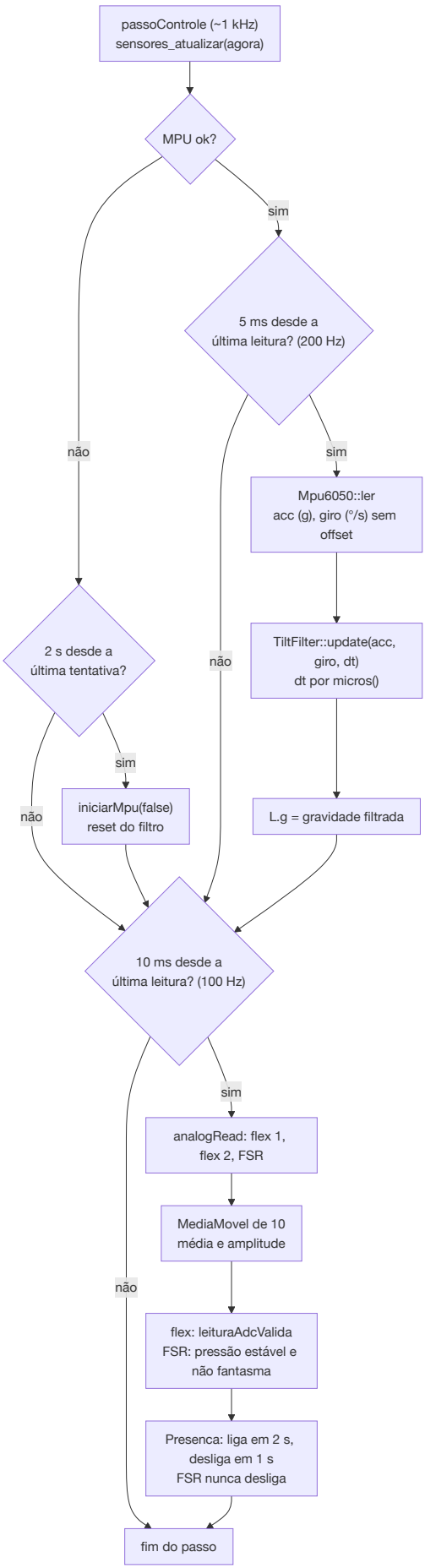
`int sensores_scanI2C(uint8_t* encontrados, int maximo)`

- **O que faz:** varre os endereços 0x01 a 0x7E com transmissão vazia; ACK = tem dispositivo.
- **Retorno:** quantos achou (até `maximo`).
- **Chamada por:** `diag` em `console.cpp` (loop).

Estáticas do `.cpp`

Função	O que faz
`iniciarMpu(bool medirGiro)`	`mpu.iniciar()`, copia endereço/`WHO_AM_I`, mede o offset se `medirGiro`, reinicia o filtro pela primeira leitura e marca `ultimoMpuUs`
`lerAdc(pino, media, canal)`	`analogRead` → `push` na média → atualiza `bruto`, `media`, `amplitude`

Pipeline de um passo de `sensores_atualizar`:


passoControle (~1 kHz)
sensores_atualizar(agora)
MPU ok?
sim
não
5 ms desde a
última leitura? (200 Hz)
sim
não
Mpu6050::ler
acc (g), giro (°/s) sem
offset
TiltFilter::update(acc,
giro, dt)
dt por micros()
L.g = gravidade filtrada
2 s desde a
última tentativa?
sim
não
iniciarMpu(false)
reset do filtro
10 ms desde a
última leitura? (100 Hz)
sim
não
analogRead: flex 1,
flex 2, FSR
MediaMovel de 10
média e amplitude
flex: leituraAdcValida
FSR: pressão estável e
não fantasma
Presenca: liga em 2 s,
desliga em 1 s
FSR nunca desliga
fim do passo

Armadilhas e cuidados

- **Sem trava nas leituras:** só a tarefa de controle escreve em `L`; o loop lê `sensores()` direto (`diag`, `app_calibrar`). Para mostrar e para checar `mpuOk` isso basta, mas uma leitura do loop pode pegar o struct no meio de uma atualização. Para decisões, usar o `Status` protegido de `main.cpp` (`app_status`).
- **Reconexão do MPU bloqueia ~150 ms** (reset do chip em `Mpu6050::iniciar`) dentro da tarefa de controle. Aceitável a cada 2 s com o sensor caído.
- **Ordem de leitura do ADC importa** para o fantasma (flex 1 → flex 2 → FSR). Mudar a ordem muda qual canal o pino solto copia.
- **ADC2 não funciona com Wi-Fi ligado:** todos os pinos analógicos estão no ADC1 (GPIO 32–39). Não mover sensor analógico para fora dessa faixa.
- **Atenuação de 11 dB:** o ADC do ESP32 não é linear perto das pontas da faixa. Como a curvatura é relativa à calibração (`flexPct`), isso não afeta a detecção.

Módulo `motor`

> Código: `src/motor.h`, `src/motor.cpp` · Relacionados: 06 — Motor e NVS, main, config

Papel no sistema

Controla a única saída do colete: o **motor vibratório**, por PWM, com o **LED azul da placa** acendendo junto (dá para ver o alerta mesmo sem motor ligado). Dois usos convivem:

Uso	Padrão	Quem pede
Alerta	300 ms ligado / 700 ms desligado, enquanto a postura estiver em alerta	`atualizarPostura` (`main.cpp`)
Sequência	pulsos curtos de aviso	`setup`, calibração, teste do motor (`main.cpp`)

A **sequência tem prioridade**: se começa durante o alerta, o alerta espera ela terminar e depois retoma. Tudo sem `delay()`: cada fase guarda o instante em que começou e `motor_atualizar` troca de fase quando a duração passa.

Circuito: GPIO25 (`PIN_MOTOR`) → 1 kΩ → base do NPN (2N2222/BC337); o transistor liga o motor (3 V, ~60 mA, mais do que um GPIO aguenta), com diodo de flyback em paralelo. Detalhes em `montagem.pdf`.

Dependências

- Usa: `Arduino.h` (LEDC, `digitalWrite`, `millis`), `config.h` (pinos, `MOTOR_*`, `PADRAO_INTENSIDADE`).
- É usado por: `main.cpp` (único usuário).
- Roda em: tarefa de controle (e no `setup()`, antes dela existir). O loop **não** chama o motor direto: pede o teste com `app_testarMotor` (pedido executado pela tarefa de controle).

Tipos e constantes

Constantes de `config.h`:

Constante	Valor	Uso
`MOTOR_LEDC_CANAL`	0	canal do LEDC (API do core 2.x)
`MOTOR_LEDC_FREQ`	20 000 Hz	frequência do PWM: acima da faixa audível, o motor não "chia"
`MOTOR_LEDC_BITS`	8	resolução do duty (0–255, passos de ~0,4 %)
`MOTOR_ALERTA_ON_MS` / `MOTOR_ALERTA_OFF_MS`	300 / 700 ms	padrão do alerta (pulsado incomoda menos que contínuo e gasta menos bateria)
`MOTOR_TESTE_MS`	1000 ms	teste do motor (`motor`, `diag`, botão do app)
`MOTOR_CONFIRMA_MS`	120 ms	pulso curto de confirmação
`MOTOR_ERRO_MS`	600 ms	pulso longo: calibração recusada

Estado interno (`motor.cpp`):

Nome	Descrição
`alertaAtivo`	postura em alerta (último valor de `motor_alerta`)
`intensidade`	potência em %
`saida`	motor ligado neste instante
`marco`	`millis()` do início da fase atual

Nome	Descrição
`seqRestantes`	fases restantes da sequência (cada pulso = 2 fases: ligado e desligado)
`seqOn`, `seqOff`	duração das fases da sequência (ms)

Padrões de pulso usados pelo firmware:

Situação	Chamada (em `main.cpp`)	O que a pessoa sente
Ligou	`motor_pulsos(1, 120, 0)`	1 pulso curto
Calibração começou	`motor_pulsos(1, 120, 0)`	1 pulso curto: "fique parado"
Calibração concluída	`motor_pulsos(2, 120, 120)`	2 pulsos curtos
Calibração recusada (mexeu)	`motor_pulsos(1, 600, 0)`	1 pulso longo
Teste do motor	`motor_pulsos(1, 1000, 0)`	1 s ligado
Alerta	`motor_alerta(true)`	300 ms ligado / 700 ms desligado, repetindo

Funções

`void motor_iniciar()`

- **O que faz:** configura o LED como saída e o LEDC (20 kHz, 8 bits) no `PIN_MOTOR`, com o motor desligado.
- **Chamada por:** `setup()`.
- **Observações:** o core Arduino-ESP32 3.x mudou a API do LEDC (por pino: `ledcAttach`/`ledcWrite(pino, ...)`) em relação ao 2.x (por canal: `ledcSetup` + `ledcAttachPin`/`ledcWrite(canal, ...)`). O projeto usa o core 2.x (`espressif32@6.9.0`), mas o código compila nos dois via `ESP_ARDUINO_VERSION_MAJOR`. No simulador a macro não existe, vale 0 e cai no ramo 2.x (que `sim/Arduino.h` imita).

`void motor_atualizar(uint32_t agoraMs)`

- **O que faz:** avança as fases. Prioridade: sequência > alerta > desligado.
- **Parâmetros:** `agoraMs` — `millis()`.
- **Chamada por:** `passoControle` (`main.cpp`), a cada passo da tarefa de controle.
- **Observações:**
  1. Sequência em andamento: quando a fase atual acaba, decrementa `seqRestantes` e alterna ligado/desligado; ao terminar a última fase, volta para o estado do alerta (`escrever(alertaAtivo)`).
  2. Sem alerta: garante o motor desligado.
  3. Alerta: alterna 300 ms ligado / 700 ms desligado.

`void motor_alerta(bool ativo)`

- **O que faz:** liga/desliga o padrão de alerta.
- **Parâmetros:** `ativo` — `estado == ESTADO_ALERTA`.
- **Chamada por:** `atualizarPostura` (`main.cpp`) a 50 Hz, sempre com o valor atual.
- **Observações:** só age quando o valor muda. Ao entrar em alerta, o motor liga **na hora** (resposta imediata). Se houver uma sequência em andamento, só registra; a sequência termina e o alerta assume.

`void motor_pulsos(uint8_t quantidade, uint16_t ligadoMs, uint16_t desligadoMs)`

- **O que faz:** começa uma sequência de `quantidade` pulsos (`ligadoMs` ligado + `desligadoMs` desligado). Começa ligado na hora.
- **Chamada por:** `setup`, `iniciarCalibracao`, `concluirCalibracao`, `processarPedidos` (`main.cpp`).
- **Observações:** `seqRestantes = 2 × quantidade` (fases). Uma sequência nova substitui a que estiver em andamento. `quantidade = 0` não faz nada.

`void motor_intensidade(uint8_t pct)`

- **O que faz:** muda a potência (duty = 255 × pct / 100). Se o motor estiver ligado, aplica na hora.
- **Chamada por:** `processarPedidos` (`main.cpp`), quando `Params::intensidade` muda.

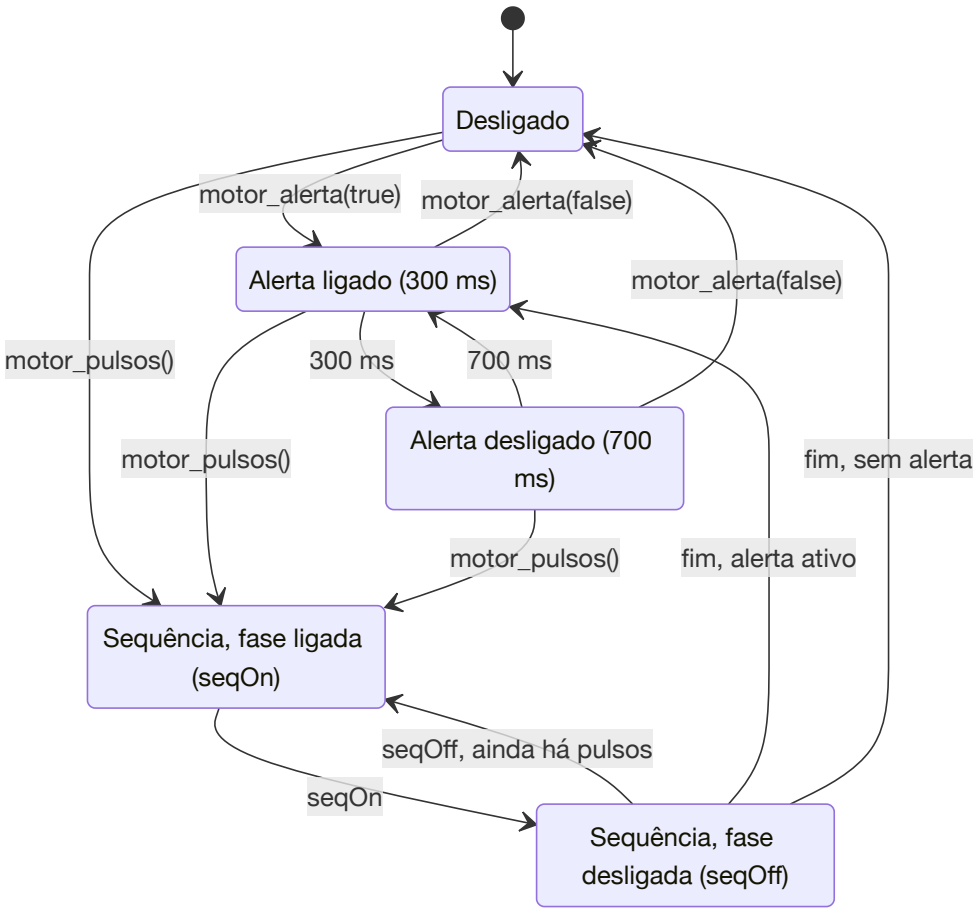
`bool motor_ligado()`

- **Retorno:** motor ligado neste instante. Vai para o `Status::motor` e para o app (campo `motor`).

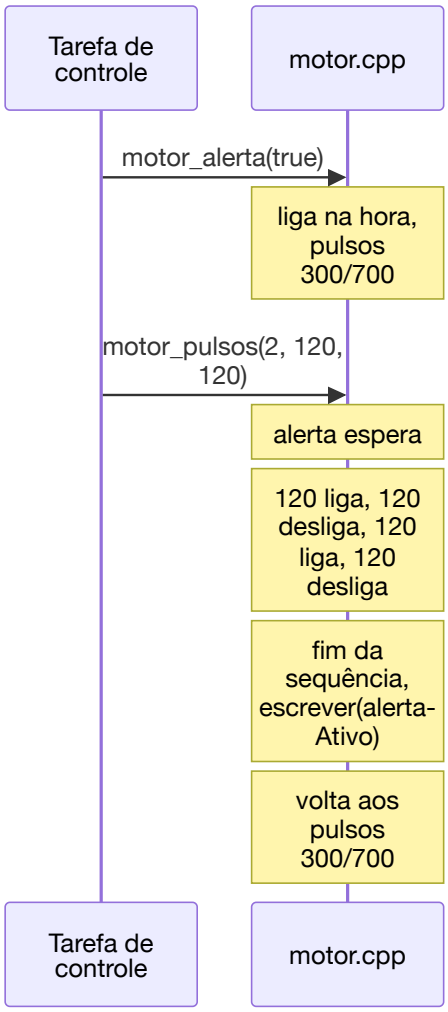
`static void escrever(bool ligado)` (privada)

- Calcula o duty com a intensidade, escreve no LEDC e copia o estado para o LED (`PIN_LED`, GPIO2).

Máquina de estados do motor (a sequência interrompe o alerta e devolve o controle a ele no fim):



Exemplo de linha do tempo: alerta ativo e, no meio dele, uma calibração concluída (`motor_pulsos(2, 120, 120)`):



Armadilhas e cuidados

- **Não usar `delay()`** aqui: bloquearia a tarefa de controle (sensores e postura param).
- **20 kHz:** alguns transistores ou motores respondem mal a PWM rápido. Se a 100 % vibrar fraco ou fizer ruído, testar `MOTOR_LEDC_FREQ` = 1000 (`VALIDACAO.md`, V4.3).
- **Reset quando o motor liga:** o pico de corrente pode derrubar a tensão do ESP32. Mitigações no hardware (capacitor de 100 µF, power bank bom) e no firmware (potência do Wi-Fi reduzida em `web.cpp`).
- **`motor_alerta` e `motor_pulsos` usam `millis()` internamente**, não o `agoraMs` do passo. A diferença é de no máximo um passo (~1 ms).
- **Só a tarefa de controle chama este módulo.** Chamar do loop criaria acesso concorrente às variáveis estáticas sem trava.

Módulo `settings`

> Código: `src/settings.h`, `src/settings.cpp` · Relacionados: 06 — Motor e NVS, main, posture

Papel no sistema

Guarda a configuração e a calibração na **NVS** (*Non-Volatile Storage*), para sobreviverem a desligar a placa. O ESP32 não tem EEPROM: a NVS é uma área da flash organizada em pares chave/valor, acessada pela biblioteca `Preferences` do core Arduino. Também define os tipos `Config`, `Calibracao` e `ModoSensor`, usados pelo resto do firmware.

Este módulo **não guarda** os valores em uso: quem guarda é `main.cpp` (`cfg`, `cal`). Aqui só se lê (no boot) e grava (quando algo muda).

Dependências

- Usa: `Preferences` (core Arduino-ESP32; no simulador, `sim/Preferences.h` guarda em memória), posture (`Params`, `paramsPadrao`, `validarParams`), tilt (`Vec3`, `v3`, `norm`).
- É usado por: `main.cpp` (carregar no boot, salvar config e calibração), `console.cpp` e `protocolo.cpp` (`configPadrao`, `modoNome`, `modoDeTexto`).
- Roda em: `settings_carregar` no `setup`; `settings_salvarConfig` no loop (via `app_definirConfig`); `settings_salvarCalibracao` na tarefa de controle (via `concluirCalibracao`).

Tipos e constantes

`enum ModoSensor : uint8_t`

Valor	Número	Texto	Significado
`MODO_AUTO`	0	`auto`	usa o sensor se ele for detectado (e tiver referência)
`MODO_ON`	1	`on`	força ligado (precisa de referência da calibração)
`MODO_OFF`	2	`off`	nunca usa

O número vai para a NVS: não reordenar o enum.

`struct Config`

Campo	Tipo	Descrição
`params`	`Params`	limite, histerese, tempo mínimo, limite do flex, intensidade (posture)
`flexModo`	`ModoSensor`	flex 1 (GPIO34); padrão `auto`
`flex2Modo`	`ModoSensor`	flex 2 (GPIO35); padrão `off`

`struct Calibracao`

Campo	Tipo	Unidade	Descrição
`valida`	bool	—	`false` = nunca calibrado (o colete não vibra)
`g0`	`Vec3`	unitário	vetor gravidade na postura correta (a referência do desvio)
`flex0`	float	ADC	leitura do flex 1 na postura correta (0 = sem referência)
`flex20`	float	ADC	leitura do flex 2 na postura correta (0 = sem referência)

Chaves na NVS (namespace `colete`)

Chave	Tipo	Campo	Padrão (chave ausente)
`limite`	float	`params.limiteGraus`	`PADRAO_LIMITE_GRAUS` (20)
`hist`	float	`params.histerese`	`PADRAO_HISTERESE` (3)
`tempo`	uint32	`params.tempoMs`	`PADRAO_TEMPO_MS` (3000)
`flexlim`	float	`params.limiteFlex`	`PADRAO_LIMITE_FLEX` (15)
`motor`	uint8	`params.intensidade`	`PADRAO_INTENSIDADE` (100)
`flexmodo`	uint8	`flexModo`	0 (`auto`)
`flex2modo`	uint8	`flex2Modo`	2 (`off`)
`cal_ok`	bool	`cal.valida`	`false`
`g0x`, `g0y`, `g0z`	float	`cal.g0`	(0, 1, 0)
`flex0`	float	`cal.flex0`	0
`flex20`	float	`cal.flex20`	0

A NVS aceita nomes de chave de no máximo 15 caracteres; por isso os nomes são curtos.

Funções

`const char* modoNome(ModoSensor m)`

- **O que faz:** `MODO_ON` → `"on"`, `MODO_OFF` → `"off"`, qualquer outro → `"auto"`.
- **Chamada por:** `console.cpp` (`get`), `protocolo.cpp` (`{"t":"config"}`).

`bool modoDeTexto(const char* txt, ModoSensor* m)`

- **O que faz:** converte texto em modo. Aceita `on`/`1`/`true`, `off`/`0`/`false` e `auto` (minúsculas; a serial já converte para minúsculas antes).
- **Retorno:** `false` se o texto não for reconhecido; nesse caso `*m` não é alterado.
- **Chamada por:** `console.cpp` (`set flex ...`), `protocolo.cpp` (`flexModo`/`flex2Modo` do `set`).

`Config configPadrao()`

- **O que faz:** valores de fábrica: `paramsPadrao()`, flex 1 `auto`, flex 2 `off`.
- **Por que o flex 2 vem desligado:** um GPIO35 solto "copia" a tensão do GPIO34 pelo capacitor de amostragem do ADC e pareceria um sensor de verdade. O 2º flex só entra se for ligado de propósito (`set flex2 auto` ou `on`) e depois de recalibrar.
- **Chamada por:** `settings_carregar` (padrões), `console.cpp` (`reset`), `protocolo.cpp` (`reset`).

`void settings_carregar(Config* c, Calibracao* cal)`

- **O que faz:** lê todas as chaves (só leitura). Chave ausente = padrão. Depois corrige o que vier estranho da flash:
  - `validarParams` nos parâmetros (mesmas faixas da serial/app);
  - modo fora de 0–2 vira `auto`;
  - calibração com `|g0| < 0,5` é descartada (`valida = false`).
- **Chamada por:** `setup` (uma vez, no boot).

`void settings_salvarConfig(const Config& c)`

- **O que faz:** grava as 7 chaves da configuração (todas, mesmo as que não mudaram).

- **Chamada por:** `app_definirConfig` (loop).

`void settings_salvarCalibracao(const Calibracao& cal)`

- **O que faz:** grava as 6 chaves da calibração. Não mexe na configuração.
- **Chamada por:** `concluirCalibracao` (tarefa de controle).

Armadilhas e cuidados

- **Renomear uma chave = perder o valor salvo** nas placas já gravadas (volta ao padrão).
- **`reset` não apaga a calibração:** são chaves diferentes; `reset` só regrava a configuração.
- **Modo inválido na flash vira `auto`, inclusive no flex 2** (cujo padrão é `off`). Só acontece com dado corrompido, mas nesse caso o GPIO35 solto poderia ser detectado como flex; ele ainda precisaria de uma referência válida da calibração para entrar na detecção.
- Para zerar tudo (inclusive a calibração), é preciso apagar a flash (`pio run -t erase`) e gravar de novo.
- No simulador a NVS vive só na memória do processo.

Módulo `console`

> Código: `src/console.h`, `src/console.cpp` · Relacionados: 07 — Protocolo e interfaces, main, settings

Papel no sistema

Interface pela porta serial (USB, 115200 baud): interpretador de comandos (`diag`, `calib`, `get`, `set`, `motor`, `plot`, `reset`, `help`), compatibilidade com os comandos do BioSpine V3 (`START`, `STOP`, `STATUS`, `CALIBRATE`, respostas em JSON) e as saídas contínuas (Serial Plotter e JSON a 1 Hz).

É a ferramenta de bancada: o `diag` existe para quem monta o hardware descobrir, sem mexer em código, qual fio está errado. A regra combinada é "se o `diag` dá ❌, o problema é fio, não código".

Dependências

- Usa: API `app_*` (main), `sensores()` e `sensores_scanI2C` (sensors), `configPadrao`, `modoNome`, `modoDeTexto` (settings), `web_ativo`, `web_ip`, `web_clientes` (web), config.
- É usado por: `main.cpp` (`console_iniciar`, `console_atualizar`, `console_enviarDados`, `console_evento`, `console_respostaJson`).
- Roda em: **loop** (nunca na tarefa de controle).

Tipos e constantes

Variável	Tipo	Descrição
`linha[256]`	char[]	linha sendo digitada, até o Enter
`tam`	uint8_t	caracteres guardados em `linha`
`plot`	bool	`plot on` ativo: imprime dados a 10 Hz
`streamJson`	bool	`START` ativo: imprime o JSON do BioSpine a 1 Hz
`ultimoJson`	uint32_t	`millis()` do último JSON contínuo
`SIM`, `NAO`	const char*	✅ e ❌ em UTF-8 escritos byte a byte

Funções

`void console_iniciar()`

- **O que faz:** imprime a dica de uso e, se não houver calibração, o lembrete ("vista o colete... aperte o botão").
- **Chamada por:** `setup`.

`void console_atualizar(uint32_t agoraMs)`

- **O que faz:** lê os caracteres disponíveis **sem bloquear** e, a cada `\r` ou `\n`, executa a linha (`executar`). Linha vazia é ignorada (aceita `\r\n`). Mais de 255 caracteres: o excesso é descartado.
- **Por quê sem bloquear:** `Serial.readStringUntil` travaria o loop até o timeout (1 s), atrasando o Wi-Fi.
- **Chamada por:** `loop`, a cada volta.

`void console_enviarDados(const Status& s, uint32_t agoraMs)`

- **O que faz:** saídas contínuas, chamada a 10 Hz pelo loop.
  - `plot: desvio:..,limite:..,pitch:..,roll:..,flexPct:..,limiteFlex:..,alerta:..` (formato do Serial Plotter do Arduino e do `tools/grafico_serial.py`). `alerta` vale 5 ou 0 para aparecer como degrau visível na mesma escala dos graus.

    - `streamJson`: uma linha JSON do BioSpine por segundo.
- **Chamada por:** `loop`.

`void console_evento(const char* tag, const char* msg)`

- **O que faz:** imprime `[tag] msg`.
- **Chamada por:** `drenarEventos` (eventos da tarefa de controle) e `app_calibrar`.

`void console_respostaJson(bool ok, const char* msg)`

- **O que faz:** imprime `{"success":true|false,"message":"..."}` (formato do BioSpine V3).
- **Chamada por:** `executar` (`START`, `STOP`, `CALIBRATE` sem MPU) e `drenarEventos` (`EVT_JSON_CALIB_OK`: "Calibration completed").

`static void executar(char* cmd)` — interpretador

1. Compara a linha **exata** com `START`, `STOP`, `STATUS`, `CALIBRATE` (maiúsculos, BioSpine V3).
2. Converte o resto para minúsculas e separa em até 3 palavras (`strtok` por espaço): comando, `a1`, `a2`.
3. Despacha: `help`/`ajuda`/`?`, `diag`, `calib`/`calibrar`, `get`, `set`, `motor`, `plot`, `status`, `reset`; senão "comando desconhecido".

- **Observação:** modifica a própria string (`tolower` e `strtok`).

`static void comandoSet(char* nome, char* valor)`

- **O que faz:** `set <nome> <valor>`: copia a config atual, troca um campo e chama `app_definirConfig` (que valida, salva na NVS e avisa o app). Depois imprime "salvo." e a config (`mostrarConfig`).
- **Conversão:** `atof` (texto não numérico vira 0 e depois é preso na faixa por `validarParams`); `tempo` negativo vira 0; `motor` preso em 0–100; `flex`/`flex2` aceitam `on`/`off`/`auto`.

`static void mostrarConfig()`

- **O que faz:** comando `get`: parâmetros, modo e situação de cada flex (`descreverSensor`) e calibração.

`static const char* descreverSensor(ModoSensor modo, bool detectado, bool ativo)`

- **O que faz:** texto da situação de um flex: `desligado`, `ativo`, `não detectado` ou `detectado, recalibre para usar` (ligado depois da calibração, ainda sem referência).

`static void diag()`

- **O que faz:** testa cada componente e diz o que conferir quando falha. Linhas, em ordem: I2C scan, MPU (ângulos, endereço, `WHO_AM_I`, offset do giroscópio), flex 1 (com o motivo provável da falha pela faixa da leitura), flex 2, FSR, botão, motor (pulsa 1 s), Wi-Fi AP (IP e clientes), calibração.
- **Observação:** lê `sensores()` sem trava (diagnóstico; um valor de um passo atrás não importa).

`static void jsonBioSpine(const Status& s)`

- **O que faz:** imprime `{"angle":..,"flex_left":..,"flex_right":..,"pressure":..,"status":"GOOD|WARNING|BAD","alert":..,"calibrated":..}`. `angle` = desvio, `flex_left` = flex 1, `flex_right` = flex 2, `pressure` = FSR, `status` = ok/atenção/alerta (indexado pelo valor do enum `Estado`).

`static void ajuda()`

- **O que faz:** imprime a lista de comandos (`help`).

Armadilhas e cuidados

- **`nomes[s.estado]` em `jsonBioSpine`** depende da ordem do enum `Estado` (0 = ok, 1 = atenção, 2 = alerta). Mudar o enum quebra o JSON do BioSpine.
- **Comandos do BioSpine só em maiúsculas exatas.** `start`, `stop` e `calibrate` minúsculos caem no interpretador normal e dão "comando desconhecido" (`status` minúsculo funciona).
- **`CALIBRATE` recusado por movimento não gera JSON** de falha: só a linha `[calib] recusada...`. Quem espera a resposta JSON do BioSpine fica sem ela nesse caso.
- **`plot` com qualquer argumento diferente de `off` liga** (`plot`, `plot on`, `plot x`).
- Texto novo com acento deve continuar em UTF-8 (o monitor serial do PlatformIO mostra certo).

Módulo `protocolo`

> Código: `src/protocolo.h`, `src/protocolo.cpp` · Relacionados: 07 — Protocolo e interfaces, web, 08 — App web

Papel no sistema

Monta e interpreta o JSON trocado com o app web pelo WebSocket. Fica separado de `web.cpp` de propósito: não depende do Wi-Fi, então o simulador no PC (`sim/sim_main.cpp`) usa **exatamente este código**, e o `tools/sim_web.py` liga a página real (`src/web/index.html`) a ele. Assim o protocolo é testado sem placa.

A definição resumida das mensagens fica no comentário do topo de `src/protocolo.h`; as tabelas completas, com tipos e unidades, estão em 07 — Protocolo e interfaces.

Dependências

- Usa: ArduinoJson 7 (`JsonDocument`, `serializeJson`, `deserializeJson`), API `app_*` (main), `estadoNome` (posture), `modoNome`, `modoDeTexto`, `configPadrao` (settings), `web_evento` (web), `FIRMWARE_VERSAO` (config).
- É usado por: `web.cpp` (ESP32) e `sim/sim_main.cpp` (PC).
- Roda em: loop.

Funções

`size_t protocolo_dados(const Status& s, uint32_t uptimeMs, char* out, size_t tam)`

- **O que faz:** escreve `{"t":"data",...}` em `out` com um campo por membro de `Status` (mesmos nomes) e `uptime`.
- **Parâmetros:** `s` — foto do colete; `uptimeMs` — `millis()` do envio; `out`/`tam` — buffer de saída.
- **Retorno:** bytes escritos.
- **Observações:** ângulos e porcentagens são arredondados para 1 casa (`r1`): menos bytes no Wi-Fi e no CSV que o app grava. No pior caso a mensagem tem ~360 bytes; `web.cpp` usa um buffer de 640.
- **Chamada por:** `web_enviarDados` (10 Hz) e o simulador.

`size_t protocolo_config(char* out, size_t tam)`

- **O que faz:** escreve `{"t":"config",...}` com os parâmetros, os modos dos flex, `calibrado` e `versao`. Lê com `app_config()` e `app_calibracao()`.
- **Chamada por:** `web_enviarConfig` (para todos) e `aoEventoWs` (só para quem conectou ou pediu `get`).

`size_t protocolo_evento(const char* nivel, const char* msg, char* out, size_t tam)`

- **O que faz:** escreve `{"t":"evt","nivel":...,"msg":...}`. O app mostra como aviso (toast).
- **Chamada por:** `web_evento`.

`bool protocolo_comando(const char* payload, size_t len)`

- **O que faz:** interpreta um comando do app e executa:

`cmd`	Ação	Resposta
`calib`	`app_calibrar(ORIGEM_APP)`	eventos da calibração, depois config
`motor_test`	`app_testarMotor()`	—
`get`	—	retorna `true`: `web.cpp` manda o config só para quem pediu
`set`	`aplicarSet`	config para todos + evento "Parâmetros salvos"

cmd	Ação	Resposta
`reset`	`app_definirConfig(configPadrao())`	config para todos + evento "Parâmetros padrão restaurados"
outro / sem `cmd`	—	evento de erro "comando desconhecido"
JSON inválido	—	evento de erro "comando inválido"

- **Retorno:** `true` só para `get`.
- **Chamada por:** `aoEventoWs` (texto recebido no WebSocket) e o simulador (`!ws {...}`).

`static void aplicarSet(JsonDocument& d)`

- **O que faz:** parte da config atual e troca só os campos presentes **com o tipo certo** (`limiteGraus`, `histerese`, `tempoMs`, `limiteFlex`, `intensidade` numéricos; `flexModo`, `flex2Modo` texto). O app pode mandar só o que mudou. Depois chama `app_definirConfig`, que prende os valores nas faixas, salva na NVS e envia o config para todos.
- **Observações:** `is<float>()` aceita inteiros também. `tempoMs` negativo vira 0; `intensidade` é presa em 0–100. Texto inválido em `flexModo`/`flex2Modo` é ignorado sem aviso de erro.

`static float r1(float v)`

- **O que faz:** arredonda para 1 casa decimal.

Armadilhas e cuidados

- **Mudou um campo?** Atualize juntos: este arquivo, o comentário do topo de `protocolo.h`, `src/web/index.html` (quem lê), `tools/mock_server.py` (imitação em Python) e 07.
- **Tamanho:** se `{"t":"data"}` crescer muito, confira o `buf[640]` de `web.cpp` e o `wsBuf[640]` de `sim/sim_main.cpp`; o config (~180 bytes no pior caso) usa um buffer de 384 bytes em `aoEventoWs`.
- **"Parâmetros salvos" sai mesmo se nada mudou** (por exemplo, só um `flexModo` com texto inválido).
- `protocolo_config` liga uma referência `const Config&` ao valor temporário devolvido por `app_config()`; é válido em C++ (a vida do temporário é estendida), mas parece um erro à primeira vista.

Módulo `web`

> Código: `src/web.h`, `src/web.cpp` · Relacionados: 07 — Protocolo e interfaces, protocolo, 08 — App web

Papel no sistema

Transporte do app web: cria o **Access Point** Wi-Fi do colete, serve a página embutida na flash (HTTP, porta 80) e mantém o **WebSocket** (porta 81) por onde passam dados, configuração, avisos e comandos. O conteúdo das mensagens é responsabilidade de protocolo.

Por que Access Point e página embutida: o colete não depende de internet, roteador nem computador da faculdade. O celular conecta direto na rede do colete e abre `http://192.168.4.1`. A página não usa CDN (o celular está sem internet nessa rede).

Bibliotecas: `WebServer` (já vem no core Arduino-ESP32) + `WebSocketsServer` (links2004, versão fixada no `platformio.ini`). A `ESPAsyncWebServer` foi evitada por ter forks e versões confusas.

Dependências

- Usa: `WiFi`, `WebServer`, `WebSocketsServer`, protocolo, config (`WIFI_SSID`, `WIFI_SENHA`, `HTTP_PORTA`, `WS_PORTA`).
- É usado por: `main.cpp` (`web_iniciar`, `web_atualizar`, `web_enviarDados`, `web_enviarConfig`, `web_evento`), `protocolo.cpp` (`web_evento`), `console.cpp` (`web_ativo`, `web_ip`, `web_clientes`).
- Roda em: **loop**. Não é compilado no simulador: `sim/sim_main.cpp` define versões próprias das funções `web_*` que imprimem o JSON no terminal.

Tipos e constantes

Nome	Tipo	Descrição
`index_html`, `manifest_json`, `icon_svg`	`const char[]`	arquivos de `src/web/` embutidos pelo `board_build.embed_txtfiles` (terminam em `'\0'`)
`http`	`WebServer`	servidor HTTP na porta 80
`ws`	`WebSocketsServer`	servidor WebSocket na porta 81
`ativo`	bool	AP no ar e servidores iniciados
`ip[16]`	char[]	IP do AP em texto (`"-"` se não subiu)
`buf[640]`	char[]	mensagem de saída (dados, config, evento); só o loop usa

Os arquivos embutidos viram símbolos do linker com o caminho no nome: `_binary_src_web_index_html_start` etc. Por isso **renomear ou mover um arquivo de `src/web/` exige mudar o `platformio.ini` e as declarações `extern` daqui**.

Rotas HTTP

Método e caminho	Resposta	Tipo
`GET /`	`index.html`	`text/html; charset=utf-8`
`GET /index.html`	`index.html`	`text/html; charset=utf-8`
`GET /manifest.json`	`manifest.json`	`application/manifest+json`
`GET /icon.svg`	`icon.svg`	`image/svg+xml`
qualquer outro	`302` para `http://<ip>/`	—

As quatro rotas servidas levam `Cache-Control: no-cache` (o 302 do `onNotFound` não): depois de gravar um firmware novo, o celular não fica com a página antiga. O manifest e o ícone permitem "Adicionar à tela inicial" (a página abre como app).

Funções

`void web_iniciar()`

- **O que faz:** `WiFi.persistent(false)` (não grava a config do Wi-Fi na flash) → modo AP → `softAP(WIFI_SSID, WIFI_SENHA)` → potência 11 dBm → rádio sem economia de energia → registra as rotas → inicia HTTP e WebSocket → heartbeat → imprime `[wifi] AP "BioSpine" ... -> abra http://192.168.4.1`.
- **Se o AP falhar:** imprime `[wifi] falha ao criar o Access Point` e retorna com `ativo = false`. O colete continua detectando e vibrando; só fica sem app.
- **Por que 11 dBm:** menos potência = menor pico de corrente (ajuda a não resetar a placa quando o motor liga junto) e ainda sobra alcance para uma sala.
- **Por que `setSleep(false)`:** sem economia de energia no rádio, os dados chegam ao app com menos atraso.
- **Chamada por:** `setup`.

`void web_atualizar()`

- **O que faz:** `http.handleClient()` e `ws.loop()`. É dentro de `ws.loop()` que `aoEventoWs` é chamado.
- **Chamada por:** `loop`, a cada volta.

`void web_enviarDados(const Status& s)`

- **O que faz:** `{"t":"data"}` para todos os clientes. Sem cliente conectado, nem monta o JSON.
- **Chamada por:** `loop`, a cada `PERIODO_ENVIO_MS` (10 Hz).

`void web_enviarConfig()`

- **O que faz:** `{"t":"config"}` para todos os clientes.
- **Chamada por:** `app_definirConfig` e `drenarEventos` (eventos com `EVT_CONFIG`, como o fim da calibração).

`void web_evento(const char* nivel, const char* msg)`

- **O que faz:** `{"t":"evt"}` para todos os clientes.
- **Chamada por:** `drenarEventos`, `app_calibrar`, `protocolo_comando`.

`static void aoEventoWs(uint8_t cliente, WStype_t tipo, uint8_t* payload, size_t len)`

- **O que faz:** callback da biblioteca. Na conexão (`WStype_CONNECTED`), ou num texto que `protocolo_comando` responda com `true` (`get`), envia o config **só para aquele cliente**. Texto = comando do app. Outros tipos (desconexão, ping/pong, binário) são ignorados.

`static void servir(const char* tipo, const char* conteudo)`

- **O que faz:** responde `200` com um arquivo embutido e o cabeçalho `no-cache`.

`bool web_ativo()` · `const char* web_ip()` · `int web_clientes()`

- **O que fazem:** AP no ar?; IP em texto; quantos clientes no WebSocket (usados pelo `diag`).

Heartbeat

`ws.enableHeartbeat(5000, 2000, 2)`: ping a cada 5 s; se o pong não chegar em 2 s, duas vezes seguidas, o cliente é desconectado. Sem isso, um celular que saiu da rede continuaria contando como conectado e receberia envios que só travam o loop.

Armadilhas e cuidados

- **A página vai inteira para a flash e para o celular a cada carga** (~85 KB, sem compressão). Comentários e código no `index.html` custam flash e tempo de carga.
- **`softAP` aceita poucos clientes ao mesmo tempo** (o padrão do ESP32 é 4). Cada cliente recebe os dados a 10 Hz.
- **Não existe servidor DNS:** só funciona quem digitar o IP. O redirecionamento `302` só pega caminhos errados no próprio IP (não é um "captive portal" completo).
- **Android "sem internet":** é preciso tocar em "Manter conexão" e, se a página não carregar, desligar os dados móveis (o Android manda o tráfego pelo 4G).
- **Nunca chamar `web_*` da tarefa de controle**: `broadcastTXT` pode bloquear quando um cliente some.

Manutenção da documentação

> Para quem (pessoa ou agente de IA) for mudar o código depois. Leia antes de editar `src/`, `sim/`, `test/` ou `tools/`.

A documentação mora junto do código, no mesmo repositório, para mudar no mesmo commit que ele. Regra única: **mudou o comportamento, mudou a documentação no mesmo commit.**

Fonte e saída. Os `.md` desta pasta são a **fonte**: é neles que se edita. Os **PDFs** em `pdf/` são a **saída** (para ler, imprimir e entregar) e são gerados a partir dos `.md` por `python3 Documentação/gerar_pdf.py`. Nunca edite um PDF; edite o `.md` e gere de novo, no mesmo commit.

---

1. Onde cada arquivo de código está documentado

Ao alterar um arquivo da coluna da esquerda, revise os documentos da direita.

Arquivo de código	Referência (detalhe por função)	Capítulos (lógica e intenção)
`src/config.h`	referencia/config.md	01 (pinagem), 03, 04, 05, 06 (constantes)
`src/app.h`, `src/main.cpp`	referencia/main.md	02, 04
`src/tilt.h`, `src/tilt.cpp`	referencia/tilt.md	03
`src/posture.h`, `src/posture.cpp`	referencia/posture.md	04
`src/filters.h`	referencia/filters.md	05
`src/mpu.h`, `src/mpu.cpp`	referencia/mpu.md	03
`src/sensors.h`, `src/sensors.cpp`	referencia/sensors.md	03, 05
`src/motor.h`, `src/motor.cpp`	referencia/motor.md	06
`src/settings.h`, `src/settings.cpp`	referencia/settings.md	06
`src/console.h`, `src/console.cpp`	referencia/console.md	07
`src/protocolo.h`, `src/protocolo.cpp`	referencia/protocolo.md	07
`src/web.h`, `src/web.cpp`	referencia/web.md	07
`src/web/index.html`, `manifest.json`, `icon.svg`	—	08
`sim/*`	—	09
`test/test_posture/test_main.cpp`	—	09
`tools/*`	—	09
`platformio.ini`	—	01, 09

Mudou uma **decisão de projeto** (algo listado em 10)? Atualize a decisão ou acrescente uma nova; não apague a antiga, marque como "substituída por D-xx". Mudou algo que o **texto do TCC** afirma (escalas do MPU, filtro, limites)? Atualize 11.

Checklist ao terminar uma mudança

- □ Comentários do arquivo alterado continuam verdadeiros (cabeçalho, funções, blocos).
- □ A página de `referencia/` do módulo bate com as assinaturas atuais (nome, parâmetros, retorno, quem chama).
- □ Capítulos da tabela acima: textos, tabelas de valores e **diagramas** continuam verdadeiros.

- □ Constante nova ou alterada em `config.h`: tabela de referencia/config.md.
- □ Campo novo no JSON ou comando novo: 07 e o comentário do topo de `src/protocolo.h`.
- □ Teste novo: lista de testes em 09.
- □ Rodar a verificação (seção 4).
- □ Mexeu em algum `.md` desta pasta: gerar os PDFs de novo (`python3 Documentação/gerar_pdf.py`) e commitar `pdf/` junto.

---

2. Convenções da documentação

- **Idioma:** português do Brasil, frases curtas, no mesmo tom do `README.md`. Sempre explicar o **porquê**, não só o quê.
- **Arquivos:** nomes em ASCII, minúsculos, com hífen. Capítulos numerados (`NN-assunto.md`); referência por módulo em `referencia/<modulo>.md` (o nome do módulo é o nome do arquivo `.cpp`/`.h`).
- **Documento novo:** acrescentar o caminho na lista `ORDEM` de `ferramentas/gerar_pdf.mjs` (define a ordem no PDF completo e no sumário; o gerador avisa se um `.md` ficou de fora) e uma linha no README.md.
- **Links:** sempre relativos entre os `.md` (`[03](03-algoritmo-inclinacao.md#5-filtro-complementar-sobre-o-vetor-gravidade)`). No PDF completo viram links internos; o gerador para com erro se a âncora não existir. Link para arquivo fora desta pasta (código, `../README.md`) aparece como texto no PDF.
- **Cabeçalho de todo documento:**

  ```
  # Título

  > Código: `src/x.h`, `src/x.cpp` · Relacionados: [03 — Inclinação](03-algoritmo-inclinacao.md)
  ```

  A linha `> Código:` é o elo com o código: um agente que muda `src/x.cpp` encontra o documento com `grep -rl "src/x.cpp" Documentação/`.
- **Citar código por nome, nunca por número de linha** (`TiltFilter::update` em `src/tilt.cpp`). Número de linha muda a cada edição.
- **Unidades sempre explícitas:** °, °/s, g, ms, Hz, %, leitura do ADC (0–4095).
- **Valores numéricos** (limites, períodos, tempos) citam o nome da constante: "20° (`PADRAO_LIMITE_GRAUS`)". Assim dá para achar e atualizar.
- **Termos** (usar sempre os mesmos):

Termo	Significado	No código
desvio	ângulo total entre a gravidade atual e a de referência	`Status::desvio`, `angleBetweenDeg`
frente/trás, lateral	decomposição do desvio, só para exibição	`pitch`, `roll`, `tiltComponentsDeg`
referência (g0)	vetor gravidade gravado na calibração	`Calibracao::g0`
limite	desvio máximo aceito	`Params::limiteGraus`
histerese	margem abaixo do limite para sair do alerta	`Params::histerese`
tempo mínimo	tempo em má postura antes de vibrar	`Params::tempoMs`
curvatura do flex	% de queda da leitura em relação à calibração	`flexPct`
estados	ok, atenção, alerta	`ESTADO_OK`, `ESTADO_ATENCAO`, `ESTADO_ALERTA`
tarefa de controle	tarefa FreeRTOS de sensores, postura e motor	`tarefaControle` em `main.cpp`
loop	`loop()` do Arduino: Wi-Fi e serial	`loop` em `main.cpp`
pedido / evento	mensagens loop → controle / controle → loop	`pedidoCalibracao`, `emitir`

Página de referência de módulo (`referencia/<modulo>.md`)

```
# Módulo `tilt`

> Código: `src/tilt.h`, `src/tilt.cpp` · Relacionados: [03 — Inclinação](../03-algoritmo-inclinacao.md)

## Papel no sistema
O que o módulo faz e por que existe (2–5 frases).

## Dependências
- Usa: ...
- É usado por: ...
- Roda em: tarefa de controle | loop | ambos | PC (testes/simulador)

## Tipos e constantes
Tabela: nome | tipo | descrição | unidade.

## Funções
### `float angleBetweenDeg(Vec3 a, Vec3 b)`
- **O que faz:** ...
- **Parâmetros:** `a`, `b` — ...
- **Retorno:** ...
- **Chamada por:** ...
- **Observações:** detalhes, casos de borda, por que foi feita assim.

## Armadilhas e cuidados
O que pode quebrar se alguém mudar sem cuidado.
```

Diagramas

- Só **Mermaid** dentro do próprio `.md` (bloco ```` ```mermaid ````). É texto: qualquer agente edita junto com o código, e o GitHub e o VS Code desenham.
- Tipos usados: `flowchart`, `sequenceDiagram`, `stateDiagram-v2`. Nada de `%%{init}` nem temas.
- Rótulo com espaço, acento ou símbolo vai **entre aspas**: `A["Filtro complementar<br/>0,98 giro"]`. Quebra de linha no rótulo: `<br/>`. Evite `;` e `#` nos rótulos; em `sequenceDiagram`, evite `;` nas mensagens.
- Uma frase antes de cada diagrama dizendo o que ele mostra.
- No PDF (A4 em pé) o gerador escolhe o tamanho sozinho: um `flowchart LR` muito largo e baixo é desenhado de cima para baixo, e o que ainda ficar largo demais vai sozinho numa página deitada. Se ele avisar "letra pequena no papel", divida ou simplifique o diagrama.
- Figuras para o texto do TCC: ver README.md.

---

3. Convenções dos comentários no código

- Comentários em **português**, estilo `//` (C/C++/JS/CSS usam também `/* */` onde já é o costume do arquivo). Python: docstring no módulo e nas funções, `#` nos blocos. Sem Doxygen.
- **Cabeçalho de arquivo:** o que o arquivo é, o papel no sistema, em que tarefa roda e onde está documentado (`// Documentação: Documentação/referencia/tilt.md`).
- **Toda função, classe, struct e campo de struct** tem comentário: o que faz, unidades dos parâmetros e do retorno, efeitos colaterais e, quando importa, de qual tarefa pode ser chamada.
- **Blocos não óbvios** recebem o **porquê** (fórmula, decisão, armadilha de hardware). Não comentar o óbvio (`i++ // incrementa i`).
- Comentário é parte do código: mudou a lógica, revise o comentário em cima dela.
- Cuidados específicos:

- `src/config.h`: o `VALIDACAO.md` cita linhas deste arquivo (`src/config.h:25`...). Não inserir nem remover linhas antes da linha 66; acrescentar comentário só no fim das linhas existentes ou depois da seção do Wi-Fi.
- `src/web/index.html` vai **inteiro** para a flash e é enviado ao celular a cada carga: comentários curtos. Nunca escrever `</script>` dentro de comentário JS nem `--` dentro de comentário HTML.
- `tools/roteiro*.txt`: comentário é linha começando com `#` **na coluna 0** (o simulador manda para a serial qualquer outra linha, inclusive `# texto`).
- `src/web/manifest.json` é JSON: não aceita comentário.

---

4. Verificação depois de mudar

Com o PlatformIO instalado (PC de desenvolvimento):

```
cd colete-postural
pio test -e native     # testes da lógica pura
tools/sim.sh           # simulador: conferir a saída dos roteiros
pio run                # compila para o ESP32
```

Sem o PlatformIO (só para conferir sintaxe):

```
g++ -std=c++11 -Wall -fsyntax-only -Isrc src/tilt.cpp src/posture.cpp
g++ -std=c++17 -fsyntax-only -DSIMULADOR -Isim -Isrc src/main.cpp src/console.cpp src/sensors.cpp
\
    src/motor.cpp src/settings.cpp src/mpu.cpp sim/sim_main.cpp
python3 -m py_compile tools/*.py
bash -n tools/*.sh
```

Depois de mexer em qualquer `.md` desta pasta (Node.js e Chrome instalados):

```
python3 Documentação/gerar_pdf.py    # gera Documentação/pdf/; termina com erro se um diagrama não
desenhar
                                     # ou um link apontar para âncora que não existe
```

O comando lista os problemas (diagrama com erro de sintaxe, link quebrado) e avisa se um `.md` ficou fora da lista `ORDEM` ou se um diagrama ficou com letra pequena. Confira também, abrindo o PDF, as páginas que você mudou.